

Table of Contents

Introductory	2
Sorting and Searching	11
Dynamic Programming	31
Graph Algorithms	44
Range Queries	78
Tree Algorithms	101
Mathematics	115
String Algorithms	140
Geometry	162
Advanced Techniques	178
Sliding Window Problems	209
Bitwise Operations	217
Construction Problems	221
Advanced Graph Problems	224
Counting Problems	239
Additional Problems I	248
Additional Problems II	262

Introductory

Two Knights

```
#include

using namespace std;
typedef long long ll;

int N;

int main(){
    scanf("%d", &N);
    for(int k = 1; k <= N; k++){
        ll cnt = 1LL + (k-1)*(k-2)/2;
        cnt = cnt * (k-1) * (k+4);
        printf("%lld\n", cnt);
    }
}
```

Trailing Zeros

```
#include

using namespace std;

int N, ans;

int main(){
    scanf("%d", &N);
    while(N > 0){
        N /= 5;
        ans += N;
    }
    printf("%d\n", ans);
}
```

Gray Code

```
#include

using namespace std;
const int maxN = 16;

int N;
bool b[maxN+1];
```

```
void print(){
    for(int i = N; i > 0; i--){
        printf("%d", b[i]);
        printf("\n");
    }

    int main(){
        scanf("%d", &N);

        print();
        for(int i = 1; i < (1<
```

Tower of Hanoi

```
#include

using namespace std;

int N;

void move(int from, int to, int depth){
    if(depth == 1){
        printf("%d %d\n", from, to);
        return;
    }

    int other = 6 - from - to;
    move(from, other, depth-1);
    printf("%d %d\n", from, to);
    move(other, to, depth-1);
}

int main(){
    scanf("%d", &N);
    printf("%d\n", (1<
```

Creating Strings

```
#include

using namespace std;

int N;
char S[9];
set perms;

int main(){
    scanf("%s", S);
    N = (int) strlen(S);
    sort(S, S+N);
```

```
perms.insert(S);
while(next_permutation(S, S+N))
    perms.insert(string(S, S+N));

cout << perms.size() << '\n';
for(string perm : perms)
    cout << perm << '\n';
}
```

Apple Division

```
#include

using namespace std;
typedef long long ll;
const ll INF = 0x3f3f3f3f3f3f3f;

int N, p[20];
ll a, b, best;

int main() {
    scanf("%d", &N);
    for (int i = 0; i < N; i++)
        scanf("%d", &p[i]);

    best = INF;
    for (int i = 0; i < (1 << N); i++) {
        a = 0;
        b = 0;
        for (int j = 0; j < N; j++) {
            if (i & (1 << j))
                a += p[j];
            else
                b += p[j];
        }
        best = min(best, abs(a - b));
    }
    printf("%lld\n", best);
}
```

Chessboard and Queens

```
#include

using namespace std;
const int N = 8;

char c;
int ans;
bool b[N][N], input[N][N];

bool place(int x, int y){
```

```

bool yes = true;
for(int i = 0; i < N; i++)
    if(b[x][i] || b[i][y])
        yes = false;
for(int i = 0; x-i >= 0 && y-i >= 0; i++)
    if(b[x-i][y-i])
        yes = false;
for(int i = 0; x-i >= 0 && y+i < N; i++)
    if(b[x-i][y+i])
        yes = false;
return yes;
}

bool check(){
    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            if(b[i][j] && input[i][j])
                return false;
    return true;
}

void dfs(int i){
    if(i == N){
        if(check())
            ans++;
        return;
    }
    for(int j = 0; j < N; j++){
        if(place(i, j)){
            b[i][j] = true;
            dfs(i+1);
            b[i][j] = false;
        }
    }
}

int main(){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            scanf(" %c", &c);
            input[i][j] = (c == '*');
        }
    }
    dfs(0);
    printf("%d\n", ans);
}

```

Raab Game I

```

#include

using namespace std;

int T;

int main(){
    scanf("%d", &T);
    for(int t = 0, n, a, b; t < T; t++){
        scanf("%d %d %d\n", &n, &a, &b);

        // Degenerate case: all ties
    }
}

```

```

if(a == 0 && b == 0){
    printf("YES\n");
    for(int i = 0; i < 2; i++)
        for(int j = 1; j <= n; j++)
            printf("%d%c", j, (" \n")[j==n]);
    continue;
}

// Otherwise, each player must win at least once
// and the total wins cannot exceed n
if(a + b > n || a == 0 || b == 0){
    printf("NO\n");
    continue;
}

// General case: trade wins then handle ties
printf("YES\n");
int ties = n - a - b;
vector xs(n+1);

// Start with both players playing 1, 2, 3, ...
// Cyclic shift a player by k to the right
// in order to give them k wins
// E.g. n = 5, shift b by k = 2
// as = 1 2 3 4 5 (3 wins)
// bs = 4 5 1 2 3 (2 wins)
for(int i = 1; i <= a+b; i++){
    int pos = (i + b - 1) % (a + b) + 1;
    xs[pos] = i;
}
for(int i = a+b+1; i <= n; i++)
    xs[i] = i;

// Print out solution
for(int i = 1; i <= n; i++)
    printf("%d%c", i, (" \n")[i==n]);
for(int i = 1; i <= n; i++)
    printf("%d%c", xs[i], (" \n")[i==n]);
}
}

```

Mex Grid Construction

```

#include

using namespace std;

int N;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            printf("%d%c", i ^ j, (" \n")[j==N-1]);
}

```

Knight Moves Grid

```
#include

using namespace std;
typedef pair pii;
const int maxN = 1e3+5;

const int M = 8;
const int h[M] = {2, 2, 1, -1, -2, -2, -1, 1};
const int v[M] = {-1, 1, 2, 2, 1, -1, -2, -2};

int N, dist[maxN][maxN];
bool vis[maxN][maxN];
queue Q;

bool in_bounds(int x, int y){
    return 0 <= x && x < N && 0 <= y && y < N;
}

int main(){
    scanf("%d", &N);

    Q.push({0, 0});
    vis[0][0] = true;
    while(!Q.empty()){
        pii P = Q.front();
        Q.pop();

        int x = P.first, y = P.second;
        for(int i = 0; i < M; i++){
            int dx = x + h[i], dy = y + v[i];
            if(in_bounds(dx, dy) && !vis[dx][dy]){
                dist[dx][dy] = dist[x][y] + 1;
                vis[dx][dy] = true;
                Q.push({dx, dy});
            }
        }
    }

    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            printf("%d%c", dist[i][j], (" \n")[j==N-1]);
}
```

Grid Coloring I

```
#include

using namespace std;

int N, M;

const char cs[2][2] = {{ 'A', 'B' }, { 'C', 'D' }};

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++){
```

```
    for(int j = 0; j < M; j++){
        char c;
        scanf(" %c", &c);

        bool odd = (i+j)&1;
        bool small = c < 'C';
        printf("%c", cs[small][odd]);
    }
    printf("\n");
}
```

Digit Queries

```
#include

using namespace std;
typedef long long ll;

int Q;
ll K;

ll pow10(int x){
    ll res = 1;
    for(int i = 0; i < x; i++)
        res *= 10;
    return res;
}

int solve(ll N){
    if(N < 9)    return (int) N+1;

    int len = 1;
    while(9 * pow10(len-1) * len < N){
        N -= 9 * pow10(len-1) * len;
        len++;
    }

    string S = to_string(pow10(len-1) + N/len);
    return (int) (S[N%len] - '0');
}

int main(){
    scanf("%d", &Q);
    for(int q = 0; q < Q; q++){
        scanf("%lld", &K);
        printf("%d\n", solve(K-1));
    }
}
```

String Reorder

```

#include

using namespace std;
const int maxN = 1e6+5;

int freq[26];
char S[maxN], ans[maxN];

bool possible(int current){
    int mode = 0, total = 0;
    for(int c = 0; c < 26; c++){
        if(freq[mode] < freq[c])
            mode = c;
        total += freq[c];
    }
    return (freq[mode] <= (total+1)/2) && (freq[current] <= total/2);
}

int main(){
    scanf("%s", S);
    const int N = (int) strlen(S);
    for(int i = 0; i < N; i++)
        freq[(int) (S[i] - 'A')]++;

    int lastchar = -1;
    for(int i = 0; i < N; i++){
        bool filled = false;
        for(int c = 0; c < 26 && !filled; c++){
            if(freq[c] > 0 && c != lastchar){
                freq[c]--;
                if(possible(c)){
                    ans[i] = (char) (c + 'A');
                    lastchar = c;
                    filled = true;
                } else freq[c]++;
            }
        }

        if(!filled){
            printf("-1\n");
            return 0;
        }
    }

    for(int i = 0; i < N; i++)
        printf("%c", ans[i]);
    printf("\n");
}

```

Grid Path Description

```

#include

using namespace std;
const int N = 7;

int ans;
char c[N*N+1];
bool vis[N+1][N+1];

```

```
bool inbounds(int x, int y){
    return 1 <= x && x <= N && 1 <= y && y <= N;
}

void dfs(int x, int y, int i){
    if(i == N*N-1 || (x == N && y == 1)){
        ans += (i == N*N-1 && (x == N && y == 1));
        return;
    }

    if((!inbounds(x+1, y) || vis[x+1][y]) && (!inbounds(x-1, y) || vis[x-1][y]))
        if(inbounds(x, y-1) && !vis[x][y-1] && inbounds(x, y+1) && !vis[x][y+1])
            return;
    if((!inbounds(x, y+1) || vis[x][y+1]) && (!inbounds(x, y-1) || vis[x][y-1]))
        if(inbounds(x+1, y) && !vis[x+1][y] && inbounds(x-1, y) && !vis[x-1][y])
            return;

    vis[x][y] = true;
    if(c[i] == 'D' || c[i] == '?')
        if(inbounds(x+1, y) && !vis[x+1][y])
            dfs(x+1, y, i+1);
    if(c[i] == 'U' || c[i] == '?')
        if(inbounds(x-1, y) && !vis[x-1][y])
            dfs(x-1, y, i+1);
    if(c[i] == 'R' || c[i] == '?')
        if(inbounds(x, y+1) && !vis[x][y+1])
            dfs(x, y+1, i+1);
    if(c[i] == 'L' || c[i] == '?')
        if(inbounds(x, y-1) && !vis[x][y-1])
            dfs(x, y-1, i+1);
    vis[x][y] = false;
}

int main(){
    scanf("%s", c);
    dfs(1, 1, 0);
    printf("%d\n", ans);
}
```

Sorting

Concert Tickets

```
#include

using namespace std;

int N, M, h, t;
multiset prices;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++){
        scanf("%d", &h);
        prices.insert(-h);
    }
    for(int i = 0; i < M; i++){
        scanf("%d", &t);
        if(prices.lower_bound(-t) == prices.end())
            printf("-1\n");
        else {
            printf("%d\n", -(*prices.lower_bound(-t)));
            prices.erase(prices.lower_bound(-t));
        }
    }
}
```

Restaurant Customers

```
#include

using namespace std;
const int maxN = 4e5;

int N, a, b, cur, ans;
struct event {int time, type;} events[maxN];

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d %d", &a, &b);
        events[2*i] = {a, 1};
        events[2*i+1] = {b, -1};
    }
    sort(events, events+2*N, [](event A, event B){
        return A.time == B.time ? A.type < B.type : A.time < B.time;
    });

    for(int i = 0; i < 2*N; i++){
        cur += events[i].type;
        ans = max(ans, cur);
    }
}
```

```
    printf("%d\n", ans);  
}
```

Movie Festival

```
#include  
  
using namespace std;  
typedef pair pii;  
const int maxN = 2e5+1;  
  
int N, a[maxN], b[maxN], dp[2*maxN];  
struct Movie {int t, id, type;} movies[2*maxN];  
map mp;  
  
int main(){  
    scanf("%d", &N);  
    for(int i = 0; i < N; i++){  
        scanf("%d %d", &a[i], &b[i]);  
        movies[2*i] = {a[i], i, 0};  
        movies[2*i+1] = {b[i], i, 1};  
    }  
  
    sort(movies, movies+2*N, [](Movie A, Movie B){  
        return A.t == B.t ? A.id < B.id : A.t < B.t;  
    });  
  
    for(int i = 0; i < 2*N; i++){  
        mp[movies[i].t] = i;  
    }  
  
    for(int i = 0; i < 2*N; i++){  
        if(movies[i].type == 0) dp[i] = dp[i-1];  
        else dp[i] = max(dp[i-1], dp[mp[a[movies[i].id]]]+1);  
    }  
    printf("%d\n", dp[2*N-1]);  
}
```

Sum of Two Values

```
#include  
  
using namespace std;  
typedef pair pii;  
const int maxN = 2e5+1;  
  
int N, x, a;  
pii p[maxN];  
  
int find(int val){  
    int l = 1, r = N;  
    while(l <= r){  
        int m = l+(r-l)/2;
```

```

        if(p[m].first == val)        return p[m].second;
        else if(p[m].first < val)    l = m+1;
        else                        r = m-1;
    }
    return 0;
}

int main(){
    scanf("%d %d", &N, &x);
    for(int i = 1; i <= N; i++){
        scanf("%d", &a);
        p[i] = {a, i};
    }
    sort(p+1, p+N+1);

    for(int i = 1; i <= N; i++){
        int other = find(x-p[i].first);
        if(other != 0 && other != p[i].second){
            printf("%d %d\n", p[i].second, other);
            return 0;
        }
    }
    printf("IMPOSSIBLE\n");
}

```

Maximum Subarray Sum

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;
const ll INF = 0x3f3f3f3f3f3f3f3f;

int N;
ll maxSum, curSum, x[maxN];

int main(){
    scanf("%d", &N);
    maxSum = -INF;
    for(int i = 0; i < N; i++){
        scanf("%lld", &x[i]);
        maxSum = max(maxSum, x[i]);
    }

    for(int i = 0; i < N; i++){
        curSum += x[i];
        maxSum = max(maxSum, curSum);
        if(curSum < 0) curSum = 0;
    }
    printf("%lld\n", maxSum);
}

```

Stick Lengths

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, p[maxN];
ll median, sum;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%d", &p[i]);
    sort(p, p+N);

    median = p[(N-1)/2];
    for(int i = 0; i < N; i++)
        sum += abs(p[i]-median);
    printf("%lld\n", sum);
}
```

Missing Coin Sum

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, x[maxN];
ll res;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%d", &x[i]);
    sort(x, x+N);

    res = 1;
    for(int i = 0; i < N && x[i] <= res; i++)
        res += x[i];
    printf("%lld\n", res);
}
```

Collecting Numbers

```
#include
```

```

using namespace std;
const int maxN = 2e5+1;

int N, x, cnt, pos[maxN];

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x);
        pos[x] = i;
    }

    cnt = 1;
    for(int i = 2; i <= N; i++)
        if(pos[i-1] > pos[i])
            cnt++;
    printf("%d\n", cnt);
}

```

Collecting Numbers II

```

#include

using namespace std;
const int maxN = 2e5+5;

int N, M, cnt, x[maxN], pos[maxN];

void update(int a, int b){
    if(pos[x[a]-1] <= pos[x[a]] && b < pos[x[a]-1]) cnt++;
    if(pos[x[a]-1] > pos[x[a]] && b >= pos[x[a]-1]) cnt--;
    if(pos[x[a]+1] >= pos[x[a]] && b > pos[x[a]+1]) cnt++;
    if(pos[x[a]+1] < pos[x[a]] && b <= pos[x[a]+1]) cnt--;
    pos[x[a]] = b;

    if(pos[x[b]-1] <= pos[x[b]] && a < pos[x[b]-1]) cnt++;
    if(pos[x[b]-1] > pos[x[b]] && a >= pos[x[b]-1]) cnt--;
    if(pos[x[b]+1] >= pos[x[b]] && a > pos[x[b]+1]) cnt++;
    if(pos[x[b]+1] < pos[x[b]] && a <= pos[x[b]+1]) cnt--;
    pos[x[b]] = a;

    swap(x[a], x[b]);
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x[i]);
        pos[x[i]] = i;
    }
    pos[N+1] = N+1;

    cnt = 1;
    for(int i = 1, ptr = 0; i <= N; i++){
        if(ptr > pos[i])
            cnt++;
        ptr = pos[i];
    }

    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
    }
}

```

```
        update(a, b);
        printf("%d\n", cnt);
    }
}
```

Playlist

```
#include

using namespace std;

int N, k, l, ans;
map mp;

int main(){
    scanf("%d", &N);

    l = 1;
    for(int r = 1; r <= N; r++){
        scanf("%d", &k);
        if(mp[k]){
            ans = max(ans, r-1);
            l = max(l, mp[k]+1);
            mp[k] = r;
        } else {
            ans = max(ans, r-1+1);
            mp[k] = r;
        }
    }

    ans = max(N-l+1, ans);
    printf("%d\n", ans);
}
```

Towers

```
#include

using namespace std;

int N, k;
multiset S;
multiset::iterator it;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d", &k);
        it = S.upper_bound(k);
        if(it != S.end())
            S.erase(it);
        S.insert(k);
    }
}
```

```

    }
    printf("%d\n", (int) S.size());
}

```

Traffic Lights

```

#include

using namespace std;
const int maxN = 2e5;

set S;
set::iterator f, c, b;
int N, X, a, best, ans[maxN];
struct light {int pos, id;} x[maxN+2];

int main(){
    scanf("%d %d", &X, &N);
    x[0] = {0, 0}; x[N+1] = {X, 1};
    S.insert(0); S.insert(X);
    for(int i = 0; i < N; i++){
        scanf("%d", &a);
        x[i+1] = {a, i+2};
        S.insert(a);
    }

    sort(x, x+maxN+2, [](light A, light B){
        return A.pos < B.pos;
    });
    for(int i = 1; i <= maxN+1; i++)
        best = max(best, x[i].pos - x[i-1].pos);

    sort(x, x+maxN+2, [](light A, light B){
        return A.id > B.id;
    });
    for(int i = 0; i < N; i++){
        ans[N-i-1] = best;
        c = S.find(x[i].pos);
        f = next(c, 1);
        b = next(c, -1);
        best = max(best, *f - *b);
        S.erase(c);
    }

    for(int i = 0; i < N; i++)
        printf("%d%c", ans[i], (" \n")[i==N]);
}

```

Distinct Values Subarrays

```

#include

```

```

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, r, x[maxN];
ll ans;
set S;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d", &x[i]);
    }

    for(int l = 0; l < N; l++){
        while(r < N && S.find(x[r]) == S.end()){
            S.insert(x[r]);
            r++;
        }
        ans += r - l;
        S.erase(x[l]);
    }
    printf("%lld\n", ans);
}

```

Distinct Values Subsequences

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;
const ll MOD = 1e9+7;

int N, x[maxN];
map freq;

ll mod_inv(ll a){
    ll res = 1;
    ll b = MOD-2;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%d", &x[i]);

    ll ans = 0;
    ll prod = 1;
    for(int i = 0; i < N; i++){
        ll seqs = (prod * mod_inv(freq[x[i]]+1)) % MOD;
        ans = (ans + seqs) % MOD;

        freq[x[i]]++;
    }
}

```

```

        prod = (prod * (freq[x[i]]+1) ) % MOD;
        prod = (prod * mod_inv(freq[x[i]])) % MOD;
    }
    printf("%lld\n", ans);
}

```

Josephus Problem I

```

#include
#include
#include

using namespace std;
using namespace __gnu_pbds;

int N, idx;
tree, rb_tree_tag,
tree_order_statistics_node_update> T;

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++)
        T.insert(i);

    idx = 1;
    while(T.size()){
        idx %= T.size();
        int x = *T.find_by_order(idx);
        T.erase(x);
        printf("%d%c", x, (" \n")[T.size() == 0]);
        idx++;
    }
}

```

Josephus Problem II

```

#include
#include
#include

using namespace std;
using namespace __gnu_pbds;

int N, K, idx;
tree, rb_tree_tag,
tree_order_statistics_node_update> T;

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 1; i <= N; i++)
        T.insert(i);
}

```

```

    idx = K;
    while(T.size()){
        idx %= T.size();
        int x = *T.find_by_order(idx);
        T.erase(x);
        printf("%d%c", x, (" \n")[T.size() == 0]);
        idx += K;
    }
}

```

Nested Ranges Check

```

#include

using namespace std;
typedef array triple;
const int maxN = 2e5+1;
const int SIZE = 2*maxN;

int N, ans[2][maxN], ds[SIZE];
triple intervals[maxN];
set unique_vals;
map mp;

void update(int idx, int val){
    for(int i = idx; i < SIZE; i += -i&i)    ds[i] += val;
}

int query(int idx){
    int cnt = 0;
    for(int i = idx; i > 0; i -= -i&i)
        cnt += ds[i];
    return cnt;
}

int main(){
    scanf("%d", &N);
    for(int i = 0, x, y; i < N; i++){
        scanf("%d %d", &x, &y);
        intervals[i] = {x, y, i};
        unique_vals.insert(x);
        unique_vals.insert(y);
    }
    sort(intervals, intervals+N, [](triple A, triple B){
        return (A[0] == B[0] ? A[1] > B[1] : A[0] < B[0]);
    });

    int val_id = 1;
    for(int x : unique_vals)
        mp[x] = val_id++;

    for(int i = N-1; i >= 0; i--){
        int y = mp[intervals[i][1]];
        int id = intervals[i][2];

        ans[0][id] = query(y);
        update(y, 1);
    }

    fill(ds, ds+SIZE, 0);
    for(int i = 0; i < N; i++){

```

```

    int y = mp[intervals[i][1]];
    int id = intervals[i][2];

    ans[1][id] = i-query(y-1);
    update(y, 1);
}

for(int i = 0; i < 2; i++)
    for(int j = 0; j < N; j++)
        printf("%d%c", (ans[i][j] ? 1 : 0), (" \n")[j==N-1]);
}

```

Nested Ranges Count

```

#include

using namespace std;
typedef array triple;
const int maxN = 2e5+1;
const int SIZE = 2*maxN;

int N, ans[2][maxN], ds[SIZE];
triple intervals[maxN];
set unique_vals;
map mp;

void update(int idx, int val){
    for(int i = idx; i < SIZE; i += -i&i)    ds[i] += val;
}

int query(int idx){
    int cnt = 0;
    for(int i = idx; i > 0; i -= -i&i)
        cnt += ds[i];
    return cnt;
}

int main(){
    scanf("%d", &N);
    for(int i = 0, x, y; i < N; i++){
        scanf("%d %d", &x, &y);
        intervals[i] = {x, y, i};
        unique_vals.insert(x);
        unique_vals.insert(y);
    }
    sort(intervals, intervals+N, [](triple A, triple B){
        return (A[0] == B[0] ? A[1] > B[1] : A[0] < B[0]);
    });

    int val_id = 1;
    for(int x : unique_vals)
        mp[x] = val_id++;

    for(int i = N-1; i >= 0; i--){
        int y = mp[intervals[i][1]];
        int id = intervals[i][2];

        ans[0][id] = query(y);
        update(y, 1);
    }
}

```

```

fill(ds, ds+SIZE, 0);
for(int i = 0; i < N; i++){
    int y = mp[intervals[i][1]];
    int id = intervals[i][2];

    ans[1][id] = i-query(y-1);
    update(y, 1);
}

for(int i = 0; i < 2; i++)
    for(int j = 0; j < N; j++)
        printf("%d%c", ans[i][j], (" \n")[j==N-1]);
}

```

Room Allocation

```

#include

using namespace std;
const int maxN = 2e5;

set S;
int N, a, b, rooms, ans[maxN+1];
struct query {int type, time, id;} queries[2*maxN];

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        S.insert(i+1);
        scanf("%d %d", &a, &b);
        queries[2*i] = {0, a, i+1};
        queries[2*i+1] = {1, b, i+1};
    }

    sort(queries, queries+2*N, [](query A, query B){
        if(A.id == B.id) return A.type < B.type;
        return A.time == B.time ? A.type < B.type : A.time < B.time;
    });

    for(int i = 0; i < 2*N; i++){
        if(queries[i].type == 0){
            ans[queries[i].id] = *S.begin();
            S.erase(S.begin());
        } else if(queries[i].type == 1){
            S.insert(ans[queries[i].id]);
        }
    }

    for(int i = 1; i <= N; i++)
        rooms = max(rooms, ans[i]);

    printf("%d\n", rooms);
    for(int i = 1; i <= N; i++)
        printf("%d%c", ans[i], (" \n")[i==N]);
}

```

Factory Machines

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N;
ll T, cnt, k[maxN];

bool check(ll t){
    cnt = 0;
    for(int i = 0; i < N; i++){
        cnt += t/k[i];
        if(cnt >= T)
            return true;
    }
    return false;
}

int main(){
    scanf("%d %lld", &N, &T);
    for(int i = 0; i < N; i++)
        scanf("%lld", &k[i]);

    ll lo = 0, hi = 1e18;
    while(lo <= hi){
        ll mid = lo + (hi-lo)/2;
        if(check(mid)) hi = mid-1;
        else lo = mid+1;
    }
    printf("%lld\n", lo);
}
```

Tasks and Deadlines

```
#include

using namespace std;
const int maxN = 2e5;

int N, a, d;
long long timer, reward;
struct task {int a, d;} tasks[maxN];

int main (){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d %d", &a, &d);
        tasks[i] = {a, d};
    }

    sort(tasks, tasks+N, [](task x, task y){
        return x.a == y.a ? x.d < y.d : x.a < y.a;
    });

    for(int i = 0; i < N; i++){
        timer += tasks[i].a;
    }
}
```

```
        reward += (tasks[i].d - timer);
    }
    printf("%lld\n", reward);
}
```

Reading Books

```
#include

using namespace std;
typedef long long ll;

int N;
ll t, mx, sum;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%lld", &t);
        mx = max(mx, t);
        sum += t;
    }
    printf("%lld\n", max(sum, 2*mx));
}
```

Sum of Three Values

```
#include

using namespace std;
typedef pair pii;
const int maxN = 5000;

int N, X, v, r;
pii a[maxN];

int main(){
    scanf("%d %d", &N, &X);
    for(int i = 0; i < N; i++){
        scanf("%d", &v);
        a[i] = {v, i+1};
    }

    sort(a, a+N);
    for(int i = 0; i < N; i++){
        v = X - a[i].first;
        r = N-1;
        for(int l = i+1; l < r; l++){
            while(l+1 < r && a[l].first + a[r].first > v)    r--;
            if(a[l].first + a[r].first == v){
                printf("%d %d %d\n", a[i].second, a[l].second, a[r].second);
                return 0;
            }
        }
    }
}
```

```

    }
}
}
printf("IMPOSSIBLE\n");
}

```

Sum of Four Values

```

#include

using namespace std;
typedef long long ll;
typedef pair pii;
const int maxN = 1001;

int N;
ll X, a[maxN];
unordered_map> mp;

int main(){
    scanf("%d %lld", &N, &X);
    mp.reserve(maxN*maxN);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &a[i]);
        for(int j = 1; j < i; j++){
            ll psum = a[i] + a[j];
            if(psum >= X) continue;
            if(mp.find(X-psum) != mp.end()){
                for(pii P : mp[X-psum]){
                    if(P.first != j && P.second != i && P.first != i && P.second != j){
                        printf("%d %d %d %d\n", P.first, P.second, j, i);
                        return 0;
                    }
                }
            } else mp[psum].push_back({j, i});
        }
    }
    printf("IMPOSSIBLE\n");
}

```

Nearest Smaller Values

```

#include

using namespace std;
const int maxN = 2e5+1;

int N, x[maxN], ds[maxN];

int main(){
    scanf("%d ", &N);
    for(int i = 1; i <= N; i++){

```

```

scanf("%d", &x[i]);
int k = i-1;
while(x[k] >= x[i])
    k = ds[k];
ds[i] = k;
printf("%d%c", ds[i], (" \n")[i==N]);
}
}

```

Subarray Sums I

```

#include

using namespace std;
const int maxN = 2e5;

int N, x, l, sum, cnt, a[maxN+1];

int main(){
    scanf("%d %d", &N, &x);
    for(int r = 1; r <= N; r++){
        scanf("%d", &a[r]);
        sum += a[r];
        while(sum > x)
            sum -= a[l++];
        if(sum == x)    cnt++;
    }
    printf("%d\n", cnt);
}

```

Subarray Sums II

```

#include

using namespace std;
typedef long long ll;

int N;
ll X, a, cnt, pre;
map freq;

int main(){
    scanf("%d %lld", &N, &X);
    freq[0] = 1;
    for(int i = 0; i < N; i++){
        scanf("%lld", &a);
        pre += a;
        cnt += freq[pre-X];
        freq[pre]++;
    }
}

```

```
    printf("%lld\n", cnt);  
}
```

Subarray Divisibility

```
#include  
  
using namespace std;  
typedef long long ll;  
const int maxN = 2e5;  
  
int N, pre, a, freq[maxN];  
ll cnt;  
  
int mod(int x){  
    return (x % N + N) % N;  
}  
  
int main(){  
    scanf("%d", &N);  
    freq[0] = 1;  
    for(int i = 0; i < N; i++){  
        scanf("%d", &a);  
        pre = mod(pre+a);  
        cnt += freq[pre];  
        freq[pre]++;  
    }  
    printf("%lld\n", cnt);  
}
```

Distinct Values Subarrays II

```
#include  
  
using namespace std;  
typedef long long ll;  
const int maxN = 2e5+5;  
  
int N, K, x[maxN];  
map freq;  
ll ans;  
  
int main(){  
    scanf("%d %d", &N, &K);  
    for(int i = 1; i <= N; i++)  
        scanf("%d", &x[i]);  
  
    int unique = 0;  
    int r = 0;  
    for(int l = 1; l <= N; l++){  
        while(r < N && (freq[x[r+1]] >= 1 || unique < K)){  
            r++;  
        }  
        ans += (r - l + 1);  
        freq[x[l]]++;  
        unique++;  
    }  
    printf("%lld\n", ans);  
}
```

```
        freq[x[r]]++;
        if(freq[x[r]] == 1)
            unique++;
    }

    ans += (r-l+1);

    freq[x[l]]--;
    if(!freq[x[l]])
        unique--;
}

printf("%lld\n", ans);
}
```

Array Division

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, K, a[maxN];
ll lo = 0, hi = 1e18;

bool check(ll X){
    int k = 1;
    ll sum = 0;
    for(int i = 0; i < N; i++){
        sum += a[i];
        if(sum > X){
            k++;
            sum = a[i];
        }
    }
    return k <= K;
}

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < N; i++){
        scanf("%d", &a[i]);
        lo = max(lo, (ll) a[i]);
    }

    while(lo <= hi){
        ll mid = lo + (hi-lo)/2;
        if(check(mid)) hi = mid-1;
        else lo = mid+1;
    }
    printf("%lld\n", lo);
}
```

Movie Festival II

```
#include

using namespace std;
const int maxN = 2e5, SIZE = 5e6;
const int INF = 0x3f3f3f3f;

int N, K, ans, A[maxN], B[maxN], lo[SIZE], hi[SIZE], d[SIZE], mx[SIZE];
struct Movie { int t, id, type; } movies[2*maxN];
map mp;

void push(int i){
    if(d[i]){
        d[2*i] += d[i];
        d[2*i+1] += d[i];
        d[i] = 0;
    }
}

void pull(int i){
    mx[i] = max(mx[2*i]+d[2*i], mx[2*i+1]+d[2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r) return;
    int m = l+(r-l)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void increment(int i, int l, int r){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        d[i]++; return;
    }
    push(i);
    increment(2*i, l, r);
    increment(2*i+1, l, r);
    pull(i);
}

int maximum(int i, int l, int r){
    if(l > hi[i] || r < lo[i]) return -INF;
    if(l <= lo[i] && hi[i] <= r) return mx[i]+d[i];

    push(i);
    int lmax = maximum(2*i, l, r);
    int rmax = maximum(2*i+1, l, r);
    pull(i);

    return max(lmax, rmax);
}

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < N; i++){
        scanf("%d %d", &A[i], &B[i]);
        movies[2*i] = {A[i], i, 0};
        movies[2*i+1] = {B[i], i, 1};
    }
    sort(movies, movies+2*N, [](Movie a, Movie b){
        return a.t == b.t ? A[a.id] > A[b.id] : a.t < b.t;
    });

    for(int i = 0; i < 2*N; i++)
```

```

        mp[movies[i].t] = 2*i+1;

    init(1, 1, 4*N);
    for(int i = 0; i < 2*N; i++){
        if(movies[i].type == 1){
            int id = movies[i].id;
            int a = mp[A[id]];
            int b = mp[B[id]];

            if(maximum(1, a, b) < K){
                increment(1, a, b-1);
                ans++;
            }
        }
    }
    printf("%d\n", ans);
}

```

Maximum Subarray Sum II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, A, B;
ll pre[maxN];
multiset S;

int main(){
    scanf("%d %d %d", &N, &A, &B);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &pre[i]);
        pre[i] += pre[i-1];
    }

    for(int i = A; i <= B; i++)
        S.insert(pre[i]);

    ll best = *S.rbegin();
    for(int i = 1; i+A <= N; i++){
        S.erase(pre[i+A-1]);
        S.insert(pre[min(i+B, N)]);
        best = max(best, *S.rbegin()-pre[i]);
    }

    printf("%lld\n", best);
}

```

Dinamic

Dice Combinations

```
#include

using namespace std;
const int maxN = 1e6;
const int MOD = 1e9+7;

int N, dp[maxN+1];

int main(){
    scanf("%d", &N);

    dp[0] = 1;
    for(int i = 1; i <= N; i++)
        for(int j = 1; j <= 6 && i-j >= 0; j++)
            dp[i] = (dp[i] + dp[i-j]) % MOD;

    printf("%d\n", dp[N]);
}
```

Minimizing Coins

```
#include

using namespace std;
const int maxX = 1e6;
const int INF = 0x3f3f3f3f;

int N, X, c, dp[maxX+1];

int main(){
    scanf("%d %d", &N, &X);
    fill(dp+1, dp+X+1, INF);
    for(int i = 0; i < N; i++){
        scanf("%d", &c);
        for(int j = 0; j <= X-c; j++){
            if(dp[j] != INF)
                dp[j+c] = min(dp[j+c], dp[j]+1);
        }
        printf("%d\n", dp[X] == INF ? -1 : dp[X]);
    }
}
```

Coin Combinations I

```

#include

using namespace std;
typedef long long ll;
const int maxN = 100;
const int maxX = 1e6;
const ll MOD = 1e9+7;

int N, X, c[maxN];
ll dp[maxX+1];

int main(){
    scanf("%d %d", &N, &X);
    for(int i = 0; i < N; i++)
        scanf("%d", &c[i]);

    dp[0] = 1;
    for(int i = 0; i < X; i++)
        if(dp[i] != 0)
            for(int j = 0; j < N; j++)
                if(i+c[j] <= X)
                    dp[i+c[j]] = (dp[i+c[j]] + dp[i]) % MOD;

    printf("%lld\n", dp[X]);
}

```

Coin Combinations II

```

#include

using namespace std;
typedef long long ll;
const int maxX = 1e6;
const ll MOD = 1e9+7;

int N, X, c;
ll dp[maxX+1];

int main(){
    scanf("%d %d", &N, &X);

    dp[0] = 1;
    for(int i = 0; i < N; i++){
        scanf("%d", &c);
        for(int j = 0; j <= X-c; j++)
            dp[j+c] = (dp[j+c] + dp[j]) % MOD;
    }

    printf("%lld\n", dp[X]);
}

```

Removing Digits

```
#include

using namespace std;
const int maxN = 1e6;
const int INF = 0x3f3f3f3f;

int N, dp[maxN+1];

int main(){
    scanf("%d", &N);
    fill(dp+1, dp+N+1, INF);

    for(int i = 1; i <= N; i++){
        int d = i;
        while(d > 0){
            if(d%10 != 0)
                dp[i] = min(dp[i], dp[i-(d%10)]+1);
            d /= 10;
        }
    }

    printf("%d\n", dp[N]);
}
```

Grid Paths I

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1000;
const ll MOD = 1e9+7;

int N;
char c[maxN+1][maxN+1];
ll dp[maxN+1][maxN+1];

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++)
        for(int j = 1; j <= N; j++)
            scanf(" %c", &c[i][j]);

    dp[1][1] = (c[1][1] == '.');
    for(int i = 1; i <= N; i++){
        for(int j = 1; j <= N; j++){
            if(c[i][j] == '.'){
                if(c[i-1][j] == '.') dp[i][j] += dp[i-1][j];
                if(c[i][j-1] == '.') dp[i][j] += dp[i][j-1];
                dp[i][j] %= MOD;
            }
        }
    }

    printf("%lld\n", dp[N][N]);
}
```

Book Shop

```
#include

using namespace std;
const int maxN = 1000;
const int maxX = 1e5;

int N, X, h[maxN], s[maxN], dp[maxX+1];

int main(){
    scanf("%d %d", &N, &X);
    for(int i = 0; i < N; i++) scanf("%d", &h[i]);
    for(int i = 0; i < N; i++) scanf("%d", &s[i]);

    fill(dp+1, dp+X+1, -1);
    for(int i = 0; i < N; i++)
        for(int j = X-h[i]; j >= 0; j--)
            if(dp[j] != -1)
                dp[j+h[i]] = max(dp[j+h[i]], dp[j]+s[i]);

    for(int i = 1; i <= X; i++)
        dp[i] = max(dp[i], dp[i-1]);
    printf("%d\n", dp[X]);
}
```

Array Description

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e5;
const int maxM = 100;
const ll MOD = 1e9+7;

int N, M, x[maxN+1];
ll ans, dp[maxN+1][maxM+1];

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++)
        scanf("%d", &x[i]);

    if(x[1])
        dp[1][x[1]] = 1;
    else
        for(int i = 1; i <= M; i++)
            dp[1][i] = 1;

    for(int i = 2; i <= N; i++){
        for(int j = 1; j <= M; j++){
            dp[i][j] = dp[i-1][j];
            if(j != 1) dp[i][j] += dp[i-1][j-1];
            if(j != M) dp[i][j] += dp[i-1][j+1];
            dp[i][j] %= MOD;
        }

        if(x[i])
```

```

        for(int j = 0; j <= M; j++)
            if(j != x[i])
                dp[i][j] = 0;
    }

    for(int i= 1; i <= M; i++)
        ans = (ans + dp[N][i]) % MOD;
    printf("%lld\n", ans);
}

```

Counting Towers

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e6;
const ll MOD = 1e9+7;

int T, N;
ll dp[maxN+1];

int main(){
    dp[1] = 2;
    dp[2] = 8;
    for(int i = 3; i <= maxN; i++)
        dp[i] = ((6*dp[i-1] - 7*dp[i-2]) % MOD + MOD) % MOD;

    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        scanf("%d", &N);
        printf("%lld\n", dp[N]);
    }
}

```

Edit Distance

```

#include

using namespace std;
const int maxN = 5e3+5;

int N, M, dp[maxN][maxN];
char a[maxN], b[maxN];

int main(){
    scanf("%s %s", a, b);
    N = (int) strlen(a);
    M = (int) strlen(b);

    memset(dp, 0x3f, sizeof(dp));
    for(int i = 0; i <= N; i++){

```

```

    for(int j = 0; j <= M; j++){
        if(i == 0) dp[i][j] = j;
        else if(j == 0) dp[i][j] = i;
        else if(a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1];
        else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
    }
}

printf("%d\n", dp[N][M]);
}

```

Longest Common Subsequence

```

#include

using namespace std;
const int maxN = 1e3+1;

int N, M, a[maxN], b[maxN];
int len[maxN][maxN];
char ptr[maxN][maxN];

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++) scanf("%d", &a[i]);
    for(int i = 1; i <= M; i++) scanf("%d", &b[i]);

    for(int i = 1; i <= N; i++){
        for(int j = 1; j <= M; j++){
            if(len[i][j] <= len[i-1][j]){
                len[i][j] = len[i-1][j];
                ptr[i][j] = 'L';
            }
            if(len[i][j] <= len[i][j-1]){
                len[i][j] = len[i][j-1];
                ptr[i][j] = 'U';
            }
            if(a[i] == b[j] && len[i][j] <= len[i-1][j-1] + 1){
                len[i][j] = len[i-1][j-1] + 1;
                ptr[i][j] = 'D';
            }
        }
    }

    vector ans;
    int i = N, j = M;
    while(i != 0 && j != 0){
        char dir = ptr[i][j];

        if(dir == 'L') i--;
        else if(dir == 'U') j--;
        else if(dir == 'D'){
            ans.push_back(a[i]);
            i--; j--;
        }
    }
    reverse(ans.begin(), ans.end());

    int l = len[N][M];
    printf("%d\n", l);
    for(int i = 0; i < l; i++)

```

```
        printf("%d%c", ans[i], (" \n")[i==l-1]);  
    }
```

Rectangle Cutting

```
#include  
  
using namespace std;  
const int INF = 0x3f3f3f3f;  
  
int A, B, dp[501][501];  
  
int solve(int a, int b){  
    if(a > b) swap(a, b);  
    if(dp[a][b] != INF) return dp[a][b];  
    if(a == b) return dp[a][b] = 0;  
    if(a == 1 || b == 1) return dp[a][b] = (a == 1 ? b-1 : a-1);  
    for(int i = 1; i < a; i++)  
        dp[a][b] = min(dp[a][b], solve(i, b) + solve(a-i, b) + 1);  
    for(int i = 1; i < b; i++)  
        dp[a][b] = min(dp[a][b], solve(a, i) + solve(a, b-i) + 1);  
    return dp[a][b];  
}  
  
int main(){  
    memset(dp, 0x3f, sizeof(dp));  
    scanf("%d %d", &A, &B);  
    printf("%d\n", solve(A, B));  
}
```

Money Sums

```
#include  
  
using namespace std;  
const int maxX = 1e5;  
  
int N, x, cnt, largest;  
bool dp[maxX+1];  
  
int main(){  
    scanf("%d", &N);  
  
    dp[0] = true;  
    for(int i = 0; i < N; i++){  
        scanf("%d", &x);  
        for(int j = maxX-x; j >= 0; j--)  
            if(dp[j])  
                dp[j+x] = true;  
    }  
  
    for(int i = 1; i <= maxX; i++){
```

```

        if(dp[i]){
            largest = i;
            cnt++;
        }
    }

    printf("%d\n", cnt++);
    for(int i = 1; i <= maxX; i++)
        if(dp[i])
            printf("%d%c", i, (" \n")[i==largest]);
}

```

Removal Game

```

#include

using namespace std;
typedef long long ll;
const int maxN = 5000;

int N, x[maxN+1];
ll p[maxN+1], dp[maxN+1][maxN+1];
bool found[maxN+1][maxN+1];

ll sum(int l, int r){
    return p[r] - p[l-1];
}

ll solve(int l, int r){
    if(found[l][r]) return dp[l][r];
    if(l == r) return x[l];
    found[l][r]=true;
    return dp[l][r] = max(x[l]+sum(l+1, r)-solve(l+1, r), x[r]+sum(l,r-1)-solve(l,r-1));
}

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x[i]);
        p[i] = p[i-1] + x[i];
    }
    printf("%lld\n", solve(1, N));
}

```

Two Sets II

```

#include
using namespace std;

long long mod = 1000000007;
long long dp[70001];

```

```

int main() {
    int n;
    cin >> n;
    long long sum = (1ll * n * (n + 1)) / 2;
    if (sum % 2) {
        cout << 0 << endl;
        return 0;
    }
    long long target = sum / 2;

    dp[0] = 1;
    // i= i; j-- ) {
        dp[j] += dp[j - i];
        dp[j] %= mod;
    }
}
cout << dp[target] << endl;
return 0;
}

```

Mountain Range

```

#include

using namespace std;
typedef pair pii;
const int maxN = 2e5+1;
const int INF = 0x3f3f3f3f;

int N, dp[maxN];
vector tallers;
vector mountains;
set< S_l;
set S_r;

int main(){
    scanf("%d", &N);

    for(int i = 1, h; i <= N; i++){
        scanf("%d", &h);
        mountains.push_back({h, i});
    }
    sort(mountains.begin(), mountains.end(), greater());

    int last_height = INF;
    for(pii m : mountains){
        int h = m.first, i = m.second;

        if(last_height != h){
            for(int t : tallers){
                S_l.insert(t);
                S_r.insert(t);
            }
            tallers.clear();
        }

        auto l_ptr = S_l.lower_bound(i);
        auto r_ptr = S_r.lower_bound(i);
        int l = (l_ptr == S_l.end() ? 0 : *l_ptr);
        int r = (r_ptr == S_r.end() ? 0 : *r_ptr);
        dp[i] = max(dp[l], dp[r]) + 1;
    }
}

```

```

        tallers.push_back(i);
        last_height = h;
    }

    int best = 0;
    for(int i = 1; i <= N; i++)
        best = max(best, dp[i]);
    printf("%d\n", best);
}

```

Increasing Subsequence

```

#include

using namespace std;

int N, a;
multiset S;
multiset::iterator it;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d", &a);
        S.insert(a);
        it = S.lower_bound(a);
        it++;
        if(it != S.end())
            S.erase(it);
    }
    printf("%d\n", (int) S.size());
}

```

Projects

```

#include

using namespace std;
typedef long long ll;
typedef pair pii;
const int maxN = 2e5+1;

int N, a[maxN], b[maxN];
ll p[maxN], dp[2*maxN];
struct project {int time, id, type;} times[2*maxN];
map mp;

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%d %d %lld", &a[i], &b[i], &p[i]);
    }
}

```

```

        times[2*i] = {a[i], i, 0};
        times[2*i+1] = {b[i], i, 1};
    }

    sort(times+2, times+2*N+2, [](project A, project B){
        return A.time == B.time ? A.id < B.id : A.time < B.time;
    });

    for(int i = 2; i <= 2*N+1; i++)
        if(!mp[times[i].time])
            mp[times[i].time] = i;

    for(int i = 2; i <= 2*N+1; i++){
        if(times[i].type == 0) dp[i] = dp[i-1];
        else dp[i] = max(dp[i-1], dp[mp[a[times[i].id]]-1] + p[times[i].id]);
    }
    printf("%lld\n", dp[2*N+1]);
}

```

Elevator Rides

```

#include

using namespace std;
typedef pair pii;
typedef long long ll;
const int maxN = 20;
const int INF = maxN+1;

int N;
pii dp[1<

```

Counting Tilings

```

#include

using namespace std;
typedef long long ll;
const int maxN = 11;
const int maxM = 1001;
const ll MOD = 1e9+7;

int N, M;
ll dp[maxN][maxM][1<

```

Counting Numbers

```
#include

using namespace std;
typedef long long ll;
const int maxN = 20;

bool tight[maxN];
ll dp[10][maxN];

ll solve(ll x){
    if(x <= 10) return x;

    vector D;
    memset(dp, 0, sizeof(dp));
    memset(tight, 0, sizeof(tight));
    for(int i = 0; x; i++){
        D.push_back(x%10);
        x /= 10;
    }
    reverse(D.begin(), D.end());
    int N = (int) D.size();

    tight[0] = true;
    for(int i = 1; i < N; i++){
        tight[i] = tight[i-1] && (D[i] != D[i-1]);
    }
    for(int d = 1; d < D[0]; d++){
        dp[d][0] = 1;
    }

    for(int i = 1; i < N; i++){
        // Prefix [0..i-1] is same as X
        if(tight[i-1])
            for(int d = 0; d < D[i]; d++){
                if(d != D[i-1])
                    dp[d][i]++;
            }

        // Prefix [0..i-1] is all leading zeros
        for(int d = 1; d <= 9; d++){
            dp[d][i]++;
        }

        // All other cases
        for(int d1 = 0; d1 <= 9; d1++){
            for(int d2 = 0; d2 <= 9; d2++){
                if(d1 != d2)
                    dp[d2][i] += dp[d1][i-1];
            }
        }

        ll cnt = tight[N-1];
        for(int d = 0; d <= 9; d++){
            cnt += dp[d][N-1];
        }
        return cnt;
    }

    int main(){
        ll a, b;
        scanf("%lld %lld", &a, &b);
        printf("%lld\n", solve(b) - solve(a-1));
    }
}
```

Increasing Subsequence II

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+5;
const ll MOD = 1e9+7;

int N;
ll ans, ds[maxN];
struct Operation { int x, idx; } ops[maxN];

void update(int idx, ll val){
    for(int i = idx; i < maxN; i += -i&i)
        ds[i] = (ds[i] + val) % MOD;
}

ll query(int idx){
    ll sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum = (sum + ds[i]) % MOD;
    return sum;
}

int main(){
    scanf("%d", &N);
    for(int i = 0, x; i < N; i++){
        scanf("%d", &x);
        ops[i] = {x, i+1};
    }
    sort(ops, ops+N, [](Operation A, Operation B){
        return A.x == B.x ? B.idx < A.idx : A.x < B.x;
    });

    for(int i = 0; i < N; i++){
        int idx = ops[i].idx;
        ll amnt = query(idx)+1;
        ans = (ans + amnt) % MOD;
        update(idx, amnt);
    }
    printf("%lld\n", ans);
}
```

Graph

Counting Rooms

```
#include

using namespace std;

char c;
int N, M, cnt;
int h[] = {1, -1, 0, 0}, v[] = {0, 0, 1, -1};
bool vis[1000][1000];

void dfs(int x, int y){
    vis[x][y] = true;
    for(int i = 0; i < 4; i++){
        int dx = x+h[i], dy = y+v[i];
        if(0 <= dx && dx < N && 0 <= dy && dy < M && !vis[dx][dy])
            dfs(dx, dy);
    }
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++){
        for(int j = 0; j < M; j++){
            scanf(" %c", &c);
            vis[i][j] = (c == '#');
        }
    }
    for(int i = 0; i < N; i++){
        for(int j = 0; j < M; j++){
            if(!vis[i][j]){
                dfs(i, j);
                cnt++;
            }
        }
    }
    printf("%d\n", cnt);
}
```

Labyrinth

```
#include

using namespace std;
typedef pair pii;
#define x first
#define y second

const int h[] = {1, -1, 0, 0}, v[] = {0, 0, 1, -1};

bool vis[1000][1000];
char c, par[1000][1000], ans[1000000];
```

```

int N, M, sx, sy, ex, ey, dist[1000][1000];
queue Q;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++){
        for(int j = 0; j < M; j++){
            scanf(" %c", &c);
            if(c == '#') vis[i][j] = true;
            else if(c == 'A'){
                sx = i; sy = j;
            } else if(c == 'B'){
                ex = i; ey = j;
            }
        }
    }

    vis[sx][sy] = true;
    Q.push({sx, sy});
    while(!Q.empty()){
        pii P = Q.front(); Q.pop();
        for(int i = 0; i < 4; i++){
            int dx = P.x+h[i];
            int dy = P.y+v[i];
            if(0 <= dx && dx < N && 0 <= dy && dy < M && !vis[dx][dy]){
                if(i == 0) par[dx][dy] = 'D';
                else if(i == 1) par[dx][dy] = 'U';
                else if(i == 2) par[dx][dy] = 'R';
                else if(i == 3) par[dx][dy] = 'L';
                dist[dx][dy] = dist[P.x][P.y]+1;
                vis[dx][dy] = true;
                Q.push({dx, dy});
            }
        }
    }

    if(!vis[ex][ey]){
        printf("NO\n");
        return 0;
    }

    printf("YES\n%d\n", dist[ex][ey]);
    pii P = {ex, ey};
    for(int i = dist[ex][ey]; i > 0; i--){
        ans[i] = par[P.x][P.y];
        if(ans[i] == 'D') P = {P.x-1, P.y};
        else if(ans[i] == 'U') P = {P.x+1, P.y};
        else if(ans[i] == 'R') P = {P.x, P.y-1};
        else if(ans[i] == 'L') P = {P.x, P.y+1};
    }
    for(int i = 1; i <= dist[ex][ey]; i++)
        printf("%c", ans[i]);
    printf("\n");
}

```

Building Roads

```

#include

using namespace std;

```

```

typedef pair pii;

int N, M, a, b, ds[100001];
vector ans;

int find(int u){
    if(ds[u] < 0)    return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v)    return false;
    if(ds[u] < ds[v])    swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++) ds[i] = -1;
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        merge(a, b);
    }

    for(int i = 1; i < N; i++)
        if(merge(i, i+1))
            ans.push_back({i, i+1});

    printf("%d\n", (int) ans.size());
    for(pii P : ans)
        printf("%d %d\n", P.first, P.second);
}

```

Message Route

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, K, a, b, p[maxN], dist[maxN];
bool vis[maxN];
vector G[maxN];
queue Q;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    Q.push(1);
    vis[1] = true;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();

```

```

        for(int v : G[u]){
            if(!vis[v]){
                dist[v] = dist[u]+1;
                vis[v] = true;
                p[v] = u;
                Q.push(v);
            }
        }
    }

    if(!vis[N]){
        printf("IMPOSSIBLE\n");
        return 0;
    }

    int u = N;
    K = dist[N];
    vector ans(K+1);
    for(int i = K; i >= 0; i--){
        ans[i] = u;
        u = p[u];
    }

    printf("%d\n", K+1);
    for(int i = 0; i <= K; i++){
        printf("%d%c", ans[i], (" \n")[i==K]);
    }
}

```

Building Teams

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, a, b;
bool possible, vis[maxN], team[maxN];
vector G[maxN];

void dfs(int u, int p = 0){
    for(int v : G[u]){
        if(v != p){
            if(!vis[v]){
                team[v] = !team[u];
                vis[v] = true;
                dfs(v, u);
            } else
                if(team[v] == team[u])
                    possible = false;
        }
    }
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }
}

```

```

possible = true;
for(int i = 1; i <= N; i++){
    if(!vis[i]){
        vis[i] = true;
        dfs(i);
    }
}

if(!possible)
    printf("IMPOSSIBLE\n");
else
    for(int i = 1; i <= N; i++)
        printf("%d%c", (team[i] ? 1 : 2), (" \n")[i==N]);
}

```

Round Trip

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, a, b, p[maxN], ds[maxN];
vector ans, G[maxN];

void dfs(int u){
    for(int v : G[u]){
        if(v != p[u]){
            p[v] = u;
            dfs(v);
        }
    }
}

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d", &N, &M);
    fill(ds+1, ds+N+1, -1);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        if(!merge(a, b)){
            dfs(a);

            int u = b;
            while(u != 0){
                ans.push_back(u);
                u = p[u];
            }
        }
    }
}

```

```

    }

    int K = ans.size();
    printf("%d\n", K+1);
    for(int j = 0; j < K; j++)
        printf("%d ", ans[j]);
    printf("%d\n", b);

    return 0;
} else {
    G[a].push_back(b);
    G[b].push_back(a);
}
}
printf("IMPOSSIBLE\n");
}

```

Monsters

```

#include

using namespace std;
const int maxN = 1000;
const int INF = 0x3f3f3f3f;

int h[] = {1, -1, 0, 0};
int v[] = {0, 0, 1, -1};
struct point { int x, y; };

int N, M, sx, sy;
bool vis[maxN][maxN];
char ans[maxN*maxN], c[maxN][maxN], p[maxN][maxN];
int d1[maxN][maxN], d2[maxN][maxN];
queue Q;

bool inbounds(int x, int y){
    return 0 <= x && x < N && 0 <= y && y < M;
}

void printsolution(int x, int y){
    int D = d2[x][y];
    printf("YES\n%d\n", D);
    for(int i = D-1; i >= 0; i--){
        ans[i] = p[x][y];
        if(ans[i] == 'D')    x--;
        else if(ans[i] == 'U')    x++;
        else if(ans[i] == 'R')    y--;
        else if(ans[i] == 'L')    y++;
    }
    for(int i = 0; i < D; i++)
        printf("%c", ans[i]);
    printf("\n");
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++){
        for(int j = 0; j < M; j++){
            scanf(" %c", &c[i][j]);
            if(c[i][j] == '#')    vis[i][j] = true;
            else if(c[i][j] == 'M'){

```

```

        vis[i][j] = true;
        Q.push({i, j});
    } else if(c[i][j] == 'A'){
        sx = i; sy = j;
    }
}
}

while(!Q.empty()){
    point P = Q.front(); Q.pop();
    for(int i = 0; i < 4; i++){
        int x = P.x + h[i];
        int y = P.y + v[i];
        if(inbounds(x, y) && !vis[x][y]){
            d1[x][y] = d1[P.x][P.y] + 1;
            vis[x][y] = true;
            Q.push({x, y});
        }
    }
}

for(int i = 0; i < N; i++){
    for(int j = 0; j < M; j++){
        if(!vis[i][j]) d1[i][j] = INF;
        vis[i][j] = (c[i][j] == '#');
    }
}

vis[sx][sy] = true;
Q.push({sx, sy});
while(!Q.empty()){
    point P = Q.front(); Q.pop();
    for(int i = 0; i < 4; i++){
        int x = P.x + h[i];
        int y = P.y + v[i];
        if(inbounds(x, y) && !vis[x][y] && d2[P.x][P.y]+1 < d1[x][y]){
            if(i == 0) p[x][y] = 'D';
            else if(i == 1) p[x][y] = 'U';
            else if(i == 2) p[x][y] = 'R';
            else if(i == 3) p[x][y] = 'L';
            d2[x][y] = d2[P.x][P.y] + 1;
            vis[x][y] = true;
            Q.push({x, y});
        }
    }
}

for(int i = 0; i < N; i++){
    if(c[i][0] != '#' && c[i][0] != 'M' && vis[i][0]){
        printsolution(i, 0);
        return 0;
    } else if(c[i][M-1] != '#' && c[i][M-1] != 'M' && vis[i][M-1]){
        printsolution(i, M-1);
        return 0;
    }
}

for(int i = 0; i < M; i++){
    if(c[0][i] != '#' && c[0][i] != 'M' && vis[0][i]){
        printsolution(0, i);
        return 0;
    } else if(c[N-1][i] != '#' && c[N-1][i] != 'M' && vis[N-1][i]){
        printsolution(N-1, i);
        return 0;
    }
}

printf("NO\n");
}

```

Shortest Routes I

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e5;

struct Edge { int v; ll w; };
struct Node {
    int id; ll dist;
    friend bool operator<(const Node &a, const Node &b){
        return a.dist > b.dist;
    }
};

int N, M, a, b;
ll dist[maxN+1], c;
vector G[maxN+1];
priority_queue Q;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d %lld", &a, &b, &c);
        G[a].push_back({b, c});
    }

    memset(dist, 0x3f, sizeof(dist));
    dist[1] = 0;
    Q.push({1, 0});
    while(!Q.empty()){
        ll d = Q.top().dist;
        int u = Q.top().id;
        Q.pop();

        if(d > dist[u]) continue;

        for(Edge e : G[u]){
            if(dist[e.v] > d+e.w){
                dist[e.v] = d+e.w;
                Q.push({e.v, d+e.w});
            }
        }
    }

    for(int i = 1; i <= N; i++)
        printf("%lld%c", dist[i], (" \n")[i==N]);
}
```

Shortest Routes II

```
#include

using namespace std;
```

```

typedef long long ll;
const int maxN = 500;
const ll INF = 0x3f3f3f3f3f3f3f3f;

int N, M, Q, a, b;
ll dist[maxN+1][maxN+1], c;

int main(){
    scanf("%d %d %d", &N, &M, &Q);
    memset(dist, 0x3f, sizeof(dist));
    for(int i = 0; i < M; i++){
        scanf("%d %d %lld", &a, &b, &c);
        dist[a][b] = min(dist[a][b], c);
        dist[b][a] = min(dist[b][a], c);
    }

    for(int i = 1; i <= N; i++) dist[i][i] = 0;
    for(int k = 1; k <= N; k++)
        for(int i = 1; i <= N; i++)
            for(int j = 1; j <= N; j++)
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);

    for(int q = 0; q < Q; q++){
        scanf("%d %d", &a, &b);
        printf("%lld\n", (dist[a][b] >= INF ? -1 : dist[a][b]));
    }
}

```

High Score

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2501;
const int maxM = 5001;
const ll INF = 0x3f3f3f3f3f3f3f3f;

struct Edge {
    int a, b; ll c;
} edges[maxM];

int N, M;
ll dp[maxN];
bool vis[maxN], visR[maxN];
vector G[maxN], GR[maxN];

void dfs(int u){
    vis[u] = true;
    for(int v : G[u])
        if(!vis[v])
            dfs(v);
}

void dfsR(int u){
    visR[u] = true;
    for(int v : GR[u])
        if(!visR[v])
            dfsR(v);
}

```

```

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        ll c;
        scanf("%d %d %lld", &a, &b, &c);
        edges[i] = {a, b, -c};
        G[a].push_back(b);
        GR[b].push_back(a);
    }
    dfs(1); dfsR(N);

    fill(dp+2, dp+N+1, INF);
    bool improvement = true;
    for(int iter = 0; iter < N && improvement; iter++){
        improvement = false;
        for(int i = 0; i < M; i++){
            int u = edges[i].a;
            int v = edges[i].b;
            ll w = edges[i].c;

            if(dp[v] > dp[u]+w){
                dp[v] = dp[u]+w;
                improvement = true;

                if(iter == N-1 && vis[v] && visR[v]){
                    printf("-1\n");
                    return 0;
                }
            }
        }
    }

    printf("%lld\n", -dp[N]);
}

```

Flight Discount

```

#include

using namespace std;
typedef long long ll;
typedef pair edge;
typedef tuple node;
const int maxN = 2e5+1;

int N, M, a, b;
ll c, dist[2][maxN];
vector G[maxN];
priority_queue, greater> Q;

int main(){
    scanf("%d %d", &N, &M);
    memset(dist, 0x3f, sizeof(dist));
    for(int i = 0; i < M; i++){
        scanf("%d %d %lld", &a, &b, &c);
        G[a].push_back({b, c});
    }

    dist[0][1] = dist[1][1] = 0;
    Q.push({0, 1, 1});
    while(!Q.empty()){

```

```

ll d = get<0>(Q.top());
int u = get<1>(Q.top());
int coupon = get<2>(Q.top());
Q.pop();

if(dist[!coupon][u] < d)    continue;

for(edge e : G[u]){
    int v = e.first;
    ll w = e.second;

    if(coupon){
        if(dist[0][v] > d + w){
            dist[0][v] = d + w;
            Q.push({d+w, v, 1});
        }
        if(dist[1][v] > d + w/2){
            dist[1][v] = d + w/2;
            Q.push({d+w/2, v, 0});
        }
    } else {
        if(dist[1][v] > d + w){
            dist[1][v] = d + w;
            Q.push({d+w, v, 0});
        }
    }
}
}

printf("%lld\n", min(dist[0][N], dist[1][N]));
}

```

Cycle Finding

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2501;
const int maxM = 5001;
const ll INF = 0x3f3f3f3f3f3f3f3f;

struct Edge {
    int a, b; ll c;
} edges[maxM];

int N, M, ptr, p[maxN];
ll dp[maxN];

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        ll c;
        scanf("%d %d %lld", &a, &b, &c);
        edges[i] = {a, b, c};
    }

    ptr = -1;
    fill(dp+2, dp+N+1, INF);
    for(int iter = 0; iter < N && ptr; iter++){
        ptr = 0;

```

```

    for(int i = 0; i < M; i++){
        int u = edges[i].a;
        int v = edges[i].b;
        ll w = edges[i].c;

        if(dp[v] > dp[u]+w){
            dp[v] = dp[u]+w;
            p[v] = u;
            ptr = v;
        }
    }
}

if(!ptr){
    printf("NO\n");
    return 0;
}

for(int i = 0; i < N; i++){
    ptr = p[ptr];

    vector cycle;
    for(int v = ptr;; v = p[v]){
        cycle.push_back(v);
        if(v == ptr && (int) cycle.size() > 1)
            break;
    }
    reverse(cycle.begin(), cycle.end());

    printf("YES\n");
    int K = (int) cycle.size();
    for(int i = 0; i < K; i++)
        printf("%d%c", cycle[i], (" \n")[i==K-1]);
}

```

Flight Routes

```

#include

using namespace std;
typedef long long ll;
typedef pair edge;
typedef pair node;
const int maxN = 2e5+1;

int N, M, K, cnt[maxN];
vector G[maxN];
priority_queue, greater> Q;

int main(){
    scanf("%d %d %d", &N, &M, &K);
    for(int i = 0, a, b, c; i < M; i++){
        scanf("%d %d %d", &a, &b, &c);
        G[a].push_back({b, c});
    }

    Q.push({0, 1});
    while(!Q.empty()){
        ll d = get<0>(Q.top());
        int u = get<1>(Q.top());
        Q.pop();

```

```

        cnt[u]++;
        if(u == N){
            printf("%lld%c", d, (" \n")[cnt[u]==K]);
            if(cnt[u] == K) return 0;
        }

        if(cnt[u] <= K){
            for(edge e : G[u]){
                int v = e.first;
                ll w = e.second;
                Q.push({d+w, v});
            }
        }
    }
}

```

Round Trip II

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, start, finish, p[maxN], vis[maxN];
vector ans, G[maxN];

void dfs(int u){
    vis[u] = 1;
    for(int v : G[u]){
        if(vis[v] == 0){
            p[v] = u;
            dfs(v);
            if(start)
                return;
        } else if(vis[v] == 1){
            finish = u;
            start = v;
            return;
        }
    }
    vis[u] = 2;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
    }

    for(int i = 1; i <= N && !start; i++)
        if(vis[i] == 0)
            dfs(i);

    if(!start){
        printf("IMPOSSIBLE\n");
        return 0;
    }

    ans.push_back(start);

```

```

    for(int u = finish; u != start; u = p[u])
        ans.push_back(u);
    ans.push_back(start);
    reverse(ans.begin(), ans.end());

    int K = (int) ans.size();
    printf("%d\n", K);
    for(int i = 0; i < K; i++)
        printf("%d%c", ans[i], (" \n")[i==K-1]);
}

```

Course Schedule

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, a, b, cnt, in[maxN];
vector ans, G[maxN];
queue Q;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        in[b]++;
    }

    for(int i = 1; i <= N; i++)
        if(in[i] == 0)
            Q.push(i);

    cnt = N;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        ans.push_back(u);
        cnt--;

        for(int v : G[u]){
            in[v]--;
            if(in[v] == 0)
                Q.push(v);
        }
    }

    if(cnt)
        printf("IMPOSSIBLE\n");
    else
        for(int i = 0; i < N; i++)
            printf("%d%c", ans[i], (" \n")[i==N-1]);
}

```

Longest Flight Route

```
#include

using namespace std;
const int maxN = 1e5+1;

bool vis[maxN];
int N, M, K, a, b, in[maxN], p[maxN], l[maxN], ans[maxN];
vector G[maxN];
queue Q;

void dfs(int u, int par = 0){
    vis[u] = true;
    for(int v : G[u])
        if(v != par && !vis[v])
            dfs(v, u);
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        in[b]++;
    }

    dfs(1);
    if(!vis[N]){
        printf("IMPOSSIBLE\n");
        return 0;
    }

    fill(l+2, l+maxN, -1);
    for(int i = 1; i <= N; i++)
        if(in[i] == 0)
            Q.push(i);

    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        for(int v : G[u]){
            if(l[u] != -1 && l[v] < l[u]+1){
                l[v] = l[u] + 1;
                p[v] = u;
            }
            in[v]--;
            if(in[v] == 0)
                Q.push(v);
        }
    }

    K = l[N] - l[1];
    printf("%d\n", K+1);
    for(int i = K, u = N; i >= 0; i--){
        ans[i] = u;
        u = p[u];
    }
    for(int i = 0; i <= K; i++)
        printf("%d%c", ans[i], (" \n")[i==K]);
}
```

Game Routes

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+1;
const ll MOD = 1e9+7;

int N, M, a, b, cnt, in[maxN];
ll dp[maxN];
vector G[maxN];
queue Q;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        in[b]++;
    }

    for(int i = 1; i <= N; i++)
        if(in[i] == 0)
            Q.push(i);

    dp[1] = 1;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        for(int v : G[u]){
            dp[v] = (dp[v] + dp[u]) % MOD;
            in[v]--;
            if(in[v] == 0)
                Q.push(v);
        }
    }

    printf("%lld\n", dp[N]);
}
```

Investigation

```
#include

using namespace std;
typedef long long ll;
typedef pair edge;
typedef pair node;
const int maxN = 1e5+1;
const ll MOD = 1e9+7;
const ll INF = 0x3f3f3f3f3f3f;

int N, M, a, b, minR[maxN], maxR[maxN];
ll c, dist[maxN], ways[maxN];
vector G[maxN];

struct Comparator {
    bool operator() (int x, int y) const {
        return dist[x] == dist[y] ? x < y : dist[x] < dist[y];
    }
}
```

```

    }
};
set PQ;

int main(){
    scanf("%d %d", &N, &M);
    memset(dist, 0x3f, sizeof(dist));
    for(int i = 0; i < M; i++){
        scanf("%d %d %lld", &a, &b, &c);
        G[a].push_back({b, c});
    }

    PQ.insert(1);
    ways[1] = 1;
    dist[1] = 0;
    while(!PQ.empty()){
        int u = *PQ.begin();
        ll d = dist[u];
        PQ.erase(PQ.begin());

        for(edge e : G[u]){
            int v = e.first;
            ll w = e.second;
            if(d + w <= dist[v]){
                if(dist[v] != INF) PQ.erase(v);
                if(d + w == dist[v]){
                    ways[v] = (ways[u] + ways[v]) % MOD;
                    minR[v] = min(minR[v], minR[u]+1);
                    maxR[v] = max(maxR[v], maxR[u]+1);
                }
                if(d + w < dist[v]){
                    dist[v] = d+w;
                    ways[v] = ways[u];
                    minR[v] = minR[u]+1;
                    maxR[v] = maxR[u]+1;
                }
                PQ.insert(v);
            }
        }
    }

    printf("%lld %lld %d %d\n", dist[N], ways[N], minR[N], maxR[N]);
}

```

Planets Queries I

```

#include

using namespace std;
const int maxN = 2e5+1;
const int logK = 31;

int N, Q, x, k, p[maxN][logK];

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++)
        scanf("%d", &p[i][0]);
    for(int j = 1; j < logK; j++)
        for(int i = 1; i <= N; i++)
            p[i][j] = p[p[i][j-1]][j-1];
}

```

```

for(int q = 0; q < Q; q++){
    scanf("%d %d", &x, &k);
    for(int i = 0; i < logK; i++)
        if(k&(1<

```

Planets Queries II

```

#include

using namespace std;
const int maxN = 2e5+1, logN = 20;

bool vis[maxN];
int N, Q, ds[maxN], d[maxN], comp[maxN], cyc[maxN], p[logN][maxN];
vector cycleSeeds, G[maxN];

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

void init_parents(){
    for(int i = 1; i < logN; i++)
        for(int j = 1; j <= N; j++)
            p[i][j] = p[i-1][p[i-1][j]];
}

int travel(int u, int k){
    int v = u;
    for(int i = logN-1; i >= 0; i--){
        if(k&(1<= 0; i--){
            int par = p[i][root];
            if(!cyc[par])
                root = par;
        }
    }
    root = p[0][root];

    return (d[a]-d[root]) + query(root, b);
}

int main(){
    scanf("%d %d", &N, &Q);
    fill(ds+1, ds+N+1, -1);
    for(int i = 1, x; i <= N; i++){
        scanf("%d", &x);
        p[0][i] = x;
        G[x].push_back(i);
        if(!merge(x, i))
            cycleSeeds.push_back(x);
    }
}

```

```

init_parents();
int compID = 1;
for(int seed : cycleSeeds)
    dfs(seed, compID++);

for(int i = 0, a, b; i < Q; i++){
    scanf("%d %d", &a, &b);
    printf("%d\n", query(a, b));
}
}

```

Planets Cycles

```

#include

using namespace std;
const int maxN = 2e5+1;

bool vis[maxN];
int N, t[maxN], in[maxN], ans[maxN];
vector G[maxN];
queue Q;

void dfs1(int u){
    for(int v : G[u]){
        if(!vis[v]){
            ans[v] = ans[u]+1;
            vis[v] = true;
            dfs1(v);
        }
    }
}

void dfs2(int u, int d = 1){
    vis[u] = true;
    int v = t[u];
    if(vis[v]){
        ans[u] = d;
    } else {
        dfs2(v, d+1);
        ans[u] = ans[v];
    }
    dfs1(u);
}

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%d", &t[i]);
        G[t[i]].push_back(i);
        in[t[i]]++;
    }

    for(int i = 1; i <= N; i++)
        if(in[i] == 0)
            Q.push(i);

    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        int v = t[u];
    }
}

```

```

        in[v]--;
        if(in[v] == 0)
            Q.push(v);
    }

    for(int i = 1; i <= N; i++)
        if(in[i] && !vis[i])
            dfs2(i);

    for(int i = 1; i <= N; i++)
        printf("%d%c", ans[i], (" \n")[i==N]);
}

```

Road Reparation

```

#include

using namespace std;
#define f first
#define s second
typedef pair pii;
typedef pair edge;
const int maxN = 1e5+1;
const int maxM = 2e5+1;

int N, M, a, b, c, ds[maxN];
long long sum;
edge edges[maxM];

int find(int u){
    if(ds[u] < 0)    return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v)    return false;
    if(ds[u] < ds[v])    swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d", &N, &M);
    fill(ds+1, ds+N+1, -1);
    for(int i = 0; i < M; i++){
        scanf("%d %d %d", &a, &b, &c);
        edges[i] = {c, {a, b}};
    }
    sort(edges, edges+M);

    for(int i = 0; i < M; i++){
        edge e = edges[i];
        if(merge(e.s.f, e.s.s)){
            sum += e.f;
            N--;
        }
    }
}

```

```
    if(N == 1) printf("%lld\n", sum);
    else      printf("IMPOSSIBLE\n");
}
```

Road Construction

```
#include

using namespace std;
const int maxN = 1e5;

int N, M, a, b, components, largest, ds[maxN+1];

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d", &N, &M);
    fill(ds+1, ds+N+1, -1);

    components = N;
    largest = 1;
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        if(merge(a, b)){
            components--;
            largest = max(largest, -ds[find(a)]);
        }
        printf("%d %d\n", components, largest);
    }
}
```

Flight Routes Check

```
#include

using namespace std;
const int maxN = 2e5+1;

int N, M, a, b;
```

```

bool vis1[maxN], vis2[maxN];
vector G1[maxN], G2[maxN];

void dfs1(int u = 1, int p = 0){
    vis1[u] = true;
    for(int v : G1[u])
        if(v != p && !vis1[v])
            dfs1(v, u);
}

void dfs2(int u = 1, int p = 0){
    vis2[u] = true;
    for(int v : G2[u])
        if(v != p && !vis2[v])
            dfs2(v, u);
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G1[a].push_back(b);
        G2[b].push_back(a);
    }
    dfs1();
    dfs2();
    for(int i = 1; i <= N; i++){
        if(!vis1[i] || !vis2[i]){
            printf("NO\n");
            if(!vis1[i])    printf("1 %d\n", i);
            else            printf("%d 1\n", i);
            return 0;
        }
    }
    printf("YES\n");
}

```

Planets and Kingdoms

```

#include

using namespace std;
const int maxN = 1e5+1;

bool vis[maxN];
int N, M, rt[maxN];
vector ord, comp, G[maxN], GR[maxN];

void dfs1(int u){
    vis[u] = true;
    for(int v : G[u])
        if(!vis[v])
            dfs1(v);
    ord.push_back(u);
}

void dfs2(int u){
    vis[u] = true;
    comp.push_back(u);
    for(int v : GR[u])
        if(!vis[v])

```

```

        dfs2(v);
    }

    int main(){
        scanf("%d %d", &N, &M);
        for(int i = 0, a, b; i < M; i++){
            scanf("%d %d", &a, &b);
            G[a].push_back(b);
            GR[b].push_back(a);
        }

        for(int i = 1; i <= N; i++)
            if(!vis[i])
                dfs1(i);

        int K = 0;
        fill(vis+1, vis+N+1, false);
        reverse(ord.begin(), ord.end());
        for(int u : ord){
            if(!vis[u]){
                dfs2(u);
                K++;
                for(int v : comp)
                    rt[v] = K;
                comp.clear();
            }
        }

        printf("%d\n", K);
        for(int i = 1; i <= N; i++)
            printf("%d%c", rt[i], (" \n")[i==N]);
    }

```

Giant Pizza

```

#include

using namespace std;
const int maxN = 2e5+1;

bool vis[maxN];
char ans[maxN];
int N, M, K, in[maxN], rt[maxN];
vector ord, comp, G[maxN], GR[maxN], C[maxN], SCC[maxN];

int flip(int x){
    return (x&1 ? x+1 : x-1);
}

void add_edge(char c1, int a, char c2, int b){
    a = 2*a - (c1 == '-');
    b = 2*b - (c2 == '-');
    G[flip(a)].push_back(b);
    G[flip(b)].push_back(a);
    GR[a].push_back(flip(b));
    GR[b].push_back(flip(a));
}

void dfs1(int u){
    vis[u] = true;
    for(int v : G[u])

```

```

        if(!vis[v])
            dfs1(v);
    ord.push_back(u);
}

void dfs2(int u){
    vis[u] = true;
    comp.push_back(u);
    for(int v : GR[u])
        if(!vis[v])
            dfs2(v);
}

int main(){
    scanf("%d %d", &M, &N);
    for(int i = 0, a, b; i < M; i++){
        char c1, c2;
        scanf(" %c %d %c %d", &c1, &a, &c2, &b);
        add_edge(c1, a, c2, b);
    }

    for(int i = 1; i <= 2*N; i++)
        if(!vis[i])
            dfs1(i);

    fill(vis+1, vis+2*N+1, false);
    reverse(ord.begin(), ord.end());
    for(int u : ord){
        if(!vis[u]){
            dfs2(u);
            K++;
            for(int v : comp){
                rt[v] = K;
                C[K].push_back(v);
            }
            comp.clear();
        }
    }

    // Impossible iff x and not(x) belong to same SCC
    for(int i = 1; i <= N; i++){
        if(rt[2*i] == rt[2*i-1]){
            printf("IMPOSSIBLE\n");
            return 0;
        }
    }

    for(int u = 1; u <= 2*N; u++){
        for(int v : G[u]){
            if(rt[u] != rt[v]){
                SCC[rt[u]].push_back(rt[v]);
                in[rt[v]]++;
            }
        }
    }

    queue Q;
    ord.clear();
    for(int u = 1; u <= K; u++){
        if(in[u] == 0){
            ord.push_back(u);
            Q.push(u);
        }
    }

    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        for(int v : SCC[u]){
            in[v]--;
            if(in[v] == 0){

```

```

        ord.push_back(v);
        Q.push(v);
    }
}

fill(vis+1, vis+N+1, false);
reverse(ord.begin(), ord.end());
for(int k : ord){
    for(int u : C[k]){
        int i = (u+1)/2;
        if(!vis[i]){
            ans[i] = (u&1 ? '-' : '+');
            vis[i] = true;
        }
    }
}

for(int i = 1; i <= N; i++)
    printf("%c%c", ans[i], (" \n")[i==N]);
}

```

Coin Collector

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+1;

bool vis[maxN];
int N, M, K, rt[maxN];
ll k[maxN], ck[maxN], dp[maxN];
vector ord, comp, G[maxN], GR[maxN], SCC[maxN];

void dfs1(int u){
    vis[u] = true;
    for(int v : G[u])
        if(!vis[v])
            dfs1(v);
    ord.push_back(u);
}

void dfs2(int u){
    vis[u] = true;
    comp.push_back(u);
    for(int v : GR[u])
        if(!vis[v])
            dfs2(v);
}

void dfs3(int u){
    vis[u] = true;
    dp[u] = ck[u];
    for(int v : SCC[u]){
        if(!vis[v])
            dfs3(v);
        dp[u] = max(dp[u], dp[v]+ck[u]);
    }
}

```

```

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++){
        scanf("%d", &k[i]);

        for(int i = 0, a, b; i < M; i++){
            scanf("%d %d", &a, &b);
            G[a].push_back(b);
            GR[b].push_back(a);
        }

        for(int i = 1; i <= N; i++){
            if(!vis[i])
                dfs1(i);

            fill(vis+1, vis+N+1, false);
            reverse(ord.begin(), ord.end());
            for(int u : ord){
                if(!vis[u]){
                    dfs2(u);
                    K++;
                    for(int v : comp){
                        ck[K] += k[v];
                        rt[v] = K;
                    }
                    comp.clear();
                }
            }

            for(int u = 1; u <= N; u++){
                for(int v : G[u])
                    if(rt[v] != rt[u])
                        SCC[rt[u]].push_back(rt[v]);

                fill(vis+1, vis+K+1, false);
                for(int i = 1; i <= K; i++){
                    if(!vis[i])
                        dfs3(i);

                    ll best = 0;
                    for(int i = 1; i <= K; i++){
                        best = max(best, dp[i]);
                    }
                    printf("%lld\n", best);
                }
            }
        }
    }
}

```

Mail Delivery

```

#include

using namespace std;
typedef pair pii;
const int maxN = 1e5+1;
const int maxM = 2e5+1;

int N, M, deg[maxN];
bool tour_exists, used[maxM];
vector G[maxN];
vector tour;
stack S;

int main(){

```

```

scanf("%d %d", &N, &M);
for(int i = 0, a, b; i < M; i++){
    scanf("%d %d", &a, &b);
    G[a].push_back({b, i});
    G[b].push_back({a, i});
    deg[a]++; deg[b]++;
}

tour_exists = true;
for(int i = 1; i <= N; i++)
    if(deg[i]&1)
        tour_exists = false;

if(!tour_exists){
    printf("IMPOSSIBLE\n");
    return 0;
}

S.push(1);
while(!S.empty()){
    int u = S.top();
    if(deg[u]){
        while(!G[u].empty()){
            int v = G[u].back().first;
            int id = G[u].back().second;
            G[u].pop_back();

            if(!used[id]){
                deg[u]--; deg[v]--;
                used[id] = true;
                S.push(v);
                break;
            }
        }
    } else {
        tour.push_back(u);
        S.pop();
    }
}

// Should be M+1 nodes in the Euler tour
// If not, it means the graph was not connected
if((int) tour.size() != M+1)
    printf("IMPOSSIBLE\n");
else
    for(int i = 0; i <= M; i++)
        printf("%d%c", tour[i], (" \n")[i==M]);
}

```

De Bruijn Sequence

```

#include

using namespace std;
const int maxN = 15;
const int SIZE = (1< G[SIZE];
stack S;

void init(){
    for(int u = 0; u < 1<<(N-1); u++){
        int v = (u<<1)%(1<<(N-1));

```

```

        G[u].push_back(v);
        G[u].push_back(v|1);
    }
}

void solve(){
    S.push(0);
    while(!S.empty()){
        int u = S.top();
        if(!G[u].empty()){
            int v = G[u].back();
            G[u].pop_back();
            S.push(v);
        } else {
            ans[ansptr++] = (char) (u&1) + '0';
            S.pop();
        }
    }
    for(int i = 0; i < N-2; i++)
        ans[ansptr++] = '0';
}

int main(){
    scanf("%d", &N);
    if(N == 1){
        printf("01");
        return 0;
    }

    init();
    solve();
    for(int i = 0; i < ansptr; i++)
        printf("%c", ans[i]);
}

```

Teleporters Path

```

#include

using namespace std;
const int maxN = 1e5+1;

bool vis[maxN];
int N, M, in[maxN];
vector tour, G[maxN];
stack S;

void dfs(int u = 1, int p = -1){
    vis[u] = true;
    for(int v : G[u])
        if(v != p && !vis[v])
            dfs(v, u);
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        in[b]++;
    }
}

```

```

dfs();
bool exists = vis[N];
exists &= (in[1]+1 == (int) G[1].size());
exists &= (in[N]-1 == (int) G[N].size());
for(int i = 2; i < N; i++){
    exists &= (in[i] == (int) G[i].size());
    if(!vis[i]) exists &= (in[i] == 0 && G[i].size() == 0);
}

if(!exists){
    printf("IMPOSSIBLE\n");
    return 0;
}

S.push(1);
while(!S.empty()){
    int u = S.top();
    if(!G[u].empty()){
        int v = G[u].back();
        G[u].pop_back();
        S.push(v);
    } else {
        tour.push_back(u);
        S.pop();
    }
}

reverse(tour.begin(), tour.end());
for(int i = 0; i < M+1; i++)
    printf("%d%c", tour[i], (" \n")[i==M]);
}

```

Hamiltonian Flights

```

#include

using namespace std;
typedef long long ll;
typedef pair pii;
const int maxN = 20;
const int SIZE = 1< G[maxN];
queue Q;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a-1].push_back(b-1);
    }

    dp[0][1] = 1;
    Q.push({0, 1});
    inqueue[0][1] = true;
    while(!Q.empty()){
        int u = Q.front().first;
        int mask = Q.front().second;
        Q.pop();

        if(u != N-1){

```

```

    for(int v : G[u]){
        int newMask = mask | (1<

```

Download Speed

```

#include

using namespace std;
typedef long long ll;
typedef pair pil;
const int maxN = 501;
const ll INF = 0x3f3f3f3f3f3f3f;

int N, M, p[maxN];
ll cap[maxN][maxN];
vector G[maxN];

ll bfs(int s = 1, int t = N){
    fill(p+1, p+N+1, -1);
    p[s] = -2;

    queue Q;
    Q.push({s, INF});
    while(!Q.empty()){
        int u = Q.front().first;
        ll f = Q.front().second;
        Q.pop();

        for(int v : G[u]){
            if(p[v] == -1 && cap[u][v]){
                p[v] = u;
                ll aug = min(f, cap[u][v]);
                if(v == t) return aug;
                Q.push({v, aug});
            }
        }
    }

    return 0;
}

ll maxflow(int s = 1, int t = N){
    ll flow = 0, aug = 0;
    while(aug = bfs()){
        flow += aug;
        int u = t;
        while(u != s){
            int v = p[u];
            cap[v][u] -= aug;
            cap[u][v] += aug;
            u = v;
        }
    }
    return flow;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        ll c;
        scanf("%d %d %lld", &a, &b, &c);

```

```

    G[a].push_back(b);
    G[b].push_back(a);
    cap[a][b] += c;
}
printf("%lld\n", maxflow());
}

```

Police Chase

```

#include

using namespace std;
typedef pair pii;
const int maxN = 501;
const int INF = 0x3f3f3f3f;

int N, M, p[maxN], cap[maxN][maxN];
bool vis[maxN];
vector G[maxN];

int bfs(int s = 1, int t = N){
    fill(p+1, p+N+1, -1);
    p[s] = -2;

    queue Q;
    Q.push({s, INF});
    while(!Q.empty()){
        int u = Q.front().first;
        int f = Q.front().second;
        Q.pop();

        for(int v : G[u]){
            if(p[v] == -1 && cap[u][v]){
                p[v] = u;
                int aug = min(f, cap[u][v]);
                if(v == t) return aug;
                Q.push({v, aug});
            }
        }
    }

    return 0;
}

void dfs(int u = 1){
    vis[u] = true;
    for(int v : G[u])
        if(!vis[v] && cap[u][v])
            dfs(v);
}

int maxflow(int s = 1, int t = N){
    int flow = 0, aug = 0;
    while(aug = bfs()){
        flow += aug;
        int u = t;
        while(u != s){
            int v = p[u];
            cap[v][u] -= aug;
            cap[u][v] += aug;
            u = v;
        }
    }
}

```

```

    }
}
return flow;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
        cap[a][b]++;
        cap[b][a]++;
    }
    printf("%d\n", maxflow());

    dfs();
    for(int u = 1; u <= N; u++){
        if(!vis[u]) continue;
        for(int v : G[u])
            if(!vis[v])
                printf("%d %d\n", u, v);
    }
}

```

School Dance

```

#include

using namespace std;
typedef pair pii;
const int maxN = 505;

bool used[maxN];
int N, M, K, cnt, mt[maxN];
vector G[maxN];
vector pairs;

bool kuhns(int u){
    if(used[u]) return false;

    used[u] = true;
    for(int v : G[u]){
        if(!mt[v] || kuhns(mt[v])){
            mt[v] = u;
            return true;
        }
    }

    return false;
}

int main(){
    scanf("%d %d %d", &N, &M, &K);
    for(int i = 0, a, b; i < K; i++){
        scanf("%d %d", &a, &b);
        G[b].push_back(a);
    }

    for(int i = 1; i <= M; i++){
        fill(used+1, used+N+1, false);
    }
}

```

```

        kuhns(i);
    }

    cnt = 0;
    for(int i = 1; i <= N; i++){
        if(mt[i]){
            pairs.push_back({i, mt[i]});
            cnt++;
        }
    }

    printf("%d\n", cnt);
    for(pii P : pairs)
        printf("%d %d\n", P.first, P.second);
}

```

Distinct Routes

```

#include

using namespace std;
typedef pair pii;
const int maxN = 501, maxM = 1001;
const int INF = 0x3f3f3f3f;

int N, M, p[maxN], cap[maxN][maxN];
bool vis[maxM];
vector path, F[maxN];
vector G[maxN];

int bfs(int s = 1, int t = N){
    fill(p+1, p+N+1, -1);
    p[s] = -2;

    queue Q;
    Q.push({s, INF});
    while(!Q.empty()){
        int u = Q.front().first;
        int f = Q.front().second;
        Q.pop();

        for(int v : F[u]){
            if(p[v] == -1 && cap[u][v]){
                p[v] = u;
                int aug = min(f, cap[u][v]);
                if(v == t) return aug;
                Q.push({v, aug});
            }
        }
    }

    return 0;
}

void dfs(int u = 1){
    path.push_back(u);
    if(u == N) return;
    for(pii e : G[u]){
        int v = e.first;
        int id = e.second;
        if(cap[u][v] == 0 && !vis[id]){

```

```
        vis[id] = true;
        dfs(v);
        return;
    }
}

int maxflow(int s = 1, int t = N){
    int flow = 0, aug = 0;
    while(aug = bfs()){
        flow += aug;
        int u = t;
        while(u != s){
            int v = p[u];
            cap[v][u] -= aug;
            cap[u][v] += aug;
            u = v;
        }
    }
    return flow;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back({b, i});
        F[a].push_back(b);
        F[b].push_back(a);
        cap[a][b]++;
    }

    int K = maxflow();
    printf("%d\n", K);
    for(int k = 0; k < K; k++){
        path.clear();

        dfs();
        int sz = (int) path.size();
        printf("%d\n", sz);
        for(int i = 0; i < sz; i++)
            printf("%d%c", path[i], (" \n")[i==sz-1]);
    }
}
```

Range

Static Range Sum Queries

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, Q, a, b;
ll x[maxN+1];

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &x[i]);
        x[i] += x[i-1];
    }
    for(int q = 0; q < Q; q++){
        scanf("%d %d", &a, &b);
        printf("%lld\n", x[b]-x[a-1]);
    }
}
```

Static Range Minimum Queries

```
#include

using namespace std;
const int maxN = 2e5+1;
const int logN = 19;

int N, Q, a, b, l, k, x[maxN][logN];

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 0; i < N; i++)
        scanf("%d", &x[i][0]);

    for(int i = 1; i < logN; i++)
        for(int j = 0; j <= N-(1<
```

Dynamic Range Sum Queries

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, Q, t, a, b;
ll x[maxN+1], ds[maxN+1];

void update(int idx, ll val){
    ll diff = val - x[idx];
    for(int i = idx; i <= N; i += -i&i)
        ds[i] += diff;
    x[idx] = val;
}

ll query(int idx){
    ll sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += ds[i];
    return sum;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &x[0]);
        update(i, x[0]);
    }
    for(int q = 0; q < Q; q++){
        scanf("%d %d %d", &t, &a, &b);
        if(t == 1) update(a, b);
        else printf("%lld\n", query(b)-query(a-1));
    }
}

```

Dynamic Range Minimum Queries

```

#include

using namespace std;
const int maxN = 2e5, SIZE = 4*maxN;
const int INF = 0x3f3f3f3f;

int N, Q, t, a, b, lo[SIZE], hi[SIZE], mn[SIZE];

void push(int i){
}

void pull(int i){
    mn[i] = min(mn[2*i], mn[2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%d", &mn[i]);
        return;
    }
}

```

```

    int m = 1+(r-l)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void assign(int i, int l, int r, int v){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        mn[i] = v; return;
    }

    push(i);
    assign(2*i, l, r, v);
    assign(2*i+1, l, r, v);
    pull(i);
}

int minimum(int i, int l, int r){
    if(l > hi[i] || r < lo[i]) return INF;
    if(l <= lo[i] && hi[i] <= r) return mn[i];

    push(i);
    int lmin = minimum(2*i, l, r);
    int rmin = minimum(2*i+1, l, r);
    pull(i);

    return min(lmin, rmin);
}

int main(){
    scanf("%d %d", &N, &Q);

    init(1, 1, N);
    for(int q = 0; q < Q; q++){
        scanf("%d %d %d", &t, &a, &b);
        if(t == 1) assign(1, a, a, b);
        else if(t == 2) printf("%d\n", minimum(1, a, b));
    }
}

```

Range Xor Queries

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, Q, a, b;
ll x[maxN+1];

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &x[i]);
        x[i] ^= x[i-1];
    }
    for(int q = 0; q < Q; q++){
        scanf("%d %d", &a, &b);
        printf("%lld\n", x[b]^x[a-1]);
    }
}

```

```
}  
}
```

Range Update Queries

```
#include  
  
using namespace std;  
typedef long long ll;  
const int maxN = 2e5+1;  
  
int N, Q, t, a, b, k;  
ll x, ds[maxN];  
  
void update(int idx, ll val){  
    for(int i = idx; i <= N; i += -i&i)  
        ds[i] += val;  
}  
  
ll query(int idx){  
    ll sum = 0;  
    for(int i = idx; i > 0; i -= -i&i)  
        sum += ds[i];  
    return sum;  
}  
  
int main(){  
    scanf("%d %d", &N, &Q);  
    for(int i = 1; i <= N; i++){  
        scanf("%lld", &x);  
        update(i, x);  
        update(i+1, -x);  
    }  
    for(int q = 0; q < Q; q++){  
        scanf("%d", &t);  
        if(t == 1){  
            scanf("%d %d %lld", &a, &b, &x);  
            update(b+1, -x);  
            update(a, x);  
        } else if(t == 2){  
            scanf("%d", &k);  
            printf("%lld\n", query(k));  
        }  
    }  
}
```

Forest Queries

```
#include  
  
using namespace std;  
const int maxN = 1001;
```

```

char c;
int N, Q, a[maxN][maxN];

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        for(int j = 1; j <= N; j++){
            scanf(" %c", &c);
            a[i][j] = a[i-1][j]+a[i][j-1]-a[i-1][j-1];
            if(c == '*')    a[i][j]++;
        }
    }
    for(int q = 0; q < Q; q++){
        int x1, y1, x2, y2;
        scanf("%d %d %d %d", &x1, &y1, &x2, &y2);
        printf("%d\n", a[x2][y2]-a[x2][y1-1]-a[x1-1][y2]+a[x1-1][y1-1]);
    }
}

```

Hotel Queries

```

#include

using namespace std;
const int maxN = 2e5+1;

int N, M, h[maxN], k, ans;
int lo[4*maxN], hi[4*maxN], mx[4*maxN], mp[maxN];

void pull(int i){
    mx[i] = (h[mx[2*i]] >= h[mx[2*i+1]] ? mx[2*i] : mx[2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        mp[l] = i;
        mx[i] = l;
        return;
    }
    int m = l+(r-l)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void update(int i, int v){
    h[i] -= v;
    i = mp[i];
    i >>= 1;
    while(i > 0){
        pull(i);
        i >>= 1;
    }
}

int find(int i, int v){
    if(lo[i] == hi[i]) return lo[i];
    return (h[mx[2*i]] >= v ? find(2*i, v) : find(2*i+1, v));
}

```

```

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++){
        scanf("%d", &h[i]);
        init(1, 1, N);

        for(int i = 0; i < M; i++){
            scanf("%d", &k);
            ans = h[mx[1]] < k ? 0 : find(1, k);
            if(ans) update(ans, k);
            printf("%d%c", ans, (" \n")[i==M-1]);
        }
    }
}

```

List Removals

```

#include
#include
#include

using namespace std;
using namespace __gnu_pbds;
const int maxN = 2e5;

int N, p, x[maxN+1];
tree, rb_tree_tag,
tree_order_statistics_node_update> T;

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x[i]);
        T.insert(i);
    }

    for(int i = 0; i < N; i++){
        scanf("%d", &p);
        printf("%d%c", x[*T.find_by_order(p-1)], (" \n")[i==N-1]);
        T.erase(T.find_by_order(p-1));
    }
}

```

Salary Queries

```

#include
#include
#include

using namespace std;
using namespace __gnu_pbds;
typedef pair pii;

```

```

const int maxN = 2e5;

char c;
int N, Q, a, b, p[maxN+1];
tree, rb_tree_tag,
tree_order_statistics_node_update> T;

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%d", &p[i]);
        T.insert({p[i], i});
    }

    for(int i = 0; i < Q; i++){
        scanf(" %c %d %d", &c, &a, &b);
        if(c == '!'){
            T.erase({p[a], a});
            T.insert({b, a});
            p[a] = b;
        } else if(c == '?'){
            printf("%ld\n", T.order_of_key({b+1, 0}) - T.order_of_key({a, 0}));
        }
    }
}

```

Prefix Sum Queries

```

#include

using namespace std;
typedef long long ll;
typedef pair pll;
const int maxN = 2e5+1;
const int SIZE = 4*maxN;

int N, Q, lo[SIZE], hi[SIZE], mp[maxN];
ll sum[SIZE], pre[SIZE];

void pull(int i){
    pre[i] = max(pre[2*i], sum[2*i]+pre[2*i+1]);
    sum[i] = sum[2*i] + sum[2*i+1];
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%lld", &sum[i]);
        pre[i] = max(0LL, sum[i]);
        mp[l] = i;
        return;
    }
    int m = (l+r)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void update(int idx, int val){
    int i = mp[idx];
    sum[i] = val;
}

```

```

pre[i] = max(0LL, sum[i]);

i >>= 1;
while(i){
    pull(i);
    i >>= 1;
}

pll query(int i, int l, int r){
    if(l > hi[i] || r < lo[i])    return {0, 0};
    if(l <= lo[i] && hi[i] <= r) return {pre[i], sum[i]};

    pll left = query(2*i, l, r);
    pll right = query(2*i+1, l, r);
    return {max(left.first, left.second+right.first), left.second+right.second};
}

int main(){
    scanf("%d %d", &N, &Q);
    init(1, 1, N);
    for(int q = 0, t, a, b; q < Q; q++){
        scanf("%d %d %d", &t, &a, &b);
        if(t == 1)
            update(a, b);
        else if(t == 2)
            printf("%lld\n", query(1, a, b).first);
    }
}

```

Pizzeria Queries

```

#include

using namespace std;
const int maxN = 2e5+1, SIZE = 4*maxN;
const int INF = 0x3f3f3f3f;

int N, Q, p[maxN], mp[maxN], lo[SIZE], hi[SIZE], mn[2][SIZE];

void pull(int i){
    mn[0][i] = min(mn[0][2*i], mn[0][2*i+1]);
    mn[1][i] = min(mn[1][2*i], mn[1][2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%d", &p[l]);
        mn[0][i] = l+p[l];
        mn[1][i] = (N-l+1)+p[l];
        mp[l] = i;
        return;
    }
    int m = (l+r)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void update(int idx, int val){

```

```

    int i = mp[idx];
    mn[0][i] = idx+val;
    mn[1][i] = (N-idx+1)+val;
    i >>= 1;
    while(i){
        pull(i);
        i >>= 1;
    }
}

int query(int i, int t, int l, int r){
    if(l > hi[i] || r < lo[i]) return INF;
    if(l <= lo[i] && hi[i] <= r) return mn[t][i];

    int left = query(2*i, t, l, r);
    int right = query(2*i+1, t, l, r);
    return min(left, right);
}

int main(){
    scanf("%d %d", &N, &Q);
    init(1, 1, N);
    for(int q = 0, t, k, x; q < Q; q++){
        scanf("%d %d", &t, &k);
        if(t == 1){
            scanf("%d", &x);
            update(k, x);
        } else if(t == 2){
            int left = query(1, 1, 1, k);
            int right = query(1, 0, k, N);
            printf("%d\n", min(left-(N-k+1), right-k));
        }
    }
}

```

Range Interval Queries

```

#include

using namespace std;
typedef array operation;
const int maxN = 2e5+5;

int N, Q, ds[maxN], ans[maxN];
vector ops;

void update(int idx, int val){
    for(int i = idx; i <= N; i += -i&i)
        ds[i] += val;
}

int query(int idx){
    int sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += ds[i];
    return sum;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1, x; i <= N; i++){

```

```

scanf("%d", &x);
ops.push_back({x, 0, i, i, -1});
}
for(int q = 1, a, b, c, d; q <= Q; q++){
    scanf("%d %d %d %d", &a, &b, &c, &d);
    ops.push_back({c, -1, a, b, q});
    ops.push_back({d, 1, a, b, q});
}

sort(ops.begin(), ops.end());
for(operation op : ops){
    int type = op[1];
    if(type == 0)    update(op[2], 1);
    else {
        int a = op[2], b = op[3], qid = op[4];
        int cnt = query(b) - query(a-1);
        ans[qid] += type * cnt;
    }
}

for(int q = 1; q <= Q; q++)
    printf("%d\n", ans[q]);
}

```

Subarray Sum Queries

```

#include

using namespace std;
typedef long long ll;
const int SIZE = 8e5;

int N, Q, k, lo[SIZE], hi[SIZE], mp[SIZE];
ll x, sum[SIZE], dp[SIZE], dpl[SIZE], dpr[SIZE];

void pull(int i){
    dpl[i] = max(dpl[2*i], sum[2*i] + dpl[2*i+1]);
    dpr[i] = max(dpr[2*i+1], sum[2*i+1] + dpr[2*i]);
    dp[i] = max(dpr[2*i] + dpl[2*i+1], max(dp[2*i], dp[2*i+1]));
    sum[i] = sum[2*i] + sum[2*i+1];
}

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%lld", &sum[i]);
        dpl[i] = dpr[i] = dp[i] = sum[i];
        mp[l] = i;
        return;
    }
    int m = l+(r-l)/2;
    build(2*i, l, m);
    build(2*i+1, m+1, r);
    pull(i);
}

void update(int a, ll b){
    a = mp[a];

    dpl[a] = dpr[a] = dp[a] = sum[a] = b;
    a >= 1;
}

```

```

    while(a > 0){
        pull(a);
        a >>= 1;
    }
}

int main(){
    scanf("%d %d", &N, &Q);

    build(1, 1, N);
    for(int q = 0; q < Q; q++){
        scanf("%d %lld", &k, &x);
        update(k, x);
        printf("%lld\n", max(0LL, dp[1]));
    }
}

```

Subarray Sum Queries II

```

#include

using namespace std;
typedef long long ll;
typedef array result;
const ll INF = 0x3f3f3f3f3f3f3f;
const int SIZE = 8e5;

int N, Q, lo[SIZE], hi[SIZE];
ll sum[SIZE], dp[SIZE], dpl[SIZE], dpr[SIZE];

result operator+(const result &a, const result &b){
    return {
        a[0] + b[0],
        max(a[1], a[0] + b[1]),
        max(b[2], b[0] + a[2]),
        max(max(a[3], b[3]), a[2] + b[1]),
    };
}

void pull(int i){
    dpl[i] = max(dpl[2*i], sum[2*i] + dpl[2*i+1]);
    dpr[i] = max(dpr[2*i+1], sum[2*i+1] + dpr[2*i]);
    dp[i] = max(dpr[2*i] + dpl[2*i+1], max(dp[2*i], dp[2*i+1]));
    sum[i] = sum[2*i] + sum[2*i+1];
}

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%lld", &sum[i]);
        dpl[i] = dpr[i] = dp[i] = sum[i];
        return;
    }
    int m = l+(r-l)/2;
    build(2*i, l, m);
    build(2*i+1, m+1, r);
    pull(i);
}

result query(int i, int l, int r){
    if(lo[i] > r || hi[i] < l) return {-INF, -INF, -INF, -INF};
}

```

```

    if(lo[i] >= l && hi[i] <= r) return {sum[i], dpl[i], dpr[i], dp[i]};
    result left = query(2*i, l, r);
    result right = query(2*i+1, l, r);
    return left + right;
}

int main(){
    scanf("%d %d", &N, &Q);

    build(1, 1, N);
    for(int q = 0, l, r; q < Q; q++){
        scanf("%d %d", &l, &r);
        result res = query(1, l, r);
        printf("%lld\n", res[3] <= 0 ? 0 : res[3]);
    }
}

```

Distinct Values Queries

```

#include

using namespace std;
typedef array triple;
const int maxN = 2e5+1;

int N, Q, a, b, x[maxN], ptr[maxN], ds[maxN], ans[maxN];
triple queries[maxN];
map mp;
set S;

void update(int idx, int val){
    for(int i = idx; i <= N; i += -i&i)
        ds[i] += val;
}

int query(int idx){
    int sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += ds[i];
    return sum;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x[i]);

        for(int i = N; i > 0; i--){
            if(mp[x[i]])
                ptr[i] = mp[x[i]];
            mp[x[i]] = i;
        }

        for(int i = 1; i <= N; i++){
            if(S.count(x[i]) == 0)
                update(i, 1);
            S.insert(x[i]);
        }

        for(int i = 0; i < Q; i++){
            scanf("%d %d", &a, &b);

```

```

        queries[i] = {a, b, i};
    }
    sort(queries, queries+Q);

    int l = 1;
    for(int q = 0; q < Q; q++){
        while(l < queries[q][0]){
            if(ptr[l])
                update(ptr[l], 1);
            l++;
        }
        ans[queries[q][2]] = query(queries[q][1]) - query(queries[q][0]-1);
    }

    for(int q = 0; q < Q; q++)
        printf("%d\n", ans[q]);
}

```

Distinct Values Queries II

```

#include

using namespace std;
typedef pair pii;
const int maxN = 2e5, SIZE = 4*maxN;
const int INF = 0x3f3f3f3f;

int N, Q, a[maxN+1];
int lo[SIZE], hi[SIZE], mn[SIZE], mx[SIZE];

unordered_map>> prv;
unordered_map> nxt;
unordered_map segtree_idx;

void pull(int i){
    mn[i] = min(mn[2*i], mn[2*i+1]);
    mx[i] = max(mx[2*i], mx[2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        int v = a[l];
        mn[i] = nxt[v].upper_bound(l) == nxt[v].end() ? INF : *nxt[v].upper_bound(l);
        mx[i] = prv[v].upper_bound(l) == prv[v].end() ? -INF : *prv[v].upper_bound(l);
        segtree_idx[l] = i;
        return;
    }

    int m = l+(r-l)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void assign(int i, int _mn, int _mx){
    i = segtree_idx[i];
    if(_mn != -1)    mn[i] = _mn;
    if(_mx != -1)    mx[i] = _mx;
    while(i > 1){
        i >>= 1;
    }
}

```

```

        pull(i);
    }
}

pii query(int i, int l, int r){
    if(l > hi[i] || r < lo[i])    return {INF, -INF};
    if(l <= lo[i] && hi[i] <= r)    return {mn[i], mx[i]};

    pii lmin = query(2*i, l, r);
    pii rmin = query(2*i+1, l, r);

    return {min(lmin.first, rmin.first), max(lmin.second, rmin.second)};
}

void update(int pos, int val){
    int old_val = a[pos];
    int old_next = nxt[old_val].upper_bound(pos) == nxt[old_val].end() ? INF : *nxt[old_val].upper_bound(pos);
    int old_prev = prv[old_val].upper_bound(pos) == prv[old_val].end() ? -INF : *prv[old_val].upper_bound(pos);

    prv[old_val].erase(pos);
    nxt[old_val].erase(pos);
    if(old_next != INF)    assign(old_next, -1, old_prev);
    if(old_prev != -INF)    assign(old_prev, old_next, -1);

    a[pos] = val;
    int new_next = nxt[val].upper_bound(pos) == nxt[val].end() ? INF : *nxt[val].upper_bound(pos);
    int new_prev = prv[val].upper_bound(pos) == prv[val].end() ? -INF : *prv[val].upper_bound(pos);
    prv[val].insert(pos);
    nxt[val].insert(pos);

    assign(pos, new_next, new_prev);
    if(new_next != INF)    assign(new_next, -1, pos);
    if(new_prev != -INF)    assign(new_prev, pos, -1);
}

int main(){
    scanf("%d %d", &N, &Q);

    segtree_idx.reserve(N);
    prv.reserve(N + Q);
    nxt.reserve(N + Q);

    for(int i = 1; i <= N; i++){
        scanf("%d", &a[i]);
        prv[a[i]].insert(i);
        nxt[a[i]].insert(i);
    }

    init(1, 1, N);
    for(int q = 0, k, x, y; q < Q; q++){
        scanf("%d %d %d", &k, &x, &y);
        if(k == 1)    update(x, y);
        else {
            pii res = query(1, x, y);
            int mn_val = res.first, mx_val = res.second;
            printf("%s\n", (mn_val <= y || mx_val >= x) ? "NO" : "YES");
        }
    }
}

```

Increasing Array Queries

```

#include

using namespace std;
typedef long long ll;
typedef pair pii;
const int maxN = 2e5+1;
const int SIZE = 4*maxN;

int N, Q, lo[SIZE], hi[SIZE];
ll a[maxN], ass[SIZE], mx[SIZE], sm[SIZE], ans[maxN];
vector queries[maxN];

int len(int i){
    return hi[i]-lo[i]+1;
}

void assign(int i, ll val){
    ass[i] = mx[i] = val;
    sm[i] = val * len(i);
}

void push(int i){
    if(ass[i]){
        assign(2*i, ass[i]);
        assign(2*i+1, ass[i]);
        ass[i] = 0;
    }
}

void pull(int i){
    sm[i] = sm[2*i] + sm[2*i+1];
    mx[i] = max(mx[2*i], mx[2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        sm[i] = a[l];
        return;
    }
    int m = (l+r)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void update(int i, int l, int r, ll val){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        assign(i, val);
        return;
    }

    push(i);
    update(2*i, l, r, val);
    update(2*i+1, l, r, val);
    pull(i);
}

int orderOf(int i, int l, int val){
    if(lo[i] == hi[i]) return lo[i];
    push(i);
    int idx = -1;
    if(hi[2*i] <= l || mx[2*i] < val) idx = orderOf(2*i+1, l, val);
    else idx = orderOf(2*i, l, val);
    pull(i);
    return idx;
}

```

```

ll sum(int i, int l, int r){
    if(l > hi[i] || r < lo[i])    return 0;
    if(l <= lo[i] && hi[i] <= r)    return sm[i];

    push(i);
    ll left = sum(2*i, l, r);
    ll right = sum(2*i+1, l, r);
    pull(i);

    return left+right;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &a[i]);

    init(1, 1, N);
    for(int q = 0, l, r; q < Q; q++){
        scanf("%d %d", &l, &r);
        queries[l].push_back({r, q});
    }

    for(int i = 2; i <= N; i++) a[i] += a[i-1];
    for(int l = N; l >= 1; l--){
        int val = a[l]-a[l-1];
        int modifyR = (mx[l] < val ? N+1 : orderOf(1, l, val));
        update(1, l, modifyR-1, val);
        for(pii q : queries[l]){
            int r = q.first;
            int id = q.second;
            ans[id] = sum(1, l, r) - (a[r]-a[l-1]);
        }
    }

    for(int i = 0; i < Q; i++)
        printf("%lld\n", ans[i]);
}

```

Movie Festival Queries

```

#include

using namespace std;
const int logN = 17, maxT = 1e6;

int N, Q, dp[maxT+1][logN+1];
vector movies[maxT+1];

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 0, a, b; i < N; i++){
        scanf("%d %d", &a, &b);
        movies[a].push_back(b);
    }

    priority_queue, greater> ends;
    for(int t = maxT; t >= 0; t--){
        for(int b : movies[t])
            ends.push(b);
    }
}

```

```

    dp[t][0] = ends.empty() ? maxT+1 : ends.top();
}
for(int k = 1; k <= logN; k++){
    for(int t = 0; t <= maxT; t++){
        dp[t][k] = dp[t][k-1] > maxT ? maxT+1 : dp[dp[t][k-1]][k-1];

    }

    for(int q = 0, a, b; q < Q; q++){
        scanf("%d %d", &a, &b);
        int ans = 0;
        for(int k = logN; k >= 0; k--){
            if(dp[a][k] <= b){
                a = dp[a][k];
                ans += (1<

```

Forest Queries II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1000;

int N, Q;
char c[maxN+1][maxN+1];
ll ds[maxN+1][maxN+1];

void update(int x, int y, ll val){
    for(int i = x; i <= N; i += -i&i)
        for(int j = y; j <= N; j += -j&j)
            ds[i][j] += val;
}

ll query(int x, int y){
    ll sum = 0;
    for(int i = x; i > 0; i -= -i&i)
        for(int j = y; j > 0; j -= -j&j)
            sum += ds[i][j];
    return sum;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        for(int j = 1; j <= N; j++){
            scanf(" %c", &c[i][j]);
            if(c[i][j] == '*')
                update(i, j, 1);
        }
    }
    for(int q = 0, t, x1, y1, x2, y2; q < Q; q++){
        scanf("%d %d %d", &t, &x1, &y1);
        if(t == 1){
            if(c[x1][y1] == '*'){
                update(x1, y1, -1);
                c[x1][y1] = '.';
            } else if(c[x1][y1] == '.'){
                update(x1, y1, 1);
                c[x1][y1] = '*';
            }
        } else if(t == 2){
            scanf("%d %d", &x2, &y2);

```

```

        printf("%lld\n", query(x2, y2)-query(x2,y1-1)-query(x1-1,y2)+query(x1-1,y1-1));
    }
}

```

Range Updates and Sums

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, Q, t, a, b, lo[4*maxN], hi[4*maxN];
ll x, del[4*maxN], ass[4*maxN], sum[4*maxN];

int len(int i){
    return hi[i]-lo[i]+1;
}

void increment(int i, ll v){
    del[i] += v;
    sum[i] += len(i) * v;
}

void assign(int i, ll v){
    ass[i] = v;
    del[i] = 0;
    sum[i] = len(i) * v;
}

void push(int i){
    if(ass[i]){
        assign(2*i, ass[i]);
        assign(2*i+1, ass[i]);
        ass[i] = 0;
    }
    if(del[i]){
        increment(2*i, del[i]);
        increment(2*i+1, del[i]);
        del[i] = 0;
    }
}

void pull(int i){
    sum[i] = sum[2*i] + sum[2*i+1];
}

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%lld", &sum[i]);
        return;
    }
    int m = l+(r-l)/2;
    build(2*i, l, m);
    build(2*i+1, m+1, r);
    pull(i);
}

void increment(int i, int l, int r, ll v){

```

```

    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        increment(i, v); return;
    }

    push(i);
    increment(2*i, l, r, v);
    increment(2*i+1, l, r, v);
    pull(i);
}

void assign(int i, int l, int r, ll v){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        assign(i, v); return;
    }

    push(i);
    assign(2*i, l, r, v);
    assign(2*i+1, l, r, v);
    pull(i);
}

ll query(int i, int l, int r){
    if(l > hi[i] || r < lo[i]) return 0;
    if(l <= lo[i] && hi[i] <= r){
        return sum[i];
    }

    push(i);
    ll lsum = query(2*i, l, r);
    ll rsum = query(2*i+1, l, r);
    pull(i);

    return lsum + rsum;
}

int main(){
    scanf("%d %d", &N, &Q);

    build(1, 1, N);
    for(int q = 0; q < Q; q++){
        scanf("%d %d %d", &t, &a, &b);
        if(t == 1){
            scanf("%lld", &x);
            increment(1, a, b, x);
        } else if(t == 2){
            scanf("%lld", &x);
            assign(1, a, b, x);
        } else if(t == 3){
            printf("%lld\n", query(1, a, b));
        }
    }
}

```

Polynomial Queries

```

#include

using namespace std;
typedef long long ll;

```

```

const int SIZE = 8e5;

int N, Q, t, a, b, lo[SIZE], hi[SIZE];
ll d1[SIZE], d2[SIZE], s1[SIZE], s2[SIZE];

int len(int i){
    return hi[i]-lo[i]+1;
}

ll sigma(int i){
    return ((ll) hi[i] * (hi[i]+1))/2 - ((ll) lo[i] * (lo[i]-1))/2;
}

void push(int i){
    if(d1[i]){
        d1[2*i] += d1[i];
        d1[2*i+1] += d1[i];
        d1[i] = 0;
    }
    if(d2[i]){
        d2[2*i] += d2[i];
        d2[2*i+1] += d2[i];
        d2[i] = 0;
    }
}

void pull(int i){
    ll l1 = s1[2*i] + (d1[2*i] * len(2*i));
    ll r1 = s1[2*i+1] + (d1[2*i+1] * len(2*i+1));
    s1[i] = l1 + r1;
    ll l2 = s2[2*i] + (d2[2*i] * sigma(2*i));
    ll r2 = s2[2*i+1] + (d2[2*i+1] * sigma(2*i+1));
    s2[i] = l2 + r2;
}

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%lld", &s1[i]);
        return;
    }
    int m = l+(r-l)/2;
    build(2*i, l, m);
    build(2*i+1, m+1, r);
    pull(i);
}

void increment(int type, int i, int l, int r, ll v){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        if(type == 1) d1[i] += v;
        if(type == 2) d2[i] += v;
        return;
    }

    push(i);
    increment(type, 2*i, l, r, v);
    increment(type, 2*i+1, l, r, v);
    pull(i);
}

ll sum(int type, int i, int l, int r){
    if(l > hi[i] || r < lo[i]) return 0;
    if(l <= lo[i] && hi[i] <= r){
        if(type == 1) return s1[i] + (d1[i] * len(i));
        if(type == 2) return s2[i] + (d2[i] * sigma(i));
        return 0;
    }

    push(i);

```

```

    ll lsum = sum(type, 2*i, l, r);
    ll rsum = sum(type, 2*i+1, l, r);
    pull(i);

    return lsum + rsum;
}

void update(int l, int r){
    increment(1, 1, l, r, -l+1);
    increment(2, 1, l, r, 1);
}

ll query(int l, int r){
    return sum(1, 1, l, r) + sum(2, 1, l, r);
}

int main(){
    scanf("%d %d", &N, &Q);

    build(1, 1, N);
    for(int q = 0; q < Q; q++){
        scanf("%d %d %d", &t, &a, &b);
        if(t == 1) update(a, b);
        else if(t == 2) printf("%lld\n", query(a, b));
    }
}

```

Range Queries and Copies

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

struct Node {
    Node *l, *r;
    int lo, hi;
    ll sum;
    Node(int low, int high, int val) : l(nullptr), r(nullptr), lo(low), hi(high), sum(val) {}
    Node(Node *left, Node *right) : l(left), r(right), sum(0) {
        if(left){
            sum += left->sum;
            lo = left->lo;
        }
        if(right){
            sum += right->sum;
            hi = right->hi;
        }
    }
};

int N, Q, type, k, a, b, x, cnt, t[maxN];
Node* seg[maxN];

Node* build(int l, int r){
    if(l == r) return new Node(l, r, t[l]);
    int m = (l+r)/2;
    return new Node(build(l, m), build(m+1, r));
}

```

```

Node* update(Node* node, int idx, int val){
    if(node->lo == node->hi) return new Node(idx, idx, val);
    int m = (node->lo+node->hi)/2;
    if(idx <= m) return new Node(update(node->l, idx, val), node->r);
    else return new Node(node->l, update(node->r, idx, val));
}

ll query(Node* node, int l, int r){
    if(l > node->hi || r < node->lo) return 0;
    if(l <= node->lo && node->hi <= r) return node->sum;
    return query(node->l, l, r) + query(node->r, l, r);
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%d", &t[i]);

        cnt = 1;
        seg[cnt++] = build(1, N);
        for(int q = 0; q < Q; q++){
            scanf("%d %d", &type, &k);
            if(type == 1){
                scanf("%d %d", &a, &x);
                seg[k] = update(seg[k], a, x);
            } else if(type == 2){
                scanf("%d %d", &a, &b);
                printf("%lld\n", query(seg[k], a, b));
            } else if(type == 3){
                seg[cnt++] = new Node(seg[k]->l, seg[k]->r);
            }
        }
    }
}

```

Missing Coin Sum Queries

```

#include

/* Fast Input and Output Template */
char _i[1<<24], _o[1<<10], __[20], _, _c; int _i0, _o0, _n;
#define readio() { fread(_i, 1, 1<<24, stdin); }
#define writeio() { fwrite(_o, 1, _o0, stdout); }
#define scanu(x) { for (x = _i[_i0++] & 15; 47 < (_ = _i[_i0++]); x = x * 10 + (_ & 15)); }
#define putnumu(x) { _ = 0; do __[_++] = x % 10 | '0'; while (x /= 10); while (_--> 0) _o[_o0++] = __[_]; }
#define putnl() { _o[_o0++] = '\n'; }
/* End of Template */

using namespace std;
typedef long long ll;
const int maxN = 2e5+2;
const int logC = 32, logN = 18;
const int INF = 0x3f3f3f3f;

int N, Q, mn[logC][logN][maxN];
ll pre[logC][maxN];

int layer(int x){
    return 31 - __builtin_clz(x);
}

ll sum(int l, int a, int b){

```

```
    return pre[l][b] - pre[l][a-1];  
}  
  
int minimum(int l, int a, int b){  
    a--; b--;  
    int len = b-a+1;  
    int k = (int) floor(log2(len));  
    return min(mn[l][k][a], mn[l][k][b-(1<= (1<
```

Tree

Subordinates

```
#include

using namespace std;
const int maxN = 2e5;

int N, p[maxN+1], sz[maxN+1];
vector G[maxN+1];

void dfs(int u){
    sz[u] = 1;
    for(int v : G[u]){
        dfs(v);
        sz[u] += sz[v];
    }
}

int main(){
    scanf("%d", &N);
    for(int i = 2; i <= N; i++){
        scanf("%d", &p[i]);
        G[p[i]].push_back(i);
    }

    dfs(1);

    for(int i = 1; i <= N; i++)
        printf("%d%c", sz[i]-1, (" \n")[i==N]);
}
```

Tree Matching

```
#include

using namespace std;
const int maxN = 2e5+1;

int N, a, b, cnt;
bool used[maxN];
vector G[maxN];

void dfs(int u = 1, int par = 0){
    for(int v : G[u])
        if(v != par)
            dfs(v, u);

    if(!used[par] && !used[u] && par != 0){
        used[par] = used[u] = true;
        cnt++;
    }
}
```

```

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }
    dfs();
    printf("%d\n", cnt);
}

```

Tree Diameter

```

#include

using namespace std;
const int maxN = 2e5+1;

int N, a, b, last, dist[maxN];
vector G[maxN];

void dfs(int u, int p = 0){
    for(int v : G[u]){
        if(v != p){
            dist[v] = dist[u]+1;
            dfs(v, u);
        }
    }
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs(1);
    for(int i = 1; i <= N; i++)
        if(dist[i] > dist[last])
            last = i;
    dist[last] = 0;
    dfs(last);
    for(int i = 1; i <= N; i++)
        if(dist[i] > dist[last])
            last = i;
    printf("%d\n", dist[last]);
}

```

Tree Distances I

```

#include

using namespace std;
const int maxN = 2e5+1;

int N, a, b, down[maxN], up[maxN], best[maxN];
vector G[maxN];

void dfs1(int u = 1, int p = 0){
    for(int v : G[u]){
        if(v != p){
            dfs1(v, u);
            down[u] = max(down[u], down[v]+1);
        }
    }
}

void dfs2(int u = 1, int p = 0){
    int first = 0, second = 0;
    for(int v : G[u]){
        if(v != p){
            if(down[v] >= down[first]){
                second = first;
                first = v;
            } else if(down[v] > down[second]){
                second = v;
            }
        }
    }

    for(int v : G[u]){
        if(v != p){
            up[v] = max(up[v], up[u]+1);
            if(v == first){
                if(second != 0){
                    up[v] = max(up[v], down[second]+2);
                }
            } else if(first != 0){
                up[v] = max(up[v], down[first]+2);
            }
        }
        dfs2(v, u);
    }

    best[u] = max(up[u], down[u]);
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs1();
    dfs2();

    for(int i = 1; i <= N; i++)
        printf("%d%c", best[i], (" \n")[i==N]);
}

```

Tree Distances II

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, a, b, sz[maxN];
ll down[maxN], up[maxN];
vector G[maxN];

void dfs1(int u = 1, int p = 0){
    sz[u] = 1;
    for(int v : G[u]){
        if(v != p){
            dfs1(v, u);
            sz[u] += sz[v];
            down[u] += down[v] + sz[v];
        }
    }
}

void dfs2(int u = 1, int p = 0){
    if(p != 0)
        up[u] = (up[p]+down[p]) + N - (2*sz[u]+down[u]);
    for(int v : G[u])
        if(v != p)
            dfs2(v, u);
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs1();
    dfs2();

    for(int i = 1; i <= N; i++)
        printf("%lld%c", down[i]+up[i], (" \n")[i==N]);
}
```

Company Queries I

```
#include

using namespace std;
const int maxN = 2e5 + 1;
const int logK = 20;

int N, Q, x, k, p[maxN][logK];

int main() {
    scanf("%d %d", &N, &Q);
    for (int i = 2; i <= N; i++)
```

```

    scanf("%d", &p[i][0]);
    for (int j = 1; j < logK; j++)
        for (int i = 1; i <= N; i++)
            p[i][j] = p[p[i][j - 1]][j - 1];

    for (int q = 0; q < Q; q++) {
        scanf("%d %d", &x, &k);
        for (int i = 0; i < logK; i++)
            if (k & (1 << i))
                x = p[x][i];
        printf("%d\n", x ? x : -1);
    }
}

```

Company Queries II

```

#include

using namespace std;
const int maxN = 2e5+1;
const int logN = 20;

int N, Q, e, a, b, p[maxN][logN];
int timer, in[maxN], out[maxN];
vector G[maxN];

void dfs(int u = 1, int par = 1){
    in[u] = ++timer;
    p[u][0] = par;
    for(int i = 1; i < logN; i++)
        p[u][i] = p[p[u][i-1]][i-1];
    for(int v : G[u])
        if(v != par)
            dfs(v, u);
    out[u] = ++timer;
}

bool ancestor(int u, int v){
    return in[u] <= in[v] && out[u] >= out[v];
}

int lca(int u, int v){
    if(ancestor(u, v)) return u;
    if(ancestor(v, u)) return v;
    for(int i = logN-1; i >= 0; i--)
        if(!ancestor(p[u][i], v))
            u = p[u][i];
    return p[u][0];
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 2; i <= N; i++){
        scanf("%d", &e);
        G[e].push_back(i);
        G[i].push_back(e);
    }

    dfs();

    for(int q = 0; q < Q; q++){

```

```

        scanf("%d %d", &a, &b);
        printf("%d\n", lca(a, b));
    }
}

```

Distance Queries

```

#include

using namespace std;
const int maxN = 2e5+1;
const int logN = 20;

int N, Q, a, b, p[maxN][logN];
int timer, d[maxN], in[maxN], out[maxN];
vector G[maxN];

void dfs(int u = 1, int par = 1){
    in[u] = ++timer;
    d[u] = d[par]+1;
    p[u][0] = par;
    for(int i = 1; i < logN; i++){
        p[u][i] = p[p[u][i-1]][i-1];
    }
    for(int v : G[u])
        if(v != par)
            dfs(v, u);
    out[u] = ++timer;
}

bool ancestor(int u, int v){
    return in[u] <= in[v] && out[u] >= out[v];
}

int lca(int u, int v){
    if(ancestor(u, v)) return u;
    if(ancestor(v, u)) return v;
    for(int i = logN-1; i >= 0; i--){
        if(!ancestor(p[u][i], v))
            u = p[u][i];
    }
    return p[u][0];
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs();

    for(int q = 0; q < Q; q++){
        scanf("%d %d", &a, &b);
        printf("%d\n", d[a] + d[b] - 2*d[lca(a, b)]);
    }
}

```

Counting Paths

```
#include

using namespace std;
const int maxN = 2e5+1;
const int logN = 20;

int N, M, a, b, sub[maxN], p[maxN][logN];
int timer, in[maxN], out[maxN];
vector G[maxN];

void dfs1(int u = 1, int par = 1){
    in[u] = ++timer;
    p[u][0] = par;
    for(int i = 1; i < logN; i++){
        p[u][i] = p[p[u][i-1]][i-1];
    }
    for(int v : G[u])
        if(v != par)
            dfs1(v, u);
    out[u] = ++timer;
}

void dfs2(int u = 1){
    for(int v : G[u]){
        if(v != p[u][0]){
            dfs2(v);
            sub[u] += sub[v];
        }
    }
}

bool ancestor(int u, int v){
    return in[u] <= in[v] && out[u] >= out[v];
}

int lca(int u, int v){
    if(ancestor(u, v)) return u;
    if(ancestor(v, u)) return v;
    for(int i = logN-1; i >= 0; i--){
        if(!ancestor(p[u][i], v))
            u = p[u][i];
    }
    return p[u][0];
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs1();

    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        int l = lca(a, b);
        sub[a]++; sub[b]++;
        sub[l]--;
        if(l != 1)
            sub[p[l][0]]--;
    }

    dfs2();

    for(int i = 1; i <= N; i++)
```

```

        printf("%d%c", sub[i], (" \n")[i==N]);
    }

```

Subtree Queries

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, Q, a, b, t, s, timer;
int in[maxN], out[maxN];
ll k[maxN], ds[2*maxN];
vector G[maxN];

void update(int idx, ll val){
    for(int i = idx; i < 2*maxN; i += i&-i)
        ds[i] += val;
}

ll query(int idx){
    ll sum = 0;
    for(int i = idx; i > 0; i -= i&-i)
        sum += ds[i];
    return sum;
}

void dfs(int u, int p = 0){
    in[u] = ++timer;
    for(int v : G[u])
        if(v != p)
            dfs(v, u);
    out[u] = ++timer;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++)
        scanf("%lld", &k[i]);

    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    timer = 0;
    dfs(1);

    for(int i = 1; i <= N; i++)
        update(out[i], k[i]);

    for(int q = 0; q < Q; q++){
        scanf("%d %d", &t, &s);
        if(t == 1){
            update(out[s], -k[s]);
            scanf("%lld", &k[s]);
            update(out[s], k[s]);
        } else if(t == 2){
            printf("%lld\n", query(out[s])-query(in[s]));
        }
    }
}

```

```

    }
}
}

```

Path Queries

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, Q, a, b, t, s;
int timer, in[maxN], out[maxN];
ll x, v[maxN], ds[2*maxN];
vector G[maxN];

void update(int idx, ll val){
    for(int i = idx; i <= timer; i += -i&i)
        ds[i] += val;
}

ll query(int idx){
    ll sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += ds[i];
    return sum;
}

void dfs(int u = 1, int p = 0){
    in[u] = ++timer;
    for(int c : G[u])
        if(c != p)
            dfs(c, u);
    out[u] = ++timer;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &v[i]);
        for(int i = 0; i < N-1; i++){
            scanf("%d %d", &a, &b);
            G[a].push_back(b);
            G[b].push_back(a);
        }
    }

    dfs();
    for(int i = 1; i <= N; i++){
        update(in[i], v[i]);
        update(out[i], -v[i]);
    }

    for(int q = 0; q < Q; q++){
        scanf("%d %d", &t, &s);
        if(t == 1){
            scanf("%lld", &x);
            update(in[s], -v[s]);
            update(out[s], v[s]);
            update(in[s], x);
            update(out[s], -x);
        }
    }
}

```

```

        v[s] = x;
    } else if(t == 2){
        printf("%lld\n", query(in[s]));
    }
}
}
}

```

Path Queries II

```

#include

/**
 * Link Cut Tree
 */

using namespace std;
const int maxN = 2e5+1;

int N, Q, k, x, y, root, idcounter;

struct Lazy {
    int a = 1, b = 0;
    bool empty(){
        return (a == 1 && b == 0);
    }
};

struct Node {
    Lazy tag;
    bool rev = false;
    Node *c[2] = {nullptr, nullptr}, *p = nullptr;
    int sz, val, sum, mn, mx, id = ++idcounter;

    Node(int v){
        sz = 1;
        val = sum = mn = mx = v;
    }

    void apply(Lazy other){
        mn = mn*other.a + other.b;
        mx = mx*other.a + other.b;
        val = val*other.a + other.b;
        sum = sum*other.a + sz*other.b;
        tag = {tag.a*other.a, tag.b*other.a+other.b};
    }

    void push(){
        if(rev){
            swap(c[0], c[1]);
            if(c[0]) c[0]->rev ^= true;
            if(c[1]) c[1]->rev ^= true;
            rev = false;
        }
        if(!tag.empty()){
            if(c[0]) c[0]->apply(tag);
            if(c[1]) c[1]->apply(tag);
            tag = Lazy();
        }
    }

    void pull(){

```

```

        sum = mn = mx = val;
        sz = 1;
        for(int i = 0; i < 2; i++){
            if(c[i]){
                mn = min(mn, c[i]->mn);
                mx = max(mx, c[i]->mx);
                sum += c[i]->sum;
                sz += c[i]->sz;
            }
        }
    } *LCT[maxN];

bool notRoot(Node *t){
    return t->p && (t->p->c[0] == t || t->p->c[1] == t);
}

void rotate(Node *t){
    Node *p = t->p;
    bool b = (p->c[0] == t);
    if((t->p = p->p) && notRoot(p)) t->p->c[(t->p->c[1] == p)] = t;
    if((p->c[!b]=t->c[b])) p->c[!b]->p = p;
    t->c[b] = p;
    p->p = t;
    p->pull();
}

void splay(Node *t){
    while(notRoot(t)){
        Node *p = t->p;
        p->push();
        t->push();
        rotate(t);
    }
    t->push();
    t->pull();
}

Node* access(Node *t){
    Node *last = nullptr;
    for(Node *u = t; u; u = u->p){
        splay(u);
        u->c[1] = last;
        last = u;
    }
    splay(t);
    return last;
}

void evert(Node *t){
    access(t);
    t->rev = true;
}

void link(Node *u, Node *v){
    evert(u);
    u->p = v;
}

void cut(Node *u, Node *v){
    evert(u);
    access(v);
    if(v->c[0]) v->c[0]->p = 0;
    v->c[0] = 0;
    v->pull();
}

Node* path(Node *u, Node *v){
    evert(u);

```

```

    access(v);
    return v;
}

Node* LCA(Node *u, Node *v){
    evert(LCT[root]);
    access(u);
    return access(v);
}

bool connected(Node *u, Node *v){
    path(u, v);
    while(v->c[0])
        v = v->c[0];
    return u == v;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x);
        LCT[i] = new Node(x);
    }
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &x, &y);
        link(LCT[x], LCT[y]);
    }

    root = 1;
    for(int i = 0; i < Q; i++){
        scanf("%d", &k);
        if(k == 1){
            scanf("%d %d", &x, &y);
            Node *p = path(LCT[x], LCT[x]);
            p->apply({0, y});
        } else if(k == 2){
            scanf("%d %d", &x, &y);
            Node *p = path(LCT[x], LCT[y]);
            printf("%d ", p->mx);
        }
    }
}

```

Distinct Colors

```

#include

using namespace std;
typedef array triple;
const int maxN = 2e5+1;

int N, a, b, c[maxN], x[2*maxN], ptr[2*maxN], ds[2*maxN], ans[maxN];
int timer, in[maxN], out[maxN];
triple queries[maxN];
vector G[maxN];
map mp;
set S;

void dfs(int u = 1, int p = 0){
    in[u] = ++ timer;
    for(int v : G[u])

```

```

        if(v != p)
            dfs(v, u);
        out[u] = ++timer;
    }

    void update(int idx, int val){
        for(int i = idx; i <= 2*N; i += -i&i)
            ds[i] += val;
    }

    int query(int idx){
        int sum = 0;
        for(int i = idx; i > 0; i -= -i&i)
            sum += ds[i];
        return sum;
    }

    int main(){
        scanf("%d", &N);
        for(int i = 1; i <= N; i++){
            scanf("%d", &c[i]);

            for(int i = 0; i < N-1; i++){
                scanf("%d %d", &a, &b);
                G[a].push_back(b);
                G[b].push_back(a);
            }
            dfs();

            for(int i = 1; i <= N; i++){
                x[in[i]] = x[out[i]] = c[i];

                for(int i = 2*N; i > 0; i--){
                    if(mp[x[i]])
                        ptr[i] = mp[x[i]];
                    mp[x[i]] = i;
                }

                for(int i = 1; i <= 2*N; i++){
                    if(S.count(x[i]) == 0)
                        update(i, 1);
                    S.insert(x[i]);
                }

                for(int i = 1; i <= N; i++){
                    queries[i-1] = {in[i], out[i], i-1};
                }
                sort(queries, queries+N);

                int l = 1;
                for(int i = 0; i < N; i++){
                    while(l < queries[i][0]){
                        if(ptr[l])
                            update(ptr[l], 1);
                        l++;
                    }
                    ans[queries[i][2]] = query(queries[i][1]) - query(queries[i][0]-1);
                }

                for(int i = 0; i < N; i++){
                    printf("%d%c", ans[i], (" \n")[i==N-1]);
                }
            }
        }
    }

```

Finding a Centroid

```
#include

using namespace std;
const int maxN = 2e5+5;

int N, a, b, p[maxN], sz[maxN];
vector G[maxN];

void dfs(int u = 1){
    sz[u] = 1;
    for(int v : G[u]){
        if(v != p[u]){
            p[v] = u;
            dfs(v);
            sz[u] += sz[v];
        }
    }
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs();

    for(int i = 1; i <= N; i++){
        bool centroid = true;

        if(p[i] != 0 && N-sz[i] > N/2)
            centroid = false;

        for(int v : G[i])
            if(v != p[i] && sz[v] > N/2)
                centroid = false;

        if(centroid){
            printf("%d\n", i);
            return 0;
        }
    }
}
```

Math

Josephus Queries

```
#include

using namespace std;
typedef pair pii;

int Q;
map f;

int solve(int N, int K){
    if(f[{N, K}]) return f[{N, K}];

    if(N == 1) return f[{N, K}] = 1;
    if(2*K <= N) return f[{N, K}] = 2*K;

    int newN = (N/2)+(N&1);
    int newK = K-N/2;

    if(N&1){
        int ans = solve(newN, newK);
        return f[{N, K}] = ((2*ans-3)+(N+1)) % (N+1);
    } else {
        return f[{N, K}] = 2*solve(newN, newK)-1;
    }
}

int main(){
    scanf("%d", &Q);
    for(int q = 0, N, K; q < Q; q++){
        scanf("%d %d", &N, &K);
        printf("%d\n", solve(N, K));
    }
}
```

Exponentiation

```
#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

int N;
ll A, B;

ll fastpow(ll a, ll b){
    ll res = 1;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
    }
}
```

```
        b >= 1;
    }
    return res;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%lld %lld", &A, &B);
        printf("%lld\n", fastpow(A, B));
    }
}
```

Exponentiation II

```
#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

int N;
ll A, B, C;

ll fastpow(ll a, ll b, ll mod){
    ll res = 1;
    while(b > 0){
        if(b&1)
            res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%lld %lld %lld", &A, &B, &C);
        printf("%lld\n", fastpow(A, fastpow(B, C, MOD-1), MOD));
    }
}
```

Counting Divisors

```
#include

using namespace std;
const int maxX = 1e6;

int N, x, d[maxX+1];
```

```

void init(){
    for(int i = 1; i <= maxX; i++)
        for(int j = i; j <= maxX; j += i)
            d[j]++;
}

int main(){
    scanf("%d", &N);

    init();
    for(int i = 0; i < N; i++){
        scanf("%d", &x);
        printf("%d\n", d[x]);
    }
}

```

Common Divisors

```

#include

using namespace std;
const int maxX = 1e6;

int N, a, cnt, d[maxX+1];

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d", &a);
        d[a]++;
    }
    for(int i = maxX; i >= 1; i--){
        cnt = 0;
        for(int j = i; j <= maxX; j += i)
            cnt += d[j];
        if(cnt >= 2){
            printf("%d\n", i);
            return 0;
        }
    }
}

```

Sum of Divisors

```

#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

ll N, sum;

```

```

11 sigma(ll x){
    return ((x%MOD)*((x+1)%MOD)/2)%MOD;
}

int main(){
    scanf("%lld", &N);
    for(ll l = 1; l <= N; l++){
        ll r = N/(N/l);
        sum += (N/l)%MOD*(sigma(r)-sigma(l-1)+MOD)%MOD;
        sum = (sum+MOD)%MOD;
        l = r;
    }
    printf("%lld\n", sum);
}

```

Divisor Analysis

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e5;
const ll MOD = 1e9+7;

int N;
ll x[maxN], k[maxN];
ll tau, sigma, pi, mu;

ll fastpow(ll a, ll b){
    ll res = 1;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%lld %lld", &x[i], &k[i]);

    tau = 1;
    for(int i = 0; i < N; i++)
        tau = (tau * (k[i]+1)) % MOD;

    sigma = 1;
    for(int i = 0; i < N; i++){
        ll numerator = (fastpow(x[i], k[i]+1)-1+MOD) % MOD;
        ll denominator = fastpow(x[i]-1, MOD-2);
        ll geoSum = numerator * denominator % MOD;
        sigma = (sigma * geoSum) % MOD;
    }

    pi = 1;
    mu = 1;
    for(int i = 0; i < N; i++){
        ll p = fastpow(x[i], k[i]*(k[i]+1)/2);
        mu = fastpow(mu, k[i]+1) * fastpow(p, pi) % MOD;
    }
}

```

```

        pi = (pi * (k[i]+1)) % (MOD-1);
    }

    printf("%lld %lld %lld\n", tau, sigma, mu);
}

```

Prime Multiples

```

#include

using namespace std;
typedef long long ll;
const double EPS = 0.001;
const int maxK = 20;

int K;
ll N, cnt, a[maxK];

int main(){
    scanf("%lld %d", &N, &K);
    for(int i = 0; i < K; i++)
        scanf("%lld", &a[i]);

    cnt = N;
    double RHS = log(N) + EPS;
    for(int mask = 0; mask < (1<

```

Counting Coprime Pairs

```

#include

using namespace std;
typedef long long ll;
const int maxX = 1e6+1;

ll ans;
int N, dp[maxX];
bool b[maxX];
vector primes;

void init(){
    fill(b+2, b+maxX, true);
    for(int i = 2; i*i < maxX; i++)
        if(b[i])
            for(int j = i*i; j < maxX; j += i)
                b[j] = false;
    for(int i = 2; i < maxX; i++)
        if(b[i])
            primes.push_back(i);
}

void compute(int x){

```

```

vector pf;
for(int p : primes){
    if(x == 1) break;
    else if(b[x]){
        pf.push_back(x);
        break;
    }

    if(x % p) continue;
    pf.push_back(p);
    while(x % p == 0)
        x /= p;
}

int K = (int) pf.size();
for(int mask = 0; mask < (1<

```

Next Prime

```

#include

using namespace std;
typedef long long ll;
const int sqrtN = 1e6+5;

bool prime[sqrtN];
vector p;

void init_primes(){
    for(int i = 2; i < sqrtN; i++) prime[i] = true;

    for(int i = 2; i * i <= sqrtN; i++)
        if(prime[i])
            for(int j = 2*i; j < sqrtN; j += i)
                prime[j] = false;

    for(int i = 2; i < sqrtN; i++)
        if(prime[i])
            p.push_back(i);
}

bool is_prime(ll n){
    for(int i = 0; i < (int) p.size() && p[i] < n; i++)
        if(n % p[i] == 0)
            return false;
    return true;
}

int main(){
    init_primes();

    int T;
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        ll n;
        scanf("%lld", &n);

        n++;
        while(!is_prime(n))
            n++;
    }
}

```

```

        printf("%lld\n", n);
    }
}

```

Binomial Coefficients

```

#include

using namespace std;
typedef long long ll;
const int maxA = 1e6;
const ll MOD = 1e9+7;

int N, a, b;
ll fact[maxA+1], inv[maxA+1];

ll inverse(ll x){
    ll res = 1;
    ll expo = MOD-2;
    while(expo > 0){
        if(expo&1)
            res = (res * x) % MOD;
        x = (x * x) % MOD;
        expo >>= 1;
    }
    return res;
}

void init(){
    fact[0] = inv[0] = 1;
    for(int i = 1; i <= maxA; i++){
        fact[i] = i * fact[i-1] % MOD;
        inv[i] = inverse(fact[i]);
    }
}

int main(){
    init();
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d %d", &a, &b);
        printf("%lld\n", fact[a] * inv[b] % MOD * inv[a-b] % MOD);
    }
}

```

Creating Strings II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e6+5;

```

```

const ll MOD = 1e9+7;

int N, freq[26];
char S[maxN];
ll fact[maxN], inv[maxN];

ll inverse(ll x){
    ll res = 1;
    ll expo = MOD-2;
    while(expo > 0){
        if(expo&1)
            res = (res * x) % MOD;
        x = (x * x) % MOD;
        expo >>= 1;
    }
    return res;
}

void init(){
    fact[0] = inv[0] = 1;
    for(int i = 1; i < maxN; i++){
        fact[i] = i * fact[i-1] % MOD;
        inv[i] = inverse(fact[i]);
    }
}

int main(){
    scanf("%s", S);
    N = (int) strlen(S);
    init();

    for(int i = 0; i < N; i++)
        freq[(int) (S[i]-'a')]++;

    ll ans = fact[N];
    for(int i = 0; i < 26; i++)
        ans = ans * inv[freq[i]] % MOD;
    printf("%lld\n", ans);
}

```

Distributing Apples

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e6;
const ll MOD = 1e9+7;

int N, M;
ll fact[maxN], inv[maxN];

ll inverse(ll x){
    ll res = 1;
    ll expo = MOD-2;
    while(expo > 0){
        if(expo&1)
            res = (res * x) % MOD;
        x = (x * x) % MOD;
        expo >>= 1;
    }
}

```

```

    return res;
}

void init(){
    fact[0] = inv[0] = 1;
    for(int i = 1; i < maxN; i++){
        fact[i] = i * fact[i-1] % MOD;
        inv[i] = inverse(fact[i]);
    }
}

ll choose(int n, int k){
    return fact[n] * inv[k] % MOD * inv[n-k] % MOD;
}

int main(){
    init();
    scanf("%d %d", &N, &M);
    printf("%lld\n", choose(N+M-1, M));
}

```

Christmas Party

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e6;
const ll MOD = 1e9+7;

int N;
ll dp[maxN+1];

void init(){
    dp[2] = 1;
    for(int i = 3; i <= maxN; i++)
        dp[i] = (i-1) * (dp[i-1] + dp[i-2]) % MOD;
}

int main(){
    init();
    scanf("%d", &N);
    printf("%lld\n", dp[N]);
}

```

Permutation Order

```

#include

using namespace std;
typedef long long ll;
const int maxN = 21;

```

```

ll fac[maxN];

void init(){
    fac[0] = 1;
    for(int i = 1; i < maxN; i++)
        fac[i] = fac[i-1] * i;
}

vector query_one(const vector& a, ll k){
    if(a.empty())    return {};

    int n = (int) a.size();
    int first_digit = (int) (k / fac[n-1]);

    ll new_k = k - (first_digit * fac[n-1]);
    vector new_a;
    for(int i = 0; i < n; i++)
        if(i != first_digit)
            new_a.push_back(a[i]);

    vector p = query_one(new_a, new_k);
    p.push_back(a[first_digit]);
    return p;
}

ll query_two(int n, const vector& p){
    ll k = 1;
    for(int i = 0; i < n; i++){
        ll place_value = fac[n-i-1];

        int num_smaller = 0;
        for(int j = 0; j < i; j++)
            if(p[j] < p[i])
                num_smaller++;

        int digit_in_place = p[i] - num_smaller - 1;

        k += place_value * digit_in_place;
    }
    return k;
}

int main(){
    init();

    int T;
    scanf("%d", &T);
    for(int t = 0, type, n; t < T; t++){
        scanf("%d %d", &type, &n);
        if(type == 1){
            ll k;
            scanf("%lld", &k);

            vector a;
            for(int i = 1; i <= n; i++)
                a.push_back(i);

            vector p = query_one(a, k-1);
            reverse(p.begin(), p.end());

            for(int i = 0; i < n; i++)
                printf("%d%c", p[i], (" \n")[i==n-1]);
        } else {
            vector p(n);
            for(int i = 0; i < n; i++)
                scanf("%d", &p[i]);

            printf("%lld\n", query_two(n, p));
        }
    }
}

```

```

    }
}

```

Permutation Rounds

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;
const ll MOD = (ll) 1e9+7;

int N, perm[maxN];
bool vis[maxN], is_prime[maxN];
vector primes;
map ans;

void init_primes(){
    for(int i = 2; i < maxN; i++) is_prime[i] = true;
    for(int i = 2; i < maxN; i++){
        if(is_prime[i]){
            primes.push_back(i);
            for(int j = 2 * i; j < maxN; j += i)
                is_prime[j] = false;
        }
    }
}

int dfs(int u){
    vis[u] = true;
    if(vis[perm[u]]) return 1;
    return dfs(perm[u]) + 1;
}

void prime_factor(int x){
    for(int i = 0; i < (int) primes.size() && primes[i] <= x; i++){
        int p = primes[i];
        int num_divisions = 0;
        while(x % p == 0){
            x /= p;
            num_divisions++;
        }

        if(num_divisions > 0)
            ans[p] = max(ans[p], num_divisions);
    }
}

int main(){
    init_primes();

    scanf("%d", &N);
    for(int i = 1; i <= N; i++)
        scanf("%d", &perm[i]);

    for(int u = 1; u <= N; u++){
        if(!vis[u]){
            int len = dfs(u);
            prime_factor(len);
        }
    }
}

```

```

ll prod = 1;
for(auto const& [prime, power] : ans)
    for(int i = 0; i < power; i++)
        prod = (prod * prime) % MOD;
printf("%lld\n", prod);
}

```

Bracket Sequences I

```

#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

ll N, numerator, denominator;

ll fastpow(ll a, ll b){
    ll res = 1;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

ll inverse(ll a){
    return fastpow(a, MOD-2);
}

int main(){
    scanf("%lld", &N);
    if(N&1) printf("0\n");
    else {
        numerator = 1;
        for(int i = 1; i <= N; i++)
            numerator = (numerator * i) % MOD;

        denominator = 1;
        for(int i = 1; i <= N/2; i++)
            denominator = (denominator * i) % MOD;
        denominator = (denominator * denominator) % MOD;
        denominator = (denominator * (N/2+1)) % MOD;

        printf("%lld\n", (numerator*inverse(denominator))%MOD);
    }
}

```

Bracket Sequences II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e6+5;
const ll MOD = 1e9+7;

char S[maxN];
int N, M, K;
ll tot, bad, fact[maxN], inv[maxN];

ll inverse(ll x){
    ll res = 1;
    ll b = MOD-2;
    while(b){
        if(b&1) res = (res * x) % MOD;
        x = (x * x) % MOD;
        b >>= 1;
    }
    return res;
}

ll choose(int x, int y){
    return (fact[x] * inv[y] % MOD) * inv[x-y] % MOD;
}

void init(){
    fact[0] = inv[0] = 1;
    for(int i = 1; i <= N; i++){
        fact[i] = (fact[i-1] * i) % MOD;
        inv[i] = (inv[i-1] * inverse(i)) % MOD;
    }
}

int main(){
    scanf("%d %s", &N, S);
    M = (int) strlen(S);
    init();

    int open = 0, closed = 0;
    for(int i = 0; i < M; i++){
        if(S[i] == '(') open++;
        else if(S[i] == ')') closed++;

        if(closed > open){
            printf("0\n");
            return 0;
        }
    }

    if(N&1 || open > N/2){
        printf("0\n");
        return 0;
    }

    tot = choose(N-open-closed, N/2-open);
    bad = choose(N-open-closed, N/2-open-1);
    printf("%lld\n", ((tot-bad)%MOD+MOD)%MOD);
}

```

Counting Necklaces

```
#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

int N, M;
ll ans;

ll fastpow(ll a, ll b){
    ll res = 1;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int k = 0; k < N; k++)
        ans = (ans + fastpow(M, __gcd(k, N))) % MOD;
    ans = (ans * fastpow(N, MOD-2)) % MOD;
    printf("%lld\n", ans);
}
```

Counting Grids

```
#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

ll N, A, B, C;

ll fastpow(ll a, ll b){
    ll res = 1;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

int main(){
    scanf("%lld", &N);
    A = N * N;
    B = (A+3*(N&1))/4;
    C = (A+(N&1))/2;
    printf("%lld\n", ((fastpow(2, A)+2*fastpow(2, B)+fastpow(2, C))*fastpow(4,MOD-2))%MOD);
}
```

Fibonacci Numbers

```
#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

ll N, x[2][2], y[2][2];

void mult(ll A[2][2], ll B[2][2]){
    ll C[2][2];
    memset(C, 0, sizeof(C));
    for(int i = 0; i < 2; i++){
        for(int j = 0; j < 2; j++){
            for(int k = 0; k < 2; k++){
                C[i][j] += A[i][k] * B[k][j];
                C[i][j] %= MOD;
            }
        }
    }
    for(int i = 0; i < 2; i++)
        for(int j = 0; j < 2; j++)
            A[i][j] = C[i][j];
}

int main(){
    x[0][1] = x[1][0] = x[1][1] = y[0][0] = y[1][1] = 1;

    scanf("%lld", &N);
    while(N){
        if(N&1)
            mult(y, x);
        mult(x, x);
        N >>= 1;
    }

    printf("%lld\n", y[0][1]);
}
```

Throwing Dice

```
#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

ll N, x[6][6], y[6][6];

void init(){
    for(int i = 0; i < 6; i++)
        x[0][i] = 1;
    for(int i = 0; i < 5; i++)
        x[i+1][i] = 1;
```

```

    for(int i = 0; i < 6; i++)
        y[i][i] = 1;
}

void mult(ll A[6][6], ll B[6][6]){
    ll C[6][6];
    memset(C, 0, sizeof(C));
    for(int i = 0; i < 6; i++){
        for(int j = 0; j < 6; j++){
            for(int k = 0; k < 6; k++){
                C[i][j] += A[i][k] * B[k][j];
                C[i][j] %= MOD;
            }
        }
    }
    for(int i = 0; i < 6; i++)
        for(int j = 0; j < 6; j++)
            A[i][j] = C[i][j];
}

int main(){
    init();

    scanf("%lld", &N);
    while(N){
        if(N&1)
            mult(y, x);
        mult(x, x);
        N >>= 1;
    }

    printf("%lld\n", y[0][0]);
}

```

Graph Paths I

```

#include

using namespace std;
typedef long long ll;
const int maxN = 100;
const ll MOD = 1e9+7;

int N, M, K, a, b;
ll X[maxN][maxN], Y[maxN][maxN];

void mult(ll A[maxN][maxN], ll B[maxN][maxN]){
    ll C[N][N];
    memset(C, 0, sizeof(C));
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            for(int k = 0; k < N; k++){
                C[i][j] += A[i][k] * B[k][j];
                C[i][j] %= MOD;
            }
        }
    }
    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            A[i][j] = C[i][j];
}

```

```

int main(){
    scanf("%d %d %d", &N, &M, &K);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        X[a-1][b-1]++;
    }
    for(int i = 0; i < N; i++)
        Y[i][i] = 1;

    while(K){
        if(K&1)
            mult(Y, X);
        mult(X, X);
        K >>= 1;
    }

    printf("%lld\n", Y[0][N-1]);
}

```

Graph Paths II

```

#include

using namespace std;
typedef unsigned long long ull;
const int maxN = 100;
const ull INF = 1e19;

int N, M, K, a, b;
ull c, X[maxN][maxN], Y[maxN][maxN];

void combine(ull A[maxN][maxN], ull B[maxN][maxN]){
    ull C[maxN][maxN];
    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            C[i][j] = INF;

    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            for(int k = 0; k < N; k++)
                if(A[i][k] != INF && B[k][j] != INF)
                    C[i][j] = min(C[i][j], A[i][k] + B[k][j]);

    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            A[i][j] = C[i][j];
}

int main(){
    scanf("%d %d %d", &N, &M, &K);

    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            X[i][j] = Y[i][j] = INF;
            Y[i][i] = 0;
        }
    }

    for(int i = 0; i < M; i++){
        scanf("%d %d %llu", &a, &b, &c);
        X[a-1][b-1] = min(X[a-1][b-1], c);
    }
}

```

```

    }

    while(K){
        if(K&1)
            combine(Y, X);
        combine(X, X);
        K >>= 1;
    }

    if(Y[0][N-1] == INF)    printf("-1\n");
    else                    printf("%llu\n", Y[0][N-1]);
}

```

Sum of Four Squares

```

#include

using namespace std;
typedef pair pii;
typedef array solution;
const int maxN = 1e7+1;

set squares;
map small_solutions;
bool prime[maxN];

void init_squares(){
    for(int i = 0; i * i <= maxN; i++)
        squares.insert(i * i);
}

void init_primes(){
    for(int i = 2; i < maxN; i++)    prime[i] = true;
    for(int i = 2; i * i < maxN; i++)
        if(prime[i])
            for(int j = 2*i; j < maxN; j += i)
                prime[j] = false;
}

void precompute_small(){
    for(int a = 0; a <= 100; a++)
        for(int b = 0; b <= 100; b++)
            for(int c = 0; c <= 20; c++)
                for(int d = 0; d <= 5; d++)
                    small_solutions[a*a + b*b + c*c + d*d] = {a, b, c, d};
}

// Returns a and b such that
// a and b have opposite parity
// and p := n - a^2 - b^2 is prime
pii find_ab(int n){
    int sqrtn = (int) sqrt(n);

    while(true){
        int a = (rand() % sqrtn) + 1;
        int b = (rand() % sqrtn) + 1;

        if((a+b) % 2 == 0)    a++;

        int p = n - a*a - b*b;
        if(p >= 2 && prime[p])

```

```

        return {a, b};
    }
}

// If prime p is congruent to 1 (mod 4),
// then there exists integers (c, d)
// such that  $c^2 + d^2 = p$ .
// Naively find such c and d.
pii find_cd(int p){
    for(int c = 0; c*c <= p; c++){
        int d_squared = p - c*c;
        if(squares.find(d_squared) != squares.end())
            return {c, (int) sqrt(d_squared)};
    }
    return {-1, -1};
}

solution solve(int n){
    if(small_solutions.find(n) != small_solutions.end()){
        return small_solutions[n];
    } else if(n == 0){
        return {0, 0, 0, 0};
    } else if(n % 4 == 0){
        solution sol = solve(n / 4);
        for(int i = 0; i < 4; i++) sol[i] *= 2;
        return sol;
    } else if(n % 4 == 1 || n % 4 == 3){
        solution sol = solve(2 * n);
        sort(sol.begin(), sol.end(), [](int a, int b){
            return (a % 2 > b % 2);
        });
        return {
            (sol[0] + sol[1]) / 2,
            (sol[0] - sol[1]) / 2,
            (sol[2] + sol[3]) / 2,
            (sol[2] - sol[3]) / 2,
        };
    } else {
        pii ab = find_ab(n);
        int a = ab.first, b = ab.second;
        int p = n - a*a - b*b;
        pii cd = find_cd(p);
        int c = cd.first, d = cd.second;
        return {a, b, c, d};
    }
}

int main(){
    init_squares();
    init_primes();
    precompute_small();

    int T;
    scanf("%d", &T);
    for(int t = 0, n; t < T; t++){
        scanf("%d", &n);
        solution sol = solve(n);
        for(int i = 0; i < 4; i++) sol[i] = abs(sol[i]);
        printf("%d %d %d %d\n", sol[0], sol[1], sol[2], sol[3]);
    }
}

```

Triangle Number Sums

```
#include

using namespace std;
typedef long long ll;
const ll maxN = (ll) 1e12;

unordered_set S;
vector nums;

bool is_two_sum(ll n){
    int l = 0, r = (int) nums.size() - 1;
    while(l < (int) nums.size() && r >= 0){
        ll cur = nums[l] + nums[r];
        if(cur == n) return true;
        else if(cur < n) l++;
        else r--;
    }
    return false;
}

int answer(ll n){
    if(S.find(n) != S.end()) return 1;
    else if(is_two_sum(n)) return 2;
    else return 3;
}

int main(){
    S.reserve((int) 1e6);
    for(ll i = 1, num = 1; num <= maxN; i++, num += i){
        nums.push_back(num);
        S.insert(num);
    }

    int T;
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        ll n;
        scanf("%lld", &n);
        printf("%d\n", answer(n));
    }
}
```

Dice Probability

```
#include

using namespace std;
const int maxN = 100;

int N, a, b;
double sum, dp[maxN+1][6*maxN+1];

int main(){
    scanf("%d %d %d", &N, &a, &b);
    dp[0][0] = 1;
    for(int i = 1; i <= N; i++)
```

```

    for(int j = 1; j <= 6*maxN; j++)
        for(int k = 1; k <= 6; k++)
            if(j-k >= 0)
                dp[i][j] += dp[i-1][j-k]/6;
    for(int i = a; i <= b; i++)
        sum += dp[N][i];
    printf("%.6f\n", sum);
}

```

Moving Robots

```

#include

using namespace std;
const int maxK = 101;
const int N = 8, SZ = N*N;

int K;
double expected, ans[SZ], dp[maxK][SZ];

int main(){
    scanf("%d", &K);
    for(int i = 0; i < SZ; i++)
        ans[i] = 1.0;

    for(int start = 0; start < SZ; start++){
        memset(dp, 0, sizeof(dp));
        dp[0][start] = 1.0;

        for(int k = 0; k < K; k++){
            for(int u = 0; u < SZ; u++){
                vector V;
                if(N <= u)          V.push_back(u-N);
                if(u < N*(N-1))    V.push_back(u+N);
                if(u % N != 0)     V.push_back(u-1);
                if(u % N != N-1)  V.push_back(u+1);

                for(int v : V)
                    dp[k+1][v] += (dp[k][u] / V.size());
            }
        }

        for(int u = 0; u < SZ; u++)
            ans[u] *= (1 - dp[K][u]);
    }

    for(int i = 0; i < SZ; i++)
        expected += ans[i];
    printf("%.6f\n", expected);
}

```

Candy Lottery

```
#include

using namespace std;

int N, K;
double ans, a, b;

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 1; i <= K; i++){
        a = b = 1.0;
        for(int j = 1; j <= N; j++){
            a *= (double) i / K;
            b *= (double) (i-1) / K;
        }
        ans += (a-b) * i;
    }
    printf("%.6f\n", ans);
}
```

Inversion Probability

```
#include

using namespace std;
typedef long long ll;
const int maxN = 101;

int N, r[maxN];
long double ans;

int f(int x){
    return x * (x-1) / 2;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d", &r[i]);
        for(int j = 0; j < i; j++){
            int tot = (r[j] <= r[i]) ? f(r[j]) : (f(r[i]) + (r[j]-r[i]) * r[i]);
            ans += (long double) tot / (r[i] * r[j]);
        }
    }
    printf("%.6Lf\n", ans);
}
```

Stick Game

```
#include
```

```

using namespace std;
const int maxN = 1e6;
const int maxK = 100;

int N, K, p[maxK];
bool dp[maxN+1];

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < K; i++)
        scanf("%d", &p[i]);

    for(int i = 1; i <= N; i++){
        for(int j = 0; j < K; j++){
            if(i-p[j] >= 0 && !dp[i-p[j]])
                dp[i] = true;
            printf("%c", ("LW")[dp[i]]);
        }
    }
}

```

Nim Game I

```

#include

using namespace std;

int T, N, x, xum;

int main(){
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        scanf("%d", &N);
        xum = 0;
        for(int i = 0; i < N; i++){
            scanf("%d", &x);
            xum ^= x;
        }
        printf("%s\n", (xum ? "first" : "second"));
    }
}

```

Nim Game II

```

#include

using namespace std;

int T, N, x, xum;

int main(){
    scanf("%d", &T);
    for(int t = 0; t < T; t++){

```

```

scanf("%d", &N);
xum = 0;
for(int i = 0; i < N; i++){
    scanf("%d", &x);
    xum ^= (x%4);
}
printf(xum ? "first\n" : "second\n");
}
}

```

Stair Game

```

#include

using namespace std;

int T, N, p, xum;

int main(){
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        scanf("%d", &N);
        xum = 0;
        for(int i = 0; i < N; i++){
            scanf("%d", &p);
            if(i%2)
                xum ^= p;
        }
        printf(xum ? "first\n" : "second\n");
    }
}

```

Grundy's Game

```

#include

using namespace std;
const int maxN = 1e6+1;

/**
 * A036685
 * Retrived from https://oeis.org/A036685
 */

int T, lose[42] = { 0, 1, 2, 4, 7, 10, 20, 23, 26, 50, 53, 270, 273, 276, 282, 285, 288, 316, 334, 337, 340, 346, 359, 362 };
bool b[maxN];

void init(){
    for(int x : lose)
        b[x] = true;
}

```

```
int main(){
    init();
    scanf("%d", &T);
    for(int t = 0, N; t < T; t++){
        scanf("%d", &N);
        printf("%s\n", b[N] ? "second" : "first");
    }
}
```

Another Game

```
#include

using namespace std;

int T, N, x;
bool even;

int main(){
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        scanf("%d", &N);
        even = true;
        for(int i = 0; i < N; i++){
            scanf("%d", &x);
            even &= !(x&1);
        }
        printf("%s\n", even ? "second" : "first");
    }
}
```

String

Word Combinations

```
#include

using namespace std;
typedef long long ll;
const int maxN = 5005, K = 26;
const ll MOD = 1e9+7;

struct Node {
    int next[K];
    bool leaf = false;
    Node(){ fill(next, next+K, -1); }
};

int N, M;
ll dp[maxN];
char S[maxN], word[maxN];
vector<Node> trie(1);

void add_word(){
    scanf("%s", word);
    int v = 0;
    M = (int) strlen(word);
    for(int i = 0; i < M; i++){
        int c = (int) (word[i] - 'a');
        if(trie[v].next[c] == -1){
            trie[v].next[c] = trie.size();
            trie.emplace_back();
        }
        v = trie[v].next[c];
    }
    trie[v].leaf = true;
}

int main(){
    scanf("%s %d", S, &N);
    for(int i = 0; i < N; i++)
        add_word();

    M = (int) strlen(S);
    dp[M] = 1;
    for(int i = M-1; i >= 0; i--){
        int v = 0;
        for(int j = i; j < M; j++){
            int c = (int) (S[j] - 'a');
            if(trie[v].next[c] == -1) break;
            v = trie[v].next[c];

            if(trie[v].leaf)
                dp[i] = (dp[i] + dp[j+1]) % MOD;
        }
    }
    printf("%lld\n", dp[0]);
}
```

String Matching

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e6+5, numP = 3;
const ll MOD = 1e9+7;
const ll prime[numP] = {31, 37, 43};

int N, M, cnt;
ll sh[numP], th[numP], pw[numP][maxN];
char S[maxN], T[maxN];

bool hashes_equal(){
    for(int p = 0; p < numP; p++){
        if(sh[p] != th[p])
            return false;
        return true;
    }
}

int main(){
    scanf("%s %s", S, T);
    N = (int) strlen(S); M = (int) strlen(T);

    if(M > N){
        printf("0\n");
        return 0;
    }

    for(int p = 0; p < numP; p++){
        pw[p][0] = 1;
        for(int i = 1; i < maxN; i++){
            pw[p][i] = (pw[p][i-1] * prime[p]) % MOD;
        }

        for(int i = 0; i < M; i++){
            for(int p = 0; p < numP; p++){
                sh[p] = (sh[p] + (S[i] - 'a' + 1) * pw[p][M-i-1]) % MOD;
                th[p] = (th[p] + (T[i] - 'a' + 1) * pw[p][M-i-1]) % MOD;
            }
        }

        if(hashes_equal()) cnt++;
        for(int i = M; i < N; i++){
            for(int p = 0; p < numP; p++){
                sh[p] = (sh[p] * prime[p] - (S[i-M] - 'a' + 1) * pw[p][M]) % MOD;
                sh[p] = (sh[p] + (S[i] - 'a' + 1) + MOD) % MOD;
            }
            if(hashes_equal()) cnt++;
        }
        printf("%d\n", cnt);
    }
}
```

Finding Borders

```
#include

using namespace std;
```

```

typedef long long ll;
const ll MOD = 1e9+7;
const ll p1 = 31;
const ll p2 = 37;
const int maxN = 1e6+5;

int N;
ll pow1[maxN], pow2[maxN], ph1, ph2, sh1, sh2;
char S[maxN];

int main(){
    scanf("%s", S);
    N = (int) strlen(S);

    pow1[0] = pow2[0] = 1;
    for(int i = 1; i < N; i++){
        pow1[i] = (pow1[i-1] * p1) % MOD;
        pow2[i] = (pow2[i-1] * p2) % MOD;
    }

    for(int i = 0; i < N-1; i++){
        int l = (S[i] - 'a' + 1);
        int r = (S[N-i-1] - 'a' + 1);

        ph1 = (ph1 + l * pow1[i]) % MOD;
        ph2 = (ph2 + l * pow2[i]) % MOD;
        sh1 = (sh1 * p1 + r) % MOD;
        sh2 = (sh2 * p2 + r) % MOD;

        if(ph1 == sh1 && ph2 == sh2)
            printf("%d ", i+1);
    }
}

```

Finding Periods

```

#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;
const ll p1 = 31, p2 = 37;
const int maxN = 1e6+5;

int N;
char S[maxN];
ll pow1[maxN], pow2[maxN], h1[maxN], h2[maxN];

void init(){
    pow1[0] = pow2[0] = 1;
    h1[0] = h2[0] = (int) (S[0] - 'a' + 1);
    for(int i = 1; i < maxN; i++){
        int c = (int) (S[i] - 'a' + 1);
        pow1[i] = (pow1[i-1] * p1) % MOD;
        pow2[i] = (pow2[i-1] * p2) % MOD;
        h1[i] = (h1[i-1] * p1 + c) % MOD;
        h2[i] = (h2[i-1] * p2 + c) % MOD;
    }
}

int main(){

```

```

scanf(" %s", S);
N = (int) strlen(S);
init();

for(int len = 1; len <= N; len++){
    ll hash1 = h1[len-1], hash2 = h2[len-1];
    for(int i = 0; i < (N/len)-1; i++){
        hash1 = (hash1 * pow1[len] + h1[len-1]) % MOD;
        hash2 = (hash2 * pow2[len] + h2[len-1]) % MOD;
    }
    hash1 = (hash1 * pow1[N%len] + h1[N%len-1]) % MOD;
    hash2 = (hash2 * pow2[N%len] + h2[N%len-1]) % MOD;
    if(hash1 == h1[N-1] && hash2 == h2[N-1]) printf("%d ", len);
}
}

```

Minimal Rotation

```

#include

using namespace std;
const int maxN = 2e6+5;

int N, F[maxN];
char S[maxN];

int main(){
    scanf("%s", S);

    N = (int) strlen(S);
    for(int i = 0; i < N; i++)
        S[i+N] = S[i];

    N = (int) strlen(S);
    fill(F, F+N, -1);

    int k = 0;
    for(int i = 1; i < N; i++){
        char c = S[i];
        int f = F[i-k-1];
        while(f != -1 && c != S[k+f+1]){
            if(c < S[k+f+1])
                k = i-f-1;
            f = F[f];
        }

        if(c != S[k+f+1]){
            if(c < S[k])
                k = i;
            F[i-k] = -1;
        } else F[i-k] = f+1;
    }

    for(int i = 0; i < N/2; i++)
        printf("%c", S[i+k]);
}

```

Longest Palindrome

```
#include

using namespace std;
const int maxN = 1e6+5;

char S[maxN];
int N, bestl, bestr, d1[maxN], d2[maxN];

int main(){
    scanf(" %s", S);
    N = (int) strlen(S);

    for(int i = 0, l = 0, r = -1; i < N; i++){
        int k = (i > r ? 1 : min(d1[l+r-i], r-i+1));
        while(0 <= i-k && i+k < N && S[i-k] == S[i+k]) k++;

        if(2*k-1 > bestr-bestl+1){
            bestl = i-k+1;
            bestr = i+k-1;
        }

        d1[i] = k--;
        if(i+k > r){
            l = i-k;
            r = i+k;
        }
    }

    for(int i = 0, l = 0, r = -1; i < N; i++){
        int k = (i > r ? 0 : min(d2[l+r-i+1], r-i+1));
        while(0 <= i-k-1 && i+k < N && S[i-k-1] == S[i+k]) k++;

        if(2*k > bestr-bestl+1){
            bestl = i-k;
            bestr = i+k-1;
        }

        d2[i] = k--;
        if(i+k > r){
            l = i-k-1;
            r = i+k;
        }
    }

    for(int i = bestl; i <= bestr; i++)
        printf("%c", S[i]);
}
```

All Palindromes

```
#include
using namespace std;

struct Node {
    int len;           // length of palindrome
    int link;          // suffix link
    array next; // transitions by char
}
```

```

Node(int l = 0) : len(l), link(0) { next.fill(-1); }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    if (!(cin >> s))
        return 0;
    int n = (int)s.size();

    vector tree;
    tree.reserve(n + 5);
    tree.emplace_back(-1); // node 0: length -1 (imaginary)
    tree.emplace_back(0); // node 1: length 0 (empty)
    tree[0].link = 0;
    tree[1].link = 0;

    int last = 1; // index of node representing longest suffix-palindrome so far
    vector ans(n);

    for (int pos = 0; pos < n; ++pos) {
        int c = s[pos] - 'a';
        int cur = last;
        // find largest suffix-palindrome of the current processed prefix that can
        // be extended
        while (true) {
            int curlen = tree[cur].len;
            if (pos - curlen - 1 >= 0 && s[pos - curlen - 1] == s[pos])
                break;
            cur = tree[cur].link;
        }

        if (tree[cur].next[c] != -1) {
            // palindrome already exists
            last = tree[cur].next[c];
        } else {
            // create new node
            int newIdx = (int)tree.size();
            tree.emplace_back(tree[cur].len + 2);
            tree[cur].next[c] = newIdx;

            if (tree[newIdx].len == 1) {
                // single character palindrome -> link to empty palindrome node
                tree[newIdx].link = 1;
            } else {
                // find suffix link for the new node
                int tmp = tree[cur].link;
                while (true) {
                    int tmpLen = tree[tmp].len;
                    if (pos - tmpLen - 1 >= 0 && s[pos - tmpLen - 1] == s[pos])
                        break;
                    tmp = tree[tmp].link;
                }
                tree[newIdx].link = tree[tmp].next[c];
            }
            last = newIdx;
        }

        ans[pos] = tree[last].len;
    }

    for (int i = 0; i < n; ++i) {
        if (i)
            cout << ' ';
        cout << ans[i];
    }
    cout << '\n';
}

```

```
    return 0;
}
```

Required Substring

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1005, maxM = 505;
const ll MOD = 1e9+7;

int N, M, best[26][maxM];
char S[maxN];
ll ans, dp[maxM][maxN];

ll pow26(ll b){
    ll a = 26;
    ll res = 1;
    while(b){
        if(b&1) res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

bool good(vector pre, int k){
    int SZ = (int) pre.size();
    for(int i = 0; i < SZ-k; i++){
        if(pre[i+k] != S[i])
            return false;
    }
    return true;
}

int main(){
    scanf("%d %s", &N, S);
    M = (int) strlen(S);
    if(M > N){
        printf("0\n");
        return 0;
    }

    for(int r = 0; r < M; r++){
        for(int c = 0; c < 26; c++){
            vector pre;
            for(int i = 0; i < r; i++){
                pre.push_back(S[i]);
                pre.push_back((char) (c+'A'));
            }

            for(int k = 0; k < r+1; k++){
                if(good(pre, k)){
                    best[c][r] = r-k+1;
                    break;
                }
            }
        }
    }

    dp[0][0] = 1;
    for(int i = 1; i <= N; i++)
```

```

    for(int j = 0; j < M; j++)
        for(int c = 0; c < 26; c++)
            dp[best[c][j]][i] = (dp[best[c][j]][i] + dp[j][i-1]) % MOD;

ans = pow26(N);
for(int i = 0; i < M; i++)
    ans = (ans - dp[i][N] + MOD) % MOD;
printf("%lld\n", ans);
}

```

Palindrome Queries

```

#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;
const ll p[2] = {31, 37};
const int maxN = 2e5+5;

char S[maxN];
int N, K;
ll pows[2][maxN], invs[2][maxN], F[2], B[2], ds[2][2][maxN];

ll inverse(ll x){
    ll res = 1;
    ll b = MOD-2;
    while(b){
        if(b&1) res = (res * x) % MOD;
        x = (x * x) % MOD;
        b >>= 1;
    }
    return res;
}

void update(int pt, int t, int idx, ll val){
    val %= MOD; val += MOD; val %= MOD;
    for(int i = idx+1; i < maxN; i += -i&i)
        ds[pt][t][i] = (ds[pt][t][i] + val) % MOD;
}

ll query(int pt, int t, int idx){
    if(idx < 0) return 0;
    ll sum = 0;
    for(int i = idx+1; i > 0; i -= -i&i)
        sum = (sum + ds[pt][t][i]) % MOD;
    return sum;
}

void init(){
    for(int t = 0; t < 2; t++){
        pows[t][0] = invs[t][0] = 1;
        for(int i = 1; i < maxN; i++){
            pows[t][i] = (pows[t][i-1] * p[t]) % MOD;
            invs[t][i] = inverse(pows[t][i]);
        }
        update(t, 0, 0, (int) (S[0]-'a'+1) * pows[t][N-1]);
        update(t, 1, 0, (int) (S[0]-'a'+1));
        for(int i = 1; i < N; i++){
            int c = (int) (S[i]-'a'+1);
            update(t, 0, i, c * pows[t][N-i-1]);
        }
    }
}

```

```

        update(t, 1, i, c * pows[t][i]);
    }
}

void modify(int a, char c){
    int newchar = (int) (c - 'a' + 1);
    int oldchar = (int) (S[a] - 'a' + 1);
    for(int t = 0; t < 2; t++){
        update(t, 0, a, -oldchar * pows[t][N-a-1]);
        update(t, 0, a, newchar * pows[t][N-a-1]);
        update(t, 1, a, -oldchar * pows[t][a]);
        update(t, 1, a, newchar * pows[t][a]);
    }
    S[a] = c;
}

bool palindrome(int a, int b){
    for(int t = 0; t < 2; t++){
        F[t] = (query(t, 0, b) - query(t, 0, a-1) + MOD) % MOD;
        F[t] = (F[t] * invs[t][N-b-1]) % MOD;
        B[t] = (query(t, 1, b) - query(t, 1, a-1) + MOD) % MOD;
        B[t] = (B[t] * invs[t][a]) % MOD;
    }
    return F[0] == B[0] && F[1] == B[1];
}

int main(){
    scanf("%d %d %s", &N, &K, S);

    init();
    for(int i = 0, t, a, b; i < K; i++){
        char c;
        scanf("%d", &t);
        if(t == 1){
            scanf("%d %c", &a, &c);
            modify(a-1, c);
        } else if(t == 2){
            scanf("%d %d", &a, &b);
            printf("%s\n", palindrome(a-1, b-1) ? "YES" : "NO");
        }
    }
}

```

Finding Patterns

```

#include

using namespace std;
const int maxN = 1e5+5;

struct Node {
    int len, link, cnt, firstpos;
    map nxt;
} node[2*maxN];

char S[maxN], T[5*maxN];
int N, M, K, sz, last;

void extend(char c){
    int cur = sz++;

```

```

node[cur].len = node[last].len + 1;
int p = last;
while(p != -1 && !node[p].nxt.count(c)){
    node[p].nxt[c] = cur;
    p = node[p].link;
}
if(p == -1){
    node[cur].link = 0;
} else {
    int q = node[p].nxt[c];
    if(node[p].len + 1 == node[q].len){
        node[cur].link = q;
    } else {
        int clone = sz++;
        node[clone].len = node[p].len + 1;
        node[clone].nxt = node[q].nxt;
        node[clone].link = node[q].link;
        while(p != -1 && node[p].nxt[c] == q){
            node[p].nxt[c] = clone;
            p = node[p].link;
        }
        node[q].link = node[cur].link = clone;
    }
}
last = cur;
}

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

bool query(){
    int u = 0;
    for(int i = 0; i < M; i++){
        char c = T[i];
        if(!node[u].nxt.count(c)) return false;
        else u = node[u].nxt[c];
    }
    return true;
}

int main(){
    scanf(" %s %d", S, &K);
    N = (int) strlen(S);
    init();

    for(int i = 0; i < N; i++)
        extend(S[i]);

    for(int i = 0; i < K; i++){
        scanf(" %s", T);
        M = (int) strlen(T);
        printf("%s\n", query() ? "YES" : "NO");
    }
}

```

Counting Patterns

```

#include

using namespace std;
const int maxN = 1e5+5;

struct Node {
    int len, link, cnt;
    map nxt;
} node[2*maxN];

char S[maxN], T[5*maxN];
int N, M, K, sz, last;

void extend(char c){
    int cur = sz++;
    node[cur].cnt = 1;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
    last = cur;
}

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void update_cnts(){
    vector states_by_len[sz];
    for(int i = 0; i < sz; i++){
        states_by_len[node[i].len].push_back(i);
    }
    for(int i = sz-1; i >= 0; i--){
        for(int u : states_by_len[i])
            if(node[u].link != -1)
                node[node[u].link].cnt += node[u].cnt;
    }
}

int query(){
    int u = 0;
    for(int i = 0; i < M; i++){
        char c = T[i];
        if(!node[u].nxt.count(c)) return 0;
        else u = node[u].nxt[c];
    }
    return node[u].cnt;
}

```

```

int main(){
    scanf(" %s %d", S, &K);
    N = (int) strlen(S);

    init();
    for(int i = 0; i < N; i++){
        extend(S[i]);
        update_cnts();
    }

    for(int i = 0; i < K; i++){
        scanf(" %s", T);
        M = (int) strlen(T);
        printf("%d\n", query());
    }
}

```

Pattern Positions

```

#include

using namespace std;
const int maxN = 1e5+5;

struct Node {
    int len, link, cnt, firstpos;
    map<int, int> nxt;
} node[2*maxN];

char S[maxN], T[5*maxN];
int N, M, K, sz, last;

void extend(char c){
    int cur = sz++;
    node[cur].cnt = 1;
    node[cur].firstpos = node[last].len;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            node[clone].firstpos = node[q].firstpos;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
}

```

```

    last = cur;
}

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void update_cnts(){
    vector states_by_len[sz];
    for(int i = 0; i < sz; i++){
        states_by_len[node[i].len].push_back(i);
    }
    for(int i = sz-1; i >= 0; i--){
        for(int u : states_by_len[i])
            if(node[u].link != -1)
                node[node[u].link].cnt += node[u].cnt;
    }
}

int query_cnt(){
    int u = 0;
    for(int i = 0; i < M; i++){
        char c = T[i];
        if(!node[u].nxt.count(c)) return 0;
        else u = node[u].nxt[c];
    }
    return node[u].cnt;
}

int query_idx(){
    int u = 0;
    for(int i = 0; i < M; i++){
        char c = T[i];
        if(!node[u].nxt.count(c)) return -1;
        else u = node[u].nxt[c];
    }
    return node[u].firstpos-M+2;
}

int main(){
    scanf(" %s %d", S, &K);
    N = (int) strlen(S);

    init();
    for(int i = 0; i < N; i++)
        extend(S[i]);
    update_cnts();

    for(int i = 0; i < K; i++){
        scanf(" %s", T);
        M = (int) strlen(T);
        printf("%d\n", query_idx());
    }
}

```

Distinct Substrings

```

#include

using namespace std;

```

```

typedef long long ll;
const int maxN = 1e5+5;

struct Node {
    ll dp;
    int len, link;
    map<int, int> nxt;
} node[2*maxN];

char S[maxN];
int N, sz, last;

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void extend(char c){
    int cur = sz++;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
    last = cur;
}

void calc(int u = 0){
    node[u].dp = 1;
    for(const auto& [c, v] : node[u].nxt){
        if(!node[v].dp) calc(v);
        node[u].dp += node[v].dp;
    }
}

int main(){
    scanf("%s", S);
    N = (int) strlen(S);

    init();
    for(int i = 0; i < N; i++)
        extend(S[i]);
    calc();

    printf("%lld\n", node[0].dp-1);
}

```

Distinct Subsequences

```
#include

using namespace std;
typedef long long ll;
const int maxN = 5e5+5;
const ll MOD = 1e9+7;

int N;
ll tot, dp[26];
char S[maxN];

int main(){
    scanf(" %s", S);
    N = (int) strlen(S);
    for(int i = 0; i < N; i++){
        int c = (int) (S[i] - 'a');
        dp[c]++;
        for(int j = 0; j < 26; j++)
            if(j != c)
                dp[c] = (dp[c] + dp[j]) % MOD;
    }

    for(int i = 0; i < 26; i++)
        tot = (tot + dp[i]) % MOD;
    printf("%lld\n", tot);
}
```

Repeating Substring

```
#include

using namespace std;
const int maxN = 1e5+5;

struct Node {
    int len, link, cnt, firstpos;
    map<int, int> nxt;
} node[2*maxN];

char S[maxN];
bool vis[2*maxN];
int N, sz, last, bestidx, bestlen;

void extend(char c){
    int cur = sz++;
    node[cur].cnt = 1;
    node[cur].firstpos = node[last].len;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
}
```

```

    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            node[clone].firstpos = node[q].firstpos;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
    last = cur;
}

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void update_cnts(){
    vector states_by_len[sz];
    for(int i = 0; i < sz; i++)
        states_by_len[node[i].len].push_back(i);
    for(int i = sz-1; i >= 0; i--)
        for(int u : states_by_len[i])
            if(node[u].link != -1)
                node[node[u].link].cnt += node[u].cnt;
}

void dfs(int u = 0){
    vis[u] = true;
    if(node[u].len > bestlen && node[u].cnt > 1 && u != 0){
        bestidx = node[u].firstpos - node[u].len + 1;
        bestlen = node[u].len;
    }
    for(const auto& [c, v] : node[u].nxt)
        if(!vis[v])
            dfs(v);
}

int main(){
    scanf(" %s", S);
    N = (int) strlen(S);

    init();
    for(int i = 0; i < N; i++)
        extend(S[i]);
    update_cnts();

    bestlen = -1;
    dfs();

    if(bestlen == -1) printf("-1\n");
    else {
        for(int i = 0; i < bestlen; i++)
            printf("%c", S[bestidx+i]);
    }
}

```

String Functions

```
#include

using namespace std;
const int maxN = 1e6+5;

char S[maxN];
int N, z[maxN], pi[maxN];

int main(){
    scanf("%s", S);
    N = (int) strlen(S);

    for(int i = 1, l = 0, r = 0; i < N; i++){
        if(i <= r) z[i] = min(r-i+1, z[i-l]);
        while(i+z[i] < N && S[z[i]] == S[i+z[i]]) z[i]++;
        if(i+z[i]-1 > r) l = i, r = i+z[i]-1;
    }
    for(int i = 0; i < N; i++)
        printf("%d%c", z[i], (" \n")[i==N-1]);

    for(int i = 1; i < N; i++){
        int j = pi[i-1];
        while(j > 0 && S[i] != S[j]) j = pi[j-1];
        if(S[i] == S[j]) j++;
        pi[i] = j;
    }
    for(int i = 0; i < N; i++)
        printf("%d%c", pi[i], (" \n")[i==N-1]);
}
```

String Transform

```
#include

using namespace std;
const int maxN = 1e6+5;

int N, nxt[maxN];
char S[maxN];

int main(){
    scanf("%s", S);
    N = (int) strlen(S);
    for(int i = 0; i < N; i++) nxt[i] = i;
    stable_sort(nxt, nxt+N, [](int a, int b){
        return S[a] < S[b];
    });

    int ptr = nxt[0];
    while(ptr != 0){
        ptr = nxt[ptr];
        printf("%c", S[ptr]);
    }
```

```
    }
}
```

Substring Order I

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+5;

struct Node {
    ll dp;
    int len, link;
    map nxt;
} node[2*maxN];

vector ans;
char S[maxN];
int N, sz, last;
ll K;

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void extend(char c){
    int cur = sz++;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
    last = cur;
}

void calc(int u = 0){
    node[u].dp = 1;
    for(const auto& [c, v] : node[u].nxt){
```

```

        if(!node[v].dp) calc(v);
        node[u].dp += node[v].dp;
    }
}

void dfs(int u, ll k){
    if(k < 0) return;
    for(const auto& [c, v] : node[u].nxt){
        if(node[v].dp <= k) k -= node[v].dp;
        else {
            ans.push_back(c);
            dfs(v, k-1);
            return;
        }
    }
}

int main(){
    scanf(" %s %lld", S, &K);
    N = (int) strlen(S);

    init();
    for(int i = 0; i < N; i++)
        extend(S[i]);
    calc();

    dfs(0, K-1);
    int M = (int) ans.size();
    for(int i = 0; i < M; i++)
        printf("%c", ans[i]);
    printf("\n");
}

```

Substring Order II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+5;

struct Node {
    ll dp;
    int len, cnt, link;
    map nxt;
} node[2*maxN];

vector ans;
char S[maxN];
int N, sz, last;
ll K;

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void extend(char c){
    int cur = sz++;

```

```

    node[cur].cnt = 1;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
    last = cur;
}

void update_cnts(){
    vector states_by_len[sz];
    for(int i = 0; i < sz; i++){
        states_by_len[node[i].len].push_back(i);
    }
    for(int i = sz-1; i >= 0; i--){
        for(int u : states_by_len[i])
            if(node[u].link != -1)
                node[node[u].link].cnt += node[u].cnt;
    }
}

void calc(int u = 0){
    node[u].dp = node[u].cnt;
    for(const auto& [c, v] : node[u].nxt){
        if(!node[v].dp) calc(v);
        node[u].dp += node[v].dp;
    }
}

void dfs(int u, ll k){
    if(k < 0) return;
    for(const auto& [c, v] : node[u].nxt){
        if(node[v].dp <= k) k -= node[v].dp;
        else {
            ans.push_back(c);
            dfs(v, k - node[v].dp);
            return;
        }
    }
}

int main(){
    scanf("%s %lld", S, &K);
    N = (int) strlen(S);

    init();
    for(int i = 0; i < N; i++)
        extend(S[i]);
    update_cnts();
    calc();

    dfs(0, K-1);
    int M = (int) ans.size();

```

```

    for(int i = 0; i < M; i++)
        printf("%c", ans[i]);
    printf("\n");
}

```

Substring Distribution

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+5;

struct Node {
    int len, link;
    map nxt;
} node[2*maxN];

char S[maxN];
bool vis[2*maxN];
int N, sz, last, dist[2*maxN];
ll ans[maxN];

void extend(char c){
    int cur = sz++;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
    last = cur;
}

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void bfs(int s = 0){
    queue Q;

```

```
vis[s] = true;
dist[s] = 0;
Q.push(s);
while(!Q.empty()){
    int u = Q.front();
    Q.pop();
    ans[dist[u]]++;
    ans[node[u].len+1]--;
    for(const auto& [c, v] : node[u].nxt){
        if(!vis[v]){
            dist[v] = dist[u]+1;
            vis[v] = true;
            Q.push(v);
        }
    }
}

int main(){
    scanf(" %s", S);
    N = (int) strlen(S);
    init();

    for(int i = 0; i < N; i++)
        extend(S[i]);

    bfs();
    for(int i = 1; i <= N; i++){
        ans[i] += ans[i-1];
        printf("%lld%c", ans[i], (" \n")[i==N]);
    }
}
```

Geometry

Point Location Test

```
#include

using namespace std;
typedef long long ll;

struct Point {
    int x, y;
    void read(){ scanf("%d %d", &x, &y); }
    Point operator +(const Point& b) const { return Point{x+b.x, y+b.y}; }
    Point operator -(const Point& b) const { return Point{x-b.x, y-b.y}; }
    ll operator *(const Point& b) const { return (ll) x * b.y - (ll) y * b.x; }
    void operator +=(const Point& b) { x += b.x; y += b.y; }
    void operator -=(const Point& b) { x -= b.x; y -= b.y; }
    void operator *=(const int k) { x *= k; y *= k; }

    ll cross(const Point& b, const Point& c) const {
        return (b - *this) * (c - *this);
    }
};

int T;
Point P[3];

int main(){
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        for(int i = 0; i < 3; i++)
            P[i].read();

        ll cross = P[0].cross(P[1], P[2]);
        if(cross < 0) printf("RIGHT\n");
        else if(cross > 0) printf("LEFT\n");
        else printf("TOUCH\n");
    }
}
```

Line Segment Intersection

```
#include

using namespace std;
typedef long long ll;

struct Point {
    int x, y;
    void read(){ scanf("%d %d", &x, &y); }
    Point operator +(const Point& b) const { return Point{x+b.x, y+b.y}; }
    Point operator -(const Point& b) const { return Point{x-b.x, y-b.y}; }
    ll operator *(const Point& b) const { return (ll) x * b.y - (ll) y * b.x; }
    void operator +=(const Point& b) { x += b.x; y += b.y; }
```

```

void operator -=(const Point &b) { x -= b.x; y -= b.y; }
void operator *=(const int k) { x *= k; y *= k; }

ll cross(const Point& b, const Point& c) const {
    return (b - *this) * (c - *this);
}
};

int T;
Point P[4];

void lineintersect(Point P1, Point P2, Point P3, Point P4){
    // Parallel case
    if((P2-P1) * (P4-P3) == 0){
        // Collinear case, check bounding boxes
        if(P1.cross(P2, P3) == 0){
            for(int i = 0; i < 2; i++){
                if(max(P1.x, P2.x) < min(P3.x, P4.x) || max(P1.y, P2.y) < min(P3.y, P4.y)){
                    printf("NO\n");
                    return;
                }
                swap(P1, P3);
                swap(P2, P4);
            }
            printf("YES\n");
            return;
        }

        // Non-collinear parallel lines never intersect
        printf("NO\n");
        return;
    }

    // Non-parallel case
    for(int i = 0; i < 2; i++){
        ll s1 = P1.cross(P2, P3);
        ll s2 = P1.cross(P2, P4);
        if((s1 < 0 && s2 < 0) || (s1 > 0 && s2 > 0)){
            printf("NO\n");
            return;
        }
        swap(P1, P3);
        swap(P2, P4);
    }

    printf("YES\n");
}

int main(){
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        for(int i = 0; i < 4; i++)
            P[i].read();
        lineintersect(P[0], P[1], P[2], P[3]);
    }
}

```

Polygon Area

```
#include
```

```
using namespace std;
typedef long long ll;
const int maxN = 1001;

int N;
ll x[maxN], y[maxN], ans;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%lld %lld", &x[i], &y[i]);
        x[N] = x[0]; y[N] = y[0];

        for(int i = 0; i < N; i++){
            ans += x[i] * y[i+1];
            ans -= y[i] * x[i+1];
        }
        printf("%lld\n", abs(ans));
    }
}
```

Point in Polygon

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1001;

struct Point {
    int x, y;
    void read(){ scanf("%d %d", &x, &y); }
    Point operator +(const Point& b) const { return Point{x+b.x, y+b.y}; }
    Point operator -(const Point& b) const { return Point{x-b.x, y-b.y}; }
    ll operator *(const Point& b) const { return (ll) x * b.y - (ll) y * b.x; }
    void operator +=(const Point& b) { x += b.x; y += b.y; }
    void operator -=(const Point& b) { x -= b.x; y -= b.y; }
    void operator *=(const int k) { x *= k; y *= k; }

    ll cross(const Point& b, const Point& c) const {
        return (b - *this) * (c - *this);
    }
};

int N, M;
Point P[maxN];

bool pointlineintersect(Point P1, Point P2, Point P3){
    if(P2.cross(P1, P3) != 0) return false;
    return (min(P2.x, P3.x) <= P1.x && P1.x <= max(P2.x, P3.x))
        && (min(P2.y, P3.y) <= P1.y && P1.y <= max(P2.y, P3.y));
}

void pointinpolygon(){
    int cnt = 0;
    bool boundary = false;
    for(int i = 1; i <= N; i++){
        int j = (i == N ? 1 : i+1);
        if(pointlineintersect(P[0], P[i], P[j]))
            boundary = true;

        if(P[i].x <= P[0].x && P[0].x < P[j].x && P[0].cross(P[i], P[j]) < 0) cnt++;
    }
}
```

```

        else if(P[j].x <= P[0].x && P[0].x < P[i].x && P[0].cross(P[j], P[i]) < 0) cnt++;
    }

    if(boundary) printf("BOUNDARY\n");
    else if(cnt&1) printf("INSIDE\n");
    else printf("OUTSIDE\n");
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++)
        P[i].read();
    for(int i = 0; i < M; i++){
        P[0].read();
        pointinpolygon();
    }
}

```

Polygon Lattice Points

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+5;

int N;
ll area, boundary, inside, x[maxN], y[maxN];

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%lld %lld", &x[i], &y[i]);
    x[N] = x[0]; y[N] = y[0];

    for(int i = 0; i < N; i++){
        area += x[i] * y[i+1];
        area -= y[i] * x[i+1];
    }
    area = abs(area);

    for(int i = 0; i < N; i++){
        if(x[i+1] == x[i]) boundary += abs(y[i+1]-y[i]);
        else if(y[i+1] == y[i]) boundary += abs(x[i+1]-x[i]);
        else boundary += __gcd(abs(x[i+1]-x[i]), abs(y[i+1]-y[i]));
    }

    printf("%lld %lld\n", (area+2-boundary)/2, boundary);
}

```

Minimum Euclidean Distance

```

#include

using namespace std;
typedef long long ll;
typedef pair pll;
const int maxN = 2e5+1;

int N;
ll best;
bool isLeft[maxN];
pll P[maxN];

ll square(ll x){
    return x*x;
}

ll dist(pll A, pll B){
    ll dx = A.first-B.first;
    ll dy = A.second-B.second;
    return square(dx) + square(dy);
}

void solve(vector PX, vector PY){
    int len = PX.size();
    if(len == 1) return;

    vector LX, LY, RX, RY;
    for(int i = 0; i < len; i++){
        if(i < len/2){
            LX.push_back(PX[i]);
            isLeft[PX[i]] = true;
        } else {
            RX.push_back(PX[i]);
            isLeft[PX[i]] = false;
        }
    }

    for(int id : PY){
        if(isLeft[id]) LY.push_back(id);
        else RY.push_back(id);
    }

    solve(LX, LY);
    solve(RX, RY);

    ll midX = P[LX.back()].first;
    vector stripe;
    for(int id : PY)
        if(square(P[id].first-midX) < best)
            stripe.push_back(id);

    for(int i = 0; i < (int) stripe.size(); i++)
        for(int j = i+1; j < (int) stripe.size() && square(P[stripe[i]].second-P[stripe[j]].second) < best; j++)
            best = min(best, dist(P[stripe[i]], P[stripe[j]]));
}

int main(){
    scanf("%d", &N);
    vector sortedX, sortedY;
    for(int i = 0, x, y; i < N; i++){
        scanf("%d %d", &x, &y);
        P[i] = {x, y};
        sortedX.push_back(i);
        sortedY.push_back(i);
    }
    sort(sortedX.begin(), sortedX.end(), [](int A, int B){
        return P[A].first < P[B].first;
    });
    sort(sortedY.begin(), sortedY.end(), [](int A, int B){

```

```

        return P[A].second < P[B].second;
    });

    best = LLONG_MAX;
    solve(sortedX, sortedY);
    printf("%lld\n", best);
}

```

Convex Hull

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+5;

struct Point {
    int x, y;
    void read(){ scanf("%d %d", &x, &y); }
    Point operator +(const Point& b) const { return Point{x+b.x, y+b.y}; }
    Point operator -(const Point& b) const { return Point{x-b.x, y-b.y}; }
    ll operator *(const Point& b) const { return (ll) x * b.y - (ll) y * b.x; }
    bool operator <(const Point& b) const { return x == b.x ? y < b.y : x < b.x; }
    void operator +=(const Point& b) { x += b.x; y += b.y; }
    void operator -=(const Point& b) { x -= b.x; y -= b.y; }
    void operator *=(const int k) { x *= k; y *= k; }

    ll cross(const Point& b, const Point& c) const {
        return (b - *this) * (c - *this);
    }
};

int N, S;
Point P[maxN];
vector hull;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        P[i].read();
    }
    sort(P, P+N);

    for(int t = 0; t < 2; t++){
        for(int i = 0; i < N; i++){
            while((int) hull.size()-S >= 2){
                Point P1 = hull[hull.size()-2];
                Point P2 = hull[hull.size()-1];
                if(P1.cross(P2, P[i]) <= 0) break;
                hull.pop_back();
            }
            hull.push_back(P[i]);
        }
        hull.pop_back();
        S = hull.size();
        reverse(P, P+N);
    }

    printf("%d\n", S);
    for(Point h : hull)

```

```
    printf("%d %d\n", h.x, h.y);  
}
```

Maximum Manhattan Distances

```
#include  
  
using namespace std;  
typedef long long ll;  
const ll INF = 0x3f3f3f3f3f3f3f3f;  
  
int N;  
ll l1, l2, l3, l4;  
  
int main(){  
    scanf("%d", &N);  
  
    l1 = l4 = -INF;  
    l2 = l3 = INF;  
  
    for(int i = 0; i < N; i++){  
        ll x, y;  
        scanf("%lld %lld", &x, &y);  
  
        l1 = max(l1, x+y);  
        l2 = min(l2, x+y);  
        l3 = min(l3, x-y);  
        l4 = max(l4, x-y);  
  
        printf("%lld\n", max(l1-l2, l4-l3));  
    }  
}
```

All Manhattan Distances

```
#include  
  
using namespace std;  
const int maxN = 2e5+1;  
  
int N, x[maxN], y[maxN];  
__int128 xcur, ycur, ans;  
  
ostream& operator<<(ostream& o, const __int128& x) {  
    if(x == numeric_limits<__int128>::min())  
        return o << "-170141183460469231731687303715884105728";  
    else if(x < 0) return o << "-" << -x;  
    else if(x < 10) return o << (char)(x + '0');  
    else return o << x / 10 << (char)(x % 10 + '0');  
}  
  
int main(){
```

```

scanf("%d", &N);
for(int i = 0; i < N; i++){
    scanf("%d %d", &x[i], &y[i]);

    sort(x, x+N);
    sort(y, y+N);

    xcur = ycur = ans = 0;
    for(int i = 1; i < N; i++){
        xcur += (__int128) (x[i] - x[i-1]) * i;
        ycur += (__int128) (y[i] - y[i-1]) * i;
        ans += xcur + ycur;
    }
    cout << ans << endl;
}

```

Intersection Points

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+1;
const int maxX = 1e6+5;
const int SIZE = 2*maxX;

struct Operation {
    int type, y, a, b;
};

int N, ds[SIZE];
vector ops;

void update(int idx, int val){
    for(int i = idx; i < SIZE; i += -i&i)
        ds[i] += val;
}

int query(int idx){
    int sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += ds[i];
    return sum;
}

int main(){
    scanf("%d", &N);
    for(int i = 0, a, b, c, d; i < N; i++){
        scanf("%d %d %d %d", &a, &b, &c, &d);
        if(a == c){
            // Vertical
            ops.push_back({2, b, a+maxX, -1});
            ops.push_back({3, d, a+maxX, -1});
        } else {
            // Horizontal
            ops.push_back({1, b, a+maxX, c+maxX});
        }
    }
    sort(ops.begin(), ops.end(), [](Operation A, Operation B){
        return A.y < B.y;
    });
}

```

```

ll ans = 0;
for(Operation O : ops){
    if(O.type == 1)        ans += query(O.b) - query(O.a-1);
    else if(O.type == 2)   update(O.a, 1);
    else if(O.type == 3)   update(O.a, -1);
}
printf("%lld\n", ans);
}

```

Line Segments Trace I

```

#include

using namespace std;
typedef long long ll;
const ll INF = 0x3f3f3f3f3f3f3f3f;

struct Line {
    ll a, b;

    Line() : a(0), b(-INF) {}
    Line(ll a, ll b) : a(a), b(b) {}

    ll get(ll x){
        return a * x + b;
    }
};

struct Node {
    Line line = Line();
    Node* lc = nullptr;
    Node* rc = nullptr;
};

struct LiChaoTree {
    Node* root;
    ll size;
    LiChaoTree(ll size) : root(nullptr), size(size) {}

private:
    void contained_insert(Node* &node, ll tl, ll tr, Line line){
        if(node == nullptr) node = new Node();
        if(node->line.get(tl) < line.get(tl)) swap(node->line, line);
        if(node->line.get(tr) >= line.get(tr)) return;
        if(tl == tr) return;

        ll tm = (tl + tr) / 2;
        if(node->line.get(tm) > line.get(tm)) {
            contained_insert(node->rc, tm+1, tr, line);
        } else {
            swap(node->line, line);
            contained_insert(node->lc, tl, tm, line);
        }
    }

    void insert(Node* &node, ll tl, ll tr, ll l, ll r, Line line){
        if(tr < l || r < tl || tl > tr || l > r) return;
        if(node == nullptr) node = new Node();
        if(l <= tl && tr <= r) return contained_insert(node, tl, tr, line);
    }
};

```

```

    ll tm = (tl + tr) / 2;
    insert(node->lc, tl, tm, l, r, line);
    insert(node->rc, tm+1, tr, l, r, line);
}

ll query(Node* &node, ll tl, ll tr, ll x) {
    if(node == nullptr) return -INF;
    if(tl == tr) return node->line.get(x);

    ll res = node->line.get(x);
    ll tm = (tl + tr) / 2;
    if(x <= tm) res = max(res, query(node->lc, tl, tm, x));
    else res = max(res, query(node->rc, tm+1, tr, x));

    return res;
}

public:
    void insert(ll l, ll r, Line line) {
        insert(root, 0, size-1, l, r, line);
    }

    ll query(ll x) {
        return query(root, 0, size-1, x);
    }
};

int main(){
    int N, M;
    scanf("%d %d", &N, &M);
    LiChaoTree tree(M+1);

    for(int i = 0, y1, y2; i < N; i++){
        scanf("%d %d", &y1, &y2);

        ll m = (y2 - y1) / M;
        tree.insert(0, M, Line(m, y1));
    }

    for(int x = 0; x <= M; x++){
        ll res = tree.query(x);
        printf("%lld%c", (res == -INF ? -1 : res), (" \n")[x==M]);
    }
}

```

Line Segments Trace II

```

#include

using namespace std;
typedef long long ll;
const ll INF = 0x3f3f3f3f3f3f3f3f;

struct Line {
    ll a, b;

    Line() : a(0), b(-INF) {}
    Line(ll a, ll b) : a(a), b(b) {}

    ll get(ll x){

```

```

        return a * x + b;
    }
};

struct Node {
    Line line = Line();
    Node* lc = nullptr;
    Node* rc = nullptr;
};

struct LiChaoTree {
    Node* root;
    ll size;
    LiChaoTree(ll size) : root(nullptr), size(size) {}

private:
    void contained_insert(Node* &node, ll tl, ll tr, Line line){
        if(node == nullptr) node = new Node();
        if(node->line.get(tl) < line.get(tl)) swap(node->line, line);
        if(node->line.get(tr) >= line.get(tr)) return;
        if(tl == tr) return;

        ll tm = (tl + tr) / 2;
        if(node->line.get(tm) > line.get(tm)) {
            contained_insert(node->rc, tm+1, tr, line);
        } else {
            swap(node->line, line);
            contained_insert(node->lc, tl, tm, line);
        }
    }

    void insert(Node* &node, ll tl, ll tr, ll l, ll r, Line line){
        if(tr < l || r < tl || tl > tr || l > r) return;
        if(node == nullptr) node = new Node();
        if(l <= tl && tr <= r) return contained_insert(node, tl, tr, line);

        ll tm = (tl + tr) / 2;
        insert(node->lc, tl, tm, l, r, line);
        insert(node->rc, tm+1, tr, l, r, line);
    }

    ll query(Node* &node, ll tl, ll tr, ll x) {
        if(node == nullptr) return -INF;
        if(tl == tr) return node->line.get(x);

        ll res = node->line.get(x);
        ll tm = (tl + tr) / 2;
        if(x <= tm) res = max(res, query(node->lc, tl, tm, x));
        else res = max(res, query(node->rc, tm+1, tr, x));

        return res;
    }

public:
    void insert(ll l, ll r, Line line) {
        insert(root, 0, size-1, l, r, line);
    }

    ll query(ll x) {
        return query(root, 0, size-1, x);
    }
};

int main(){
    int N, M;
    scanf("%d %d", &N, &M);
    LiChaoTree tree(M+1);

    for(int i = 0, x1, y1, x2, y2; i < N; i++){

```

```

scanf("%d %d %d %d", &x1, &y1, &x2, &y2);

ll m = (y2 - y1) / (x2 - x1);
ll b = y1 - m * x1;
tree.insert(x1, x2, Line(m, b));
}

for(int x = 0; x <= M; x++){
    ll res = tree.query(x);
    printf("%lld%c", (res == -INF ? -1 : res), (" \n")[x==M]);
}
}

```

Lines and Queries I

```

#include

using namespace std;
typedef long long ll;
const ll INF = 0x3f3f3f3f3f3f3f;
const ll maxX = 1e5+5;

struct Line {
    ll a, b;

    Line() : a(0), b(-INF) {}
    Line(ll a, ll b) : a(a), b(b) {}

    ll get(ll x){
        return a * x + b;
    }
};

struct Node {
    Line line = Line();
    Node* lc = nullptr;
    Node* rc = nullptr;
};

struct LiChaoTree {
    Node* root;
    ll size;
    LiChaoTree(ll size) : root(nullptr), size(size) {}

private:
    void contained_insert(Node* &node, ll tl, ll tr, Line line){
        if(node == nullptr) node = new Node();
        if(node->line.get(tl) < line.get(tl)) swap(node->line, line);
        if(node->line.get(tr) >= line.get(tr)) return;
        if(tl == tr) return;

        ll tm = (tl + tr) / 2;
        if(node->line.get(tm) > line.get(tm)) {
            contained_insert(node->rc, tm+1, tr, line);
        } else {
            swap(node->line, line);
            contained_insert(node->lc, tl, tm, line);
        }
    }

    void insert(Node* &node, ll tl, ll tr, ll l, ll r, Line line){

```

```

    if(tr < l || r < t1 || t1 > tr || l > r) return;
    if(node == nullptr) node = new Node();
    if(l <= t1 && tr <= r) return contained_insert(node, t1, tr, line);

    ll tm = (t1 + tr) / 2;
    insert(node->lc, t1, tm, l, r, line);
    insert(node->rc, tm+1, tr, l, r, line);
}

ll query(Node* &node, ll t1, ll tr, ll x) {
    if(node == nullptr) return -INF;
    if(t1 == tr) return node->line.get(x);

    ll res = node->line.get(x);
    ll tm = (t1 + tr) / 2;
    if(x <= tm) res = max(res, query(node->lc, t1, tm, x));
    else res = max(res, query(node->rc, tm+1, tr, x));

    return res;
}

public:
    void insert(ll l, ll r, Line line) {
        insert(root, 0, size-1, l, r, line);
    }

    ll query(ll x) {
        return query(root, 0, size-1, x);
    }
};

int main(){
    LiChaoTree tree(maxX+1);

    int T;
    scanf("%d", &T);
    for(int t = 0, type, a, b, x; t < T; t++){
        scanf("%d", &type);
        if(type == 1){
            scanf("%d %d", &a, &b);
            tree.insert(0, maxX, Line(a, b));
        } else if(type == 2){
            scanf("%d", &x);
            ll res = tree.query(x);
            if(res == -INF) printf("NO\n");
            else printf("%lld\n", res);
        }
    }
}

```

Lines and Queries II

```

#include

using namespace std;
typedef long long ll;
const ll INF = 0x3f3f3f3f3f3f3f3f;
const ll maxX = 1e5+5;

struct Line {

```

```

    ll a, b;

    Line() : a(0), b(-INF) {}
    Line(ll a, ll b) : a(a), b(b) {}

    ll get(ll x){
        return a * x + b;
    }
};

struct Node {
    Line line = Line();
    Node* lc = nullptr;
    Node* rc = nullptr;
};

struct LiChaoTree {
    Node* root;
    ll size;
    LiChaoTree(ll size) : root(nullptr), size(size) {}

private:
    void contained_insert(Node* &node, ll tl, ll tr, Line line){
        if(node == nullptr) node = new Node();
        if(node->line.get(tl) < line.get(tl)) swap(node->line, line);
        if(node->line.get(tr) >= line.get(tr)) return;
        if(tl == tr) return;

        ll tm = (tl + tr) / 2;
        if(node->line.get(tm) > line.get(tm)) {
            contained_insert(node->rc, tm+1, tr, line);
        } else {
            swap(node->line, line);
            contained_insert(node->lc, tl, tm, line);
        }
    }

    void insert(Node* &node, ll tl, ll tr, ll l, ll r, Line line){
        if(tr < l || r < tl || tl > tr || l > r) return;
        if(node == nullptr) node = new Node();
        if(l <= tl && tr <= r) return contained_insert(node, tl, tr, line);

        ll tm = (tl + tr) / 2;
        insert(node->lc, tl, tm, l, r, line);
        insert(node->rc, tm+1, tr, l, r, line);
    }

    ll query(Node* &node, ll tl, ll tr, ll x) {
        if(node == nullptr) return -INF;
        if(tl == tr) return node->line.get(x);

        ll res = node->line.get(x);
        ll tm = (tl + tr) / 2;
        if(x <= tm) res = max(res, query(node->lc, tl, tm, x));
        else res = max(res, query(node->rc, tm+1, tr, x));

        return res;
    }
};

public:
    void insert(ll l, ll r, Line line) {
        insert(root, 0, size-1, l, r, line);
    }

    ll query(ll x) {
        return query(root, 0, size-1, x);
    }
};

```

```

int main(){
    LiChaoTree tree(maxX);

    int T;
    scanf("%d", &T);
    for(int t = 0, type, a, b, l, r, x; t < T; t++){
        scanf("%d", &type);
        if(type == 1){
            scanf("%d %d %d %d", &a, &b, &l, &r);
            tree.insert(l, r, Line(a, b));
        } else if(type == 2){
            scanf("%d", &x);
            ll res = tree.query(x);
            if(res == -INF) printf("NO\n");
            else printf("%lld\n", res);
        }
    }
}

```

Area of Rectangles

```

#include

using namespace std;
typedef long long ll;
typedef array<int, 2> Operation;
const int maxN = 1e5;
const int SZ = 9e6;

int N, lo[SZ], hi[SZ];
ll area, delta[SZ], score[SZ];
Operation op[2*maxN];

int len(int i){
    return hi[i]-lo[i]+1;
}

void pull(int i){
    if(lo[i] == hi[i]) score[i] = (delta[i] > 0 ? 1 : 0);
    else score[i] = (delta[i] > 0 ? len(i) : score[2*i] + score[2*i+1]);
}

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r) return;
    int m = l+(r-l)/2;
    build(2*i, l, m);
    build(2*i+1, m+1, r);
}

void increment(int i, int l, int r, ll val){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        delta[i] += val;
        pull(i);
        return;
    }
    increment(2*i, l, r, val);
    increment(2*i+1, l, r, val);
    pull(i);
}

```

```
ll query(){
    return score[1];
}

int main(){
    scanf("%d", &N);
    for(int i = 0, a, b, c, d; i < N; i++){
        scanf("%d %d %d %d", &a, &b, &c, &d);
        op[2*i] = {1, b, a+1, c};
        op[2*i+1] = {-1, d, a+1, c};
    }
    sort(op, op+2*N, [](Operation A, Operation B){
        return (A[1] == B[1] ? A[0] < B[0] : A[1] < B[1]);
    });

    build(1, -1e6-5, 1e6+5);
    int lst = -1e6;
    for(int i = 0; i < 2*N; i++){
        int t = op[i][0], y = op[i][1], x1 = op[i][2], x2 = op[i][3];
        area += (y-lst) * query();
        increment(1, x1, x2, t);
        lst = y;
    }

    printf("%lld\n", area);
}
```

Advanced

Meet in the Middle

```
#include

using namespace std;
typedef long long ll;
const int maxN = 40;

int N, t[maxN];
ll x, sum, cnt;
unordered_map freq;

int main(){
    scanf("%d %lld", &N, &x);
    for(int i = 0; i < N; i++)
        scanf("%d", &t[i]);
    sort(t, t+N);

    if(N == 1){
        printf("%d\n", x == t[0]);
        return 0;
    }

    freq.reserve(1<<(N/2-1));
    for(int i = 0; i < (1<<(N/2-1)); i++){
        sum = 0;
        for(int j = 0; j < N/2-1; j++)
            if(i&(1<
```

Hamming Distance

```
#include

using namespace std;
const int maxN = 2e4;

int N, ans, b[maxN];

int scanBinary(){
    char c;
    int res = 0;
    while((c = getchar()) != '\n'){
        res <= 1;
        res += (c-'0')&1;
    }
    return res;
}

int main(){
    scanf("%d %d ", &N, &ans);
    for(int i = 0; i < N; i++)
        b[i] = scanBinary();
```

```

for(int i = 0; i < N; i++)
    for(int j = i+1; j < N; j++)
        ans = min(ans, __builtin_popcount(b[i]^b[j]));

printf("%d\n", ans);
}

```

Corner Subgrid Count

```

#include
#pragma GCC target("popcnt")

using namespace std;
typedef long long ll;
const int maxN = 3005;

int N;
ll ans;
bitset B[maxN];

int f(int X){
    return X*(X-1);
}

int main(){
    cin.sync_with_stdio(0);
    cin.tie(0);

    cin >> N;
    for(int i = 0; i < N; i++)
        cin >> B[i];

    for(int i = 0; i < N; i++)
        for(int j = i+1; j < N; j++)
            ans += f((B[i]&B[j]).count());

    cout << (ans>>1);
}

```

Reachable Nodes

```

#include

using namespace std;
const int maxN = 5e4+1;

int N, M, a, b, in[maxN];
bitset ans[maxN];
vector G[maxN];
queue Q;

```

```

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[b].push_back(a);
        in[a]++;
    }

    for(int i = 1; i <= N; i++){
        if(in[i] == 0){
            ans[i].set(i);
            Q.push(i);
        }
    }

    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        for(int v : G[u]){
            ans[v] |= ans[u];
            in[v]--;
            if(in[v] == 0){
                ans[v].set(v);
                Q.push(v);
            }
        }
    }

    for(int i = 1; i <= N; i++)
        printf("%d%c", (int) ans[i].count(), (" \n")[i==N]);
}

```

Reachability Queries

```

#include

using namespace std;
const int maxN = 5e4+5;

bool vis[maxN];
int N, M, Q, ds[maxN];
vector order, component;
vector G[maxN], H[maxN], SCC[maxN];
bitset reachable[maxN];

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]){
        ds[u] += ds[v];
        ds[v] = u;
    } else {
        ds[v] += ds[u];
        ds[u] = v;
    }
    return true;
}

```

```

}

void dfs1(int u){
    vis[u] = true;
    for(int v : G[u])
        if(!vis[v])
            dfs1(v);
    order.push_back(u);
}

void dfs2(int u){
    vis[u] = true;
    component.push_back(u);
    for(int v : H[u])
        if(!vis[v])
            dfs2(v);
}

void dfs3(int u){
    vis[u] = true;
    for(int v : SCC[u]){
        if(!vis[v])
            dfs3(v);
        reachable[u] |= reachable[v];
    }
}

int main(){
    scanf("%d %d %d", &N, &M, &Q);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        H[b].push_back(a);
    }

    fill(vis+1, vis+N+1, false);
    for(int i = 1; i <= N; i++)
        if(!vis[i])
            dfs1(i);

    fill(ds+1, ds+N+1, -1);
    fill(vis+1, vis+N+1, false);
    for(int i = 1; i <= N; i++){
        int u = order[N-i];
        if(!vis[u]){
            dfs2(u);
            for(int j = 0; j+1 < (int) component.size(); j++)
                merge(component[j], component[j+1]);
            component.clear();
        }
    }

    for(int i = 1; i <= N; i++){
        int rep = find(i);
        reachable[rep].set(i);
        for(int v : G[i])
            SCC[rep].push_back(find(v));
    }

    fill(vis+1, vis+N+1, false);
    for(int i = 1; i <= N; i++)
        if(!vis[i])
            dfs3(i);

    for(int i = 0, a, b; i < Q; i++){
        scanf("%d %d", &a, &b);
        a = find(a); b = find(b);
        printf(reachable[a][b] ? "YES\n" : "NO\n");
    }
}

```

```

    }
}

```

Cut and Paste

```

#include

using namespace std;
const int maxN = 2e5+5;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution dist(1, (int) 2e9);

struct Node {
    char ch;
    int prior, sz;
    Node *l, *r;

    Node(){}
    Node(char c, int p){
        prior = p;
        ch = c;
        sz = 1;
        l = r = nullptr;
    }
};

int N, M, a, b;
char S[maxN];
Node *root;

int sz(Node *t){ return t ? t->sz : 0; }

void pull(Node *t){
    if(!t) return;
    t->sz = sz(t->l) + sz(t->r) + 1;
}

void push(Node *t){
    if(!t) return;
}

Node* merge(Node *x, Node *y){
    if(!x || !y) return x ? x : y;

    push(x); push(y);
    if(x->prior < y->prior){
        x->r = merge(x->r, y);
        pull(x);
        return x;
    } else {
        y->l = merge(x, y->l);
        pull(y);
        return y;
    }
}

pair split(Node *x, int k){
    if(!x) return {nullptr, nullptr};

    pair y = {nullptr, nullptr};

```

```

    push(x);
    if(k <= sz(x->l)){
        y = split(x->l, k);
        x->l = y.second;
        pull(x);
        y.second = x;
    } else {
        y = split(x->r, k-sz(x->l)-1);
        x->r = y.first;
        pull(x);
        y.first = x;
    }
    return y;
}

void heapify(Node *t){
    if(!t) return;

    Node *mx = t;
    if(t->l && t->l->prior > mx->prior) mx = t->l;
    if(t->r && t->r->prior > mx->prior) mx = t->r;
    if(mx != t){
        swap(t->prior, mx->prior);
        heapify(mx);
    }
}

Node* build(int x, int k){
    if(k == 0) return nullptr;

    int mid = k/2;
    Node *t = new Node(S[x+mid], dist(rng));
    t->l = build(x, mid);
    t->r = build(x+mid+1, k-mid-1);
    heapify(t);
    pull(t);
    return t;
}

void cut(int x, int k){
    pair y, z;
    y = split(root, x-1);
    z = split(y.second, k);
    y.second = merge(z.second, z.first);
    root = merge(y.first, y.second);
}

void print(Node *t){
    if(!t) return;
    push(t);
    print(t->l);
    printf("%c", t->ch);
    print(t->r);
}

int main(){
    scanf("%d %d %s", &N, &M, S);
    root = build(0, N);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        int len = (b-a+1);
        cut(a, len);
    }
    print(root);
}

```

Substring Reversals

```
#include

using namespace std;
const int maxN = 2e5+5;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution dist(1, (int) 2e9);

struct Node {
    bool rev;
    char ch;
    int prior, sz;
    Node *l, *r;

    Node(){}
    Node(char c, int p){
        prior = p;
        ch = c;
        sz = 1;
        l = r = nullptr;
    }
};

int N, M, a, b;
char S[maxN];
Node *root;

int sz(Node *t){ return t ? t->sz : 0; }

void flip(Node *t){
    if(!t) return;
    t->rev ^= true;
}

void pull(Node *t){
    if(!t) return;
    t->sz = sz(t->l) + sz(t->r) + 1;
}

void push(Node *t){
    if(!t) return;
    if(t->rev){
        swap(t->l, t->r);
        flip(t->l); flip(t->r);
        t->rev = false;
    }
}

Node* merge(Node *x, Node *y){
    if(!x || !y) return x ? x : y;

    push(x); push(y);
    if(x->prior < y->prior){
        x->r = merge(x->r, y);
        pull(x);
        return x;
    } else {
        y->l = merge(x, y->l);
        pull(y);
        return y;
    }
}

pair split(Node *x, int k){
    if(!x) return {nullptr, nullptr};
```

```

    pair y = {nullptr, nullptr};
    push(x);
    if(k <= sz(x->l)){
        y = split(x->l, k);
        x->l = y.second;
        pull(x);
        y.second = x;
    } else {
        y = split(x->r, k-sz(x->l)-1);
        x->r = y.first;
        pull(x);
        y.first = x;
    }
    return y;
}

void heapify(Node *t){
    if(!t) return;

    Node *mx = t;
    if(t->l && t->l->prior > mx->prior) mx = t->l;
    if(t->r && t->r->prior > mx->prior) mx = t->r;
    if(mx != t){
        swap(t->prior, mx->prior);
        heapify(mx);
    }
}

Node* build(int x, int k){
    if(k == 0) return nullptr;

    int mid = k/2;
    Node *t = new Node(S[x+mid], dist(rng));
    t->l = build(x, mid);
    t->r = build(x+mid+1, k-mid-1);
    heapify(t);
    pull(t);
    return t;
}

void reverse(int x, int k){
    pair y, z;
    y = split(root, x-1);
    z = split(y.second, k);
    flip(z.first);
    y.second = merge(z.first, z.second);
    root = merge(y.first, y.second);
}

void print(Node *t){
    if(!t) return;
    push(t);
    print(t->l);
    printf("%c", t->ch);
    print(t->r);
}

int main(){
    scanf("%d %d %s", &N, &M, S);
    root = build(0, N);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        int len = (b-a+1);
        reverse(a, len);
    }
    print(root);
}

```

Reversals and Sums

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution dist(1, (int) 2e9);

struct Node {
    bool rev;
    int prior, sz;
    ll val, sum;
    Node *l, *r;

    Node(){}
    Node(ll v, int p){
        prior = p;
        sum = val = v;
        sz = 1;
        l = r = nullptr;
    }
};

int N, M, type, a, b;
ll val[maxN];
Node *root;

int sz(Node *t){ return t ? t->sz : 0; }
ll sum(Node *t){ return t ? t->sum : 0; }

void flip(Node *t){
    if(!t) return;
    t->rev ^= true;
}

void pull(Node *t){
    if(!t) return;
    t->sz = sz(t->l) + sz(t->r) + 1;
    t->sum = sum(t->l) + sum(t->r) + t->val;
}

void push(Node *t){
    if(!t) return;
    if(t->rev){
        swap(t->l, t->r);
        flip(t->l); flip(t->r);
        t->rev = false;
    }
}

Node* merge(Node *x, Node *y){
    if(!x || !y) return x ? x : y;

    push(x); push(y);
    if(x->prior < y->prior){
        x->r = merge(x->r, y);
        pull(x);
        return x;
    } else {
        y->l = merge(x, y->l);
        pull(y);
    }
}
```

```

        return y;
    }
}

pair split(Node *x, int k){
    if(!x) return {nullptr, nullptr};

    pair y = {nullptr, nullptr};
    push(x);
    if(k <= sz(x->l)){
        y = split(x->l, k);
        x->l = y.second;
        pull(x);
        y.second = x;
    } else {
        y = split(x->r, k-sz(x->l)-1);
        x->r = y.first;
        pull(x);
        y.first = x;
    }
    return y;
}

void heapify(Node *t){
    if(!t) return;

    Node *mx = t;
    if(t->l && t->l->prior > mx->prior) mx = t->l;
    if(t->r && t->r->prior > mx->prior) mx = t->r;
    if(mx != t){
        swap(t->prior, mx->prior);
        heapify(mx);
    }
}

Node* build(int x, int k){
    if(k == 0) return nullptr;

    int mid = k/2;
    Node *t = new Node(val[x+mid], dist(rng));
    t->l = build(x, mid);
    t->r = build(x+mid+1, k-mid-1);
    heapify(t);
    pull(t);
    return t;
}

void reverse(int x, int k){
    pair y, z;
    y = split(root, x-1);
    z = split(y.second, k);
    flip(z.first);
    y.second = merge(z.first, z.second);
    root = merge(y.first, y.second);
}

ll getSum(int x, int k){
    pair y, z;
    y = split(root, x-1);
    z = split(y.second, k);
    ll ans = sum(z.first);
    y.second = merge(z.first, z.second);
    root = merge(y.first, y.second);
    return ans;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++)
        scanf("%lld", &val[i]);
}

```

```

root = build(0, N);

for(int i = 0; i < M; i++){
    scanf("%d %d %d", &type, &a, &b);
    int len = (b-a+1);
    if(type == 1)
        reverse(a, len);
    else if(type == 2)
        printf("%lld\n", getSum(a, len));
}
}

```

Necessary Roads

```

#include

using namespace std;
typedef pair pii;
const int maxN = 1e5+1;

int N, M, timer, tin[maxN], low[maxN];
bool vis[maxN];
vector G[maxN];
vector bridges;

void dfs(int u = 1, int p = 0){
    vis[u] = true;
    tin[u] = low[u] = ++timer;
    for(int v : G[u]){
        if(v != p){
            if(vis[v]) low[u] = min(low[u], tin[v]);
            else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if(low[v] > tin[u])
                    bridges.push_back({u, v});
            }
        }
    }
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs();

    printf("%d\n", (int) bridges.size());
    for(pii P : bridges)
        printf("%d %d\n", P.first, P.second);
}

```

Necessary Cities

```
#include

using namespace std;
typedef pair pii;
const int maxN = 1e5+1;

bool vis[maxN], is_artic[maxN];
int N, M, timer, tin[maxN], low[maxN];
vector ans, G[maxN];

void dfs(int u = 1, int p = -1){
    vis[u] = true;
    tin[u] = low[u] = ++timer;

    int children = 0;
    for(int v : G[u]){
        if(v != p){
            if(vis[v]) low[u] = min(low[u], tin[v]);
            else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if(low[v] >= tin[u] && p != -1){
                    if(!is_artic[u]){
                        ans.push_back(u);
                        is_artic[u] = true;
                    }
                }
                children++;
            }
        }
    }

    if(p == -1 && children > 1){
        if(!is_artic[u]){
            ans.push_back(u);
            is_artic[u] = true;
        }
    }
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs();

    int K = (int) ans.size();
    printf("%d\n", K);
    for(int i = 0; i < K; i++)
        printf("%d%c", ans[i], (" \n")[i==K-1]);
}
```

Eulerian Subgraphs

```

#include

using namespace std;
const int maxN = 1e5+1;
const int MOD = 1e9+7;

int N, M, ans, ds[maxN];

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d", &N, &M);
    fill(ds+1, ds+N+1, -1);
    ans = 1;
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        if(!merge(a, b))
            ans = (2 * ans) % MOD;
    }
    printf("%d\n", ans);
}

```

Monster Game I

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;
const int SIZE = 4e6;
const ll INF = 0x3f3f3f3f3f3f3f3f;

struct Line {
    ll m, b;
    ll operator()(const ll x) const {
        return m * x + b;
    }
} seg[SIZE];

int N, lo[SIZE], hi[SIZE];
ll X, s[maxN], f[maxN];

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    seg[i] = {0, INF};
    if(l == r) return;
    int m = (l+r)/2;

```

```

    build(2*i, l, m);
    build(2*i+1, m+1, r);
}

void insert(int i, Line L){
    int l = lo[i], r = hi[i];
    if(l == r){
        if(L(l) < seg[i](l))
            seg[i] = L;
        return;
    }

    int m = (l+r)/2;
    if(seg[i].m < L.m) swap(seg[i], L);
    if(seg[i](m) > L(m)){
        swap(seg[i], L);
        insert(2*i, L);
    } else insert(2*i+1, L);
}

ll query(int i, ll x){
    int l = lo[i], r = hi[i];
    if(l == r) return seg[i](x);

    int m = (l+r)/2;
    if(x < m) return min(seg[i](x), query(2*i, x));
    else return min(seg[i](x), query(2*i+1, x));
}

int main(){
    scanf("%d %lld", &N, &X);
    for(int i = 0; i < N; i++) scanf("%lld", &s[i]);
    for(int i = 0; i < N; i++) scanf("%lld", &f[i]);

    build(1, 1, 1e6);
    insert(1, {X, 0});
    for(int i = 0; i < N-1; i++){
        ll best = query(1, s[i]);
        insert(1, {f[i], best});
    }
    printf("%lld\n", query(1, s[N-1]));
}

```

Monster Game II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;
const int SIZE = 4e6;
const ll INF = 0x3f3f3f3f3f3f3f3f;

struct Line {
    ll m, b;
    ll operator()(const ll x) const {
        return m * x + b;
    }
} seg[SIZE];

int N, lo[SIZE], hi[SIZE];

```

```

ll X, s[maxN], f[maxN];

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    seg[i] = {0, INF};
    if(l == r) return;
    int m = (l+r)/2;
    build(2*i, l, m);
    build(2*i+1, m+1, r);
}

void insert(int i, Line L){
    int l = lo[i], r = hi[i];
    if(l == r){
        if(L(l) < seg[i](l))
            seg[i] = L;
        return;
    }

    int m = (l+r)/2;
    if(seg[i].m < L.m) swap(seg[i], L);
    if(seg[i](m) > L(m)){
        swap(seg[i], L);
        insert(2*i, L);
    } else insert(2*i+1, L);
}

ll query(int i, ll x){
    int l = lo[i], r = hi[i];
    if(l == r) return seg[i](x);

    int m = (l+r)/2;
    if(x < m) return min(seg[i](x), query(2*i, x));
    else return min(seg[i](x), query(2*i+1, x));
}

int main(){
    scanf("%d %lld", &N, &X);
    for(int i = 0; i < N; i++) scanf("%lld", &s[i]);
    for(int i = 0; i < N; i++) scanf("%lld", &f[i]);

    build(1, 1, 1e6);
    insert(1, {X, 0});
    for(int i = 0; i < N-1; i++){
        ll best = query(1, s[i]);
        insert(1, {f[i], best});
    }
    printf("%lld\n", query(1, s[N-1]));
}

```

Subarray Squares

```

#include

using namespace std;
typedef long long ll;
typedef pair pll;
const int maxN = 3005;

struct Line {
    ll m, b, c;

```

```

    ll operator()(ll x){
        return m * x + b;
    }
};

struct CHT {
    Line dq[2*maxN];
    int fptr, bptr;

    void clear(){
        dq[0] = {0, 0, 0};
        fptr = 0; bptr = 1;
    }

    bool pop_back(Line& L, Line& L1, Line& L2){
        ll v1 = (L.b - L2.b) * (L2.m - L1.m);
        ll v2 = (L2.m - L.m) * (L1.b - L2.b);
        return (v1 == v2 ? L.c > L1.c : v1 < v2);
    }

    bool pop_front(Line& L1, Line& L2, ll x){
        ll v1 = L1(x);
        ll v2 = L2(x);
        return (v1 == v2 ? L1.c < L2.c : v1 > v2);
    }

    void insert(Line L){
        while(bptr-fptr >= 2 && pop_back(L, dq[bptr-1], dq[bptr-2]))    bptr--;
        dq[bptr++] = L;
    }

    pll query(ll x){
        while(bptr-fptr >= 2 && pop_front(dq[fptr], dq[fptr+1], x))    fptr++;
        return {dq[fptr](x), dq[fptr].c};
    }
};

CHT cht;
int N, K, cnt[maxN];
ll pre[maxN], dp[maxN];

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &pre[i]);
        pre[i] += pre[i-1];
        dp[i] = pre[i]*pre[i];
    }

    for(int k = 1; k <= K-1; k++){
        cht.clear();
        for(int i = 1; i <= k; i++)
            cht.insert({-2*pre[i], dp[i]+pre[i]*pre[i], cnt[i]});
        for(int i = k+1; i <= N; i++){
            pll P = cht.query(pre[i]);
            cht.insert({-2*pre[i], dp[i]+pre[i]*pre[i], cnt[i]});
            dp[i] = pre[i]*pre[i] + P.first;
            cnt[i] = P.second + 1;
        }
    }

    printf("%lld\n", dp[N]);
}

```

Houses and Schools

```
#include

using namespace std;
typedef long long ll;
const int maxN = 3001;
const ll INF = 0x3f3f3f3f3f3f3f3f;

int N, K;
ll best, p[3][maxN], dp[maxN][maxN];

ll travel(int d, int a, int b){
    return (p[d][b] - p[d][a-1]) - (p[0][b] - p[0][a-1]) * (d == 1 ? a : N-b+1);
}

ll C(int a, int b){
    int m = (a+b)/2;
    return travel(1, a, m) + travel(2, m+1, b);
}

void solve(int k, int a = 1, int b = N, int optl = 1, int optr = N){
    if(a > b) return;
    int m = (a+b)/2;
    int opt = -1;
    dp[k][m] = INF;
    for(int i = optl; i <= m; i++){
        if(dp[k-1][i] + C(i, m) < dp[k][m]){
            dp[k][m] = dp[k-1][i] + C(i, m);
            opt = i;
        }
    }
    solve(k, a, m-1, optl, opt);
    solve(k, m+1, b, opt, optr);
}

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 1; i <= N; i++){
        ll x;
        scanf("%lld", &x);
        p[0][i] = p[0][i-1] + x;
        p[1][i] = p[1][i-1] + i * x;
        p[2][i] = p[2][i-1] + (N-i+1) * x;
    }

    for(int i = 1; i <= N; i++)
        dp[1][i] = travel(2, 1, i);
    for(int k = 2; k <= K; k++)
        solve(k);

    best = INF;
    for(int i = 1; i <= N; i++)
        best = min(best, dp[K][i] + travel(1, i, N));
    printf("%lld\n", best);
}
```

Knuth Division

```

#include

using namespace std;
typedef long long ll;
const ll INF = 0x3f3f3f3f3f3f3f;
const int maxN = 5005;

int N, split[maxN][maxN];
ll pre[maxN], dp[maxN][maxN];

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &pre[i]);
        pre[i] += pre[i-1];
    }

    for(int len = 0; len <= N; len++){
        for(int l = 0; l+len <= N; l++){
            int r = l+len;
            if(len < 2){
                dp[l][r] = 0;
                split[l][r] = l;
                continue;
            }

            dp[l][r] = INF;
            for(int m = split[l][r-1]; m <= split[l+1][r]; m++){
                ll possible = dp[l][m] + dp[m][r] + pre[r] - pre[l];
                if(possible < dp[l][r]){
                    dp[l][r] = possible;
                    split[l][r] = m;
                }
            }
        }
    }

    printf("%lld\n", dp[0][N]);
}

```

Apples and Bananas

```

#include

using namespace std;
typedef double ld;
typedef complex cd;
const int SIZE = 1<<19;
const ld PI = acos(-1);

int N, M, K;
vector A(SIZE), B(SIZE);

void fft(vector &a, bool inv){
    int N = (int) a.size();

    for(int i = 1, j = 0; i < N; i++){
        int bit = N>>1;
        for(; j&bit; bit >>= 1)
            j ^= bit;
    }
}

```

```

        j ^= bit;

        if(i < j)
            swap(a[i], a[j]);
    }

    for(int len = 2; len <= N; len <= 1){
        ld theta = 2*PI / len * (inv ? -1 : 1);
        cd wlen(cos(theta), sin(theta));
        for(int i = 0; i < N; i += len){
            cd w(1);
            for(int j = 0; j < len / 2; j++){
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if(inv)
        for(cd &z : a)
            z /= N;
}

int main(){
    scanf("%d %d %d", &K, &N, &M);
    for(int i = 0, x; i < N; i++){
        scanf("%d", &x);
        A[x] += 1;
    }
    for(int i = 0, x; i < M; i++){
        scanf("%d", &x);
        B[x] += 1;
    }

    fft(A, false);
    fft(B, false);
    for(int i = 0; i < SIZE; i++)
        A[i] *= B[i];
    fft(A, true);

    for(int i = 2; i <= 2*K; i++)
        printf("%lld%c", llround(A[i].real()), (" \n")[i==2*K]);
}

```

One Bit Positions

```

#include

using namespace std;
typedef double ld;
typedef complex cd;
const int maxN = 2e5+5;
const int SIZE = 1<<19;
const ld PI = acos(-1);

char S[maxN];
vector A(SIZE), B(SIZE);

void fft(vector &a, bool inv){

```

```

int N = (int) a.size();

for(int i = 1, j = 0; i < N; i++){
    int bit = N>>1;
    for(; j&bit; bit >>= 1)
        j ^= bit;
    j ^= bit;

    if(i < j)
        swap(a[i], a[j]);
}

for(int len = 2; len <= N; len <= 1){
    ld theta = 2*PI / len * (inv ? -1 : 1);
    cd wlen(cos(theta), sin(theta));
    for(int i = 0; i < N; i += len){
        cd w(1);
        for(int j = 0; j < len / 2; j++){
            cd u = a[i+j], v = a[i+j+len/2] * w;
            a[i+j] = u + v;
            a[i+j+len/2] = u - v;
            w *= wlen;
        }
    }
}

if(inv)
    for(cd &z : a)
        z /= N;
}

int main(){
    scanf(" %s", S);
    int N = (int) strlen(S);
    for(int i = 0; i < N; i++){
        if(S[i] == '1'){
            A[i] += 1;
            B[N-i-1] += 1;
        }
    }

    fft(A, false);
    fft(B, false);
    for(int i = 0; i < SIZE; i++)
        A[i] *= B[i];
    fft(A, true);

    for(int i = N; i < 2*N-1; i++)
        printf("%lld%c", llround(A[i].real()), (" \n")[i==2*N-2]);
}

```

Signal Processing

```

#include

using namespace std;
typedef double ld;
typedef complex cd;
const int maxN = 2e5+5;
const int SIZE = 1<<19;
const ld PI = acos(-1);

```

```

int N, M;
vector A(SIZE), B(SIZE);

void fft(vector &a, bool inv){
    int n = (int) a.size();

    for(int i = 1, j = 0; i < n; i++){
        int bit = n>>1;
        for(; j&bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if(i < j)
            swap(a[i], a[j]);
    }

    for(int len = 2; len <= n; len <= 1){
        ld theta = 2*PI / len * (inv ? -1 : 1);
        cd wlen(cos(theta), sin(theta));
        for(int i = 0; i < n; i += len){
            cd w(1);
            for(int j = 0; j < len / 2; j++){
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if(inv)
        for(cd &z : a)
            z /= n;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a; i < N; i++){
        scanf("%d", &a);
        A[i] = a;
    }
    for(int i = 0, b; i < M; i++){
        scanf("%d", &b);
        B[M-i-1] = b;
    }

    fft(A, false);
    fft(B, false);
    for(int i = 0; i < SIZE; i++)
        A[i] *= B[i];
    fft(A, true);

    for(int i = 0; i < N+M-1; i++)
        printf("%lld%c", llround(A[i].real()), (" \n")[i==N+M-2]);
}

```

New Roads Queries

```
#include
```

```

using namespace std;
const int maxN = 4e5+1;

int N, M, Q, x, y, root, idcounter;

struct Node {
    bool rev = false;
    Node *c[2] = {nullptr, nullptr}, *p = nullptr;
    int val, mx, id = ++idcounter;

    Node(int v){
        val = mx = v;
    }

    void push(){
        if(rev){
            swap(c[0], c[1]);
            if(c[0]) c[0]->rev ^= true;
            if(c[1]) c[1]->rev ^= true;
            rev = false;
        }
    }

    void pull(){
        mx = val;
        for(int i = 0; i < 2; i++)
            if(c[i])
                mx = max(mx, c[i]->mx);
    }
} *LCT[maxN];

bool notRoot(Node *t){
    return t->p && (t->p->c[0] == t || t->p->c[1] == t);
}

void rotate(Node *t){
    Node *p = t->p;
    bool b = (p->c[0] == t);
    if((t->p = p->p) && notRoot(p)) t->p->c[(t->p->c[1] == p)] = t;
    if((p->c[!b]=t->c[b])) p->c[!b]->p = p;
    t->c[b] = p;
    p->p = t;
    p->pull();
}

void splay(Node *t){
    while(notRoot(t)){
        Node *p = t->p;
        p->push();
        t->push();
        rotate(t);
    }
    t->push();
    t->pull();
}

Node* access(Node *t){
    Node *last = nullptr;
    for(Node *u = t; u; u = u->p){
        splay(u);
        u->c[1] = last;
        last = u;
    }
    splay(t);
    return last;
}

void evert(Node *t){
    access(t);
}

```

```

    t->rev = true;
}

void link(Node *u, Node *v){
    evert(u);
    u->p = v;
}

void cut(Node *u, Node *v){
    evert(u);
    access(v);
    if(v->c[0]) v->c[0]->p = 0;
    v->c[0] = 0;
    v->pull();
}

Node* path(Node *u, Node *v){
    evert(u);
    access(v);
    return v;
}

Node* LCA(Node *u, Node *v){
    evert(LCT[root]);
    access(u);
    return access(v);
}

bool connected(Node *u, Node *v){
    path(u, v);
    while(v->c[0])
        v = v->c[0];
    return u == v;
}

int main(){
    scanf("%d %d %d", &N, &M, &Q);
    for(int i = 1; i <= N; i++)
        LCT[i] = new Node(0);
    for(int i = 1; i <= M; i++){
        scanf("%d %d", &x, &y);
        LCT[N+i] = new Node(i);
        if(!connected(LCT[x], LCT[y])){
            link(LCT[x], LCT[N+i]);
            link(LCT[y], LCT[N+i]);
        }
    }

    root = 1;
    for(int q = 0; q < Q; q++){
        scanf("%d %d", &x, &y);
        if(connected(LCT[x], LCT[y])){
            Node *p = path(LCT[x], LCT[y]);
            printf("%d\n", p->mx);
        } else printf("-1\n");
    }
}

```

Dynamic Connectivity

```

#include

using namespace std;
typedef pair pii;
typedef array triple;
const int maxN = 1e5+1;
const int SIZE = 3e5+1;
const int INF = 0x3f3f3f3f;

int N, M, K, root, components, del_time[SIZE];
bool used[SIZE];
pii edges[SIZE];
triple ops[maxN];
map> edge_id_map;

struct Node {
    bool rev = false;
    Node *c[2] = {nullptr, nullptr}, *p = nullptr;
    int mn, id;

    Node(int i){
        mn = id = i;
    }

    void push(){
        if(rev){
            swap(c[0], c[1]);
            if(c[0]) c[0]->rev ^= true;
            if(c[1]) c[1]->rev ^= true;
            rev = false;
        }
    }

    void pull(){
        mn = id;
        for(int i = 0; i < 2; i++)
            if(c[i] && del_time[c[i]->mn] < del_time[mn])
                mn = c[i]->mn;
    }
} *LCT[SIZE];

bool notRoot(Node *t){
    return t->p && (t->p->c[0] == t || t->p->c[1] == t);
}

void rotate(Node *t){
    Node *p = t->p;
    bool b = (p->c[0] == t);
    if((t->p = p->p) && notRoot(p)) t->p->c[(t->p->c[1] == p)] = t;
    if((p->c[!b]=t->c[b])) p->c[!b]->p = p;
    t->c[b] = p;
    p->p = t;
    p->pull();
}

void splay(Node *t){
    while(notRoot(t)){
        Node *p = t->p;
        p->push();
        t->push();
        rotate(t);
    }
    t->push();
    t->pull();
}

Node* access(Node *t){
    Node *last = nullptr;

```

```

    for(Node *u = t; u; u = u->p){
        splay(u);
        u->c[1] = last;
        last = u;
    }
    splay(t);
    return last;
}

void evert(Node *t){
    access(t);
    t->rev = true;
}

void link(Node *u, Node *v){
    evert(u);
    u->p = v;
}

void cut(Node *u, Node *v){
    evert(u);
    access(v);
    if(v->c[0]) v->c[0]->p = 0;
    v->c[0] = 0;
    v->pull();
}

Node* path(Node *u, Node *v){
    evert(u);
    access(v);
    return v;
}

bool connected(Node *u, Node *v){
    path(u, v);
    while(v->c[0])
        v = v->c[0];
    return u == v;
}

void create_edge(int u, int v){
    int id = edge_id_map[{u, v}].front();
    if(!connected(LCT[u], LCT[v])){
        components--;
        link(LCT[id], LCT[u]);
        link(LCT[id], LCT[v]);
        used[id] = true;
    } else {
        int mn = path(LCT[u], LCT[v])->mn;
        if(del_time[mn] < del_time[id]){
            int cu = edges[mn].first;
            int cv = edges[mn].second;
            cut(LCT[mn], LCT[cu]);
            cut(LCT[mn], LCT[cv]);
            used[mn] = false;
            link(LCT[id], LCT[u]);
            link(LCT[id], LCT[v]);
            used[id] = true;
        }
    }
}

void destroy_edge(int u, int v){
    int id = edge_id_map[{u, v}].front();
    edge_id_map[{u, v}].pop_front();
    if(!used[id]) return;
    cut(LCT[id], LCT[u]);
    cut(LCT[id], LCT[v]);
    used[id] = false;
    components++;
}

```

```

}

int main(){
    scanf("%d %d %d", &N, &M, &K);

    root = 1;
    for(int i = 1; i <= N; i++){
        LCT[i] = new Node(i);
        del_time[i] = INF;
    }

    for(int i = 1, a, b; i <= M; i++){
        scanf("%d %d", &a, &b);
        if(a > b) swap(a, b);
        edges[N+i] = {a, b};
        del_time[N+i] = INF;
        LCT[N+i] = new Node(N+i);
        edge_id_map[{a, b}].push_back(N+i);
    }

    for(int i = 1, t, a, b; i <= K; i++){
        scanf("%d %d %d", &t, &a, &b);
        if(a > b) swap(a, b);
        ops[i] = {t, a, b};
        edges[N+M+i] = {a, b};

        if(t == 2){
            int id = edge_id_map[{a, b}].back();
            del_time[id] = i;
        } else {
            del_time[N+M+i] = INF;
            LCT[N+M+i] = new Node(N+M+i);
            edge_id_map[{a, b}].push_back(N+M+i);
        }
    }

    components = N;
    for(int i = N+1; i <= N+M; i++){
        int u = edges[i].first;
        int v = edges[i].second;
        create_edge(u, v);
    }
    printf("%d ", components);

    for(int i = 1; i <= K; i++){
        int t = ops[i][0];
        int u = ops[i][1];
        int v = ops[i][2];
        if(t == 1) create_edge(u, v);
        else destroy_edge(u, v);
        printf("%d%c", components, (" \n")[i==K]);
    }
}

```

Parcel Delivery

```

#include

using namespace std;
typedef long long ll;
const int maxN = 501, maxM = 1001;

```

```

const ll INF = 0x3f3f3f3f3f3f3f3f;

int N, M, K, p[maxN];
ll d[maxN], cap[maxN][maxN], cost[maxN][maxN];
bool inq[maxN];
vector G[maxN];

struct Edge {
    int u, v;
    ll r, c;
} edges[maxM], redges[maxM];

void bellman_ford(){
    fill(inq+1, inq+N+1, false);
    fill(d+1, d+N+1, INF);
    fill(p+1, p+N+1, 0);

    queue Q;
    Q.push(1);
    d[1] = 0;
    inq[1] = true;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        inq[u] = false;

        for(int i : G[u]){
            Edge e = (i < 0 ? redges[-i] : edges[i]);
            if(e.r > 0 && d[e.v] > d[u] + e.c){
                d[e.v] = d[u] + e.c;
                p[e.v] = i;
                if(!inq[e.v]){
                    inq[e.v] = true;
                    Q.push(e.v);
                }
            }
        }
    }
}

ll minimum_cost_flow(){
    ll flow = 0, cost = 0;
    while(flow < K){
        bellman_ford();
        if(d[N] == INF) break;

        ll aug = K-flow;
        int u = N;
        while(u != 1){
            Edge e = (p[u] < 0 ? redges[-p[u]] : edges[p[u]]);
            aug = min(aug, e.r);
            u = e.u;
        }

        flow += aug;
        cost += aug * d[N];
        u = N;
        while(u != 1){
            if(p[u] < 0){
                redges[-p[u]].r -= aug;
                edges[-p[u]].r += aug;
            } else {
                redges[p[u]].r += aug;
                edges[p[u]].r -= aug;
            }
            u = (p[u] < 0 ? redges[-p[u]].u : edges[p[u]].u);
        }
    }
    return (flow < K ? -1 : cost);
}

```

```

int main(){
    scanf("%d %d %d", &N, &M, &K);
    for(int i = 1, u, v, r, c; i <= M; i++){
        scanf("%d %d %d %d", &u, &v, &r, &c);
        G[u].push_back(i);
        G[v].push_back(-i);
        edges[i] = {u, v, r, c};
        redges[i] = {v, u, 0, -c};
    }
    printf("%lld\n", minimum_cost_flow());
}

```

Task Assignment

```

#include

using namespace std;
typedef long long ll;
const int maxN = 402, maxM = 40401;
const ll INF = 0x3f3f3f3f3f3f3f3f;

int N, p[maxN];
ll d[maxN], cap[maxN][maxN], cost[maxN][maxN];
bool inq[maxN], vis[maxM];
vector path, G[maxN];

struct Edge {
    int u, v;
    ll r, c;
} edges[maxM], redges[maxM];

void bellman_ford(int start = 0){
    fill(inq, inq+maxN, false);
    fill(d, d+maxN, INF);
    fill(p, p+maxN, 0);

    queue Q;
    Q.push(start);
    d[start] = 0;
    inq[start] = true;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        inq[u] = false;

        for(int i : G[u]){
            Edge e = (i < 0 ? redges[-i] : edges[i]);
            if(e.r > 0 && d[e.v] > d[u] + e.c){
                d[e.v] = d[u] + e.c;
                p[e.v] = i;
                if(!inq[e.v]){
                    inq[e.v] = true;
                    Q.push(e.v);
                }
            }
        }
    }
}

ll minimum_cost_flow(){
    ll flow = 0, cost = 0;
    while(flow < N){

```

```

    bellman_ford();
    if(d[2*N+1] == INF) break;

    ll aug = N-flow;
    int u = 2*N+1;
    while(u != 0){
        Edge e = (p[u] < 0 ? redges[-p[u]] : edges[p[u]]);
        aug = min(aug, e.r);
        u = e.u;
    }

    flow += aug;
    cost += aug * d[2*N+1];
    u = 2*N+1;
    while(u != 0){
        if(p[u] < 0){
            redges[-p[u]].r -= aug;
            edges[-p[u]].r += aug;
        } else {
            redges[p[u]].r += aug;
            edges[p[u]].r -= aug;
        }
        u = (p[u] < 0 ? redges[-p[u]].u : edges[p[u]].u);
    }
}
return (flow < N ? -1 : cost);
}

void dfs(int u = 0){
    if(u == 2*N+1) return;
    if(u != 0) path.push_back(u);
    for(int i : G[u]){
        if(i > 0 && edges[i].r == 0 && !vis[i]){
            vis[i] = true;
            dfs(edges[i].v);
            return;
        }
    }
}

int main(){
    scanf("%d", &N);
    int edgeID = 1;
    for(int u = 1; u <= N; u++){
        for(int v = N+1, c; v <= 2*N; v++){
            scanf("%d", &c);
            G[u].push_back(edgeID);
            G[v].push_back(-edgeID);
            edges[edgeID] = {u, v, 1, c};
            redges[edgeID] = {v, u, 0, -c};
            edgeID++;
        }
    }
    for(int v = 1; v <= N; v++){
        G[0].push_back(edgeID);
        G[v].push_back(-edgeID);
        edges[edgeID] = {0, v, 1, 0};
        redges[edgeID] = {v, 0, 0, 0};
        edgeID++;
    }
    for(int u = N+1; u <= 2*N; u++){
        G[u].push_back(edgeID);
        G[2*N+1].push_back(-edgeID);
        edges[edgeID] = {u, 2*N+1, 1, 0};
        redges[edgeID] = {2*N+1, u, 0, 0};
        edgeID++;
    }

    printf("%lld\n", minimum_cost_flow());
    for(int i = 0; i < N; i++){

```

```

        path.clear();
        dfs();
        printf("%d %d\n", path[0], path[1]-N);
    }
}

```

Distinct Routes II

```

#include

using namespace std;
const int maxN = 501, maxM = 1001;
const int INF = 0x3f3f3f3f;

int N, M, K, p[maxN], d[maxN];
bool inq[maxN], vis[maxM];
vector path, G[maxN];

struct Edge {
    int u, v, r, c;
} edges[maxM], redges[maxM];

void bellman_ford(){
    fill(inq+1, inq+N+1, false);
    fill(d+1, d+N+1, INF);
    fill(p+1, p+N+1, 0);

    queue Q;
    Q.push(1);
    d[1] = 0;
    inq[1] = true;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        inq[u] = false;

        for(int i : G[u]){
            Edge e = (i < 0 ? redges[-i] : edges[i]);
            if(e.r > 0 && d[e.v] > d[u] + e.c){
                d[e.v] = d[u] + e.c;
                p[e.v] = i;
                if(!inq[e.v]){
                    inq[e.v] = true;
                    Q.push(e.v);
                }
            }
        }
    }
}

int minimum_cost_flow(){
    int flow = 0, cost = 0;
    while(flow < K){
        bellman_ford();
        if(d[N] == INF) break;

        int aug = K-flow;
        int u = N;
        while(u != 1){
            Edge e = (p[u] < 0 ? redges[-p[u]] : edges[p[u]]);
            aug = min(aug, e.r);
            u = e.u;
        }
    }
}

```

```

    }

    flow += aug;
    cost += aug * d[N];
    u = N;
    while(u != 1){
        if(p[u] < 0){
            redges[-p[u]].r -= aug;
            edges[-p[u]].r += aug;
        } else {
            redges[p[u]].r += aug;
            edges[p[u]].r -= aug;
        }
        u = (p[u] < 0 ? redges[-p[u]].u : edges[p[u]].u);
    }
}
return (flow < K ? -1 : cost);
}

void dfs(int u = 1){
    path.push_back(u);
    if(u == N) return;
    for(int i : G[u]){
        if(i > 0 && edges[i].r == 0 && !vis[i]){
            vis[i] = true;
            dfs(edges[i].v);
            return;
        }
    }
}

int main(){
    scanf("%d %d %d", &N, &M, &K);
    for(int i = 1, u, v; i <= M; i++){
        scanf("%d %d", &u, &v);
        G[u].push_back(i);
        G[v].push_back(-i);
        edges[i] = {u, v, 1, 1};
        redges[i] = {v, u, 0, -1};
    }

    int minCoins = minimum_cost_flow();
    if(minCoins == -1){
        printf("-1\n");
        return 0;
    }

    printf("%d\n", minCoins);
    for(int i = 0; i < K; i++){
        path.clear();
        dfs();

        int sz = (int) path.size();
        printf("%d\n", sz);
        for(int j = 0; j < sz; j++)
            printf("%d%c", path[j], (" \n")[j==sz-1]);
    }
}

```

Sliding

Sliding Window Sum

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e7+1;

int N, K;
ll a, b, c, pre[maxN];

int main(){
    scanf("%d %d", &N, &K);
    scanf("%lld %lld %lld %lld", &pre[1], &a, &b, &c);

    for(int i = 2; i <= N; i++)
        pre[i] = (a * pre[i-1] + b) % c;

    for(int i = 2; i <= N; i++)
        pre[i] += pre[i-1];

    ll xum = 0;
    for(int i = K; i <= N; i++)\
        xum ^= (pre[i] - pre[i-K]);
    printf("%lld\n", xum);
}
```

Sliding Window Minimum

```
#include

using namespace std;
typedef long long ll;
typedef pair pii;

struct Window {
    deque dq;
    int l, r;

    Window(){
        l = r = 0;
    }

    void push_back(ll x){
        while(!dq.empty() && dq.back().first >= x)    dq.pop_back();
        dq.push_back({x, r});
        r++;
    }

    void pop_front(){
        if(dq.front().second == l)    dq.pop_front();
        l++;
    }
}
```

```

    }

    ll get_min(){
        return dq.empty() ? -1 : dq.front().first;
    }
};

int N, K;
ll x, a, b, c;

int main(){
    scanf("%d %d", &N, &K);
    scanf("%lld %lld %lld %lld", &x, &a, &b, &c);

    Window window;
    for(int i = 0; i < K; i++){
        window.push_back(x);
        x = (a * x + b) % c;
    }

    ll xum = window.get_min();
    for(int i = K; i < N; i++){
        window.push_back(x);
        window.pop_front();
        xum ^= window.get_min();
        x = (a * x + b) % c;
    }

    printf("%lld\n", xum);
}

```

Sliding Window Xor

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e7+1;

int N, K;
ll x, a, b, c;

int main(){
    scanf("%d %d", &N, &K);
    scanf("%lld %lld %lld %lld", &x, &a, &b, &c);

    ll xum = 0;
    for(int i = 1; i <= N; i++){
        int l = max(1, i-K+1);
        int r = min(i, N-K+1);
        if((r - l) % 2 == 0)
            xum ^= x;

        x = (a * x + b) % c;
    }
    printf("%lld\n", xum);
}

```

Sliding Window Or

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e7+5;

int N, K, val[maxN], pre[maxN], suf[maxN];
ll x, a, b, c;

int main(){
    scanf("%d %d", &N, &K);
    scanf("%lld %lld %lld %lld", &x, &a, &b, &c);
    for(int i = 0; i < N; i++){
        val[i] = x;
        x = (a * x + b) % c;
    }

    for(int i = 0; i < N; i++)
        pre[i] = (i % K == 0) ? val[i] : (pre[i-1] | val[i]);
    for(int i = N-1; i >= 0; i--)
        suf[i] = (i % K == K-1) ? val[i] : (suf[i+1] | val[i]);

    int xor_sum = 0;
    for(int i = 0; i <= N-K; i++){
        int l = i, r = i + K - 1;
        int or_val = suf[l] | pre[r];
        xor_sum ^= or_val;
    }

    printf("%d\n", xor_sum);
}
```

Sliding Window Distinct Values

```
#include

using namespace std;
const int maxN = 2e5;

int N, K, x[maxN];
map freq;

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < N; i++)
        scanf("%d", &x[i]);

    int distinct = 0;
    for(int i = 0; i < K; i++){
        if(freq[x[i]] == 0)
            distinct++;
        freq[x[i]]++;
    }
    printf("%d%c", distinct, (" \n")[N==K]);

    for(int i = K; i < N; i++){
        if(freq[x[i-K]] == 1)
            distinct--;
```

```

        distinct--;
        freq[x[i-K]]--;

        if(freq[x[i]] == 0)
            distinct++;
        freq[x[i]]++;

        printf("%d%c", distinct, (" \n")[i==N-1]);
    }
}

```

Sliding Window Mode

```

#include

using namespace std;
const int maxN = 2e5;

int N, K, best_freq, best_val, a[maxN];
unordered_map freqs;
unordered_map> vals;

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < N; i++)
        scanf("%d", &a[i]);

    for(int i = 0; i < K; i++)
        freqs[a[i]]++;

    for(int i = 0; i < K; i++){
        vals[freqs[a[i]]].insert(a[i]);
        if(freqs[a[i]] > best_freq || (freqs[a[i]] == best_freq && a[i] < best_val)){
            best_freq = freqs[a[i]];
            best_val = a[i];
        }
    }

    printf("%d%c", best_val, (" \n")[N==K]);
    for(int i = K; i < N; i++){
        vals[freqs[a[i-K]]].erase(a[i-K]);
        vals[--freqs[a[i-K]]].insert(a[i-K]);
        if(best_val == a[i-K]){
            for(int f = freqs[a[i-K]]+1; f > 0; f--){
                if(!vals[f].empty()){
                    best_freq = f;
                    best_val = *vals[f].begin();
                    break;
                }
            }
        }
    }

    vals[freqs[a[i]]].erase(a[i]);
    vals[++freqs[a[i]]].insert(a[i]);
    if(freqs[a[i]] > best_freq || (freqs[a[i]] == best_freq && a[i] < best_val)){
        best_freq = freqs[a[i]];
        best_val = a[i];
    }

    printf("%d%c", best_val, (" \n")[i==N]);
}

```

```

    }
}

```

Sliding Window Mex

```

#include

using namespace std;
const int maxN = 2e5;

int N, K, a[maxN];
set mex_set;
unordered_map freq;

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < N; i++)
        scanf("%d", &a[i]);

    for(int i = 0; i <= N; i++)
        mex_set.insert(i);

    for(int i = 0; i < K; i++){
        freq[a[i]]++;
        if(freq[a[i]] == 1)
            mex_set.erase(a[i]);
    }

    printf("%d%c", *mex_set.begin(), (" \n")[N==K]);
    for(int i = K; i < N; i++) {
        freq[a[i-K]]--;
        if(freq[a[i-K]] == 0)
            mex_set.insert(a[i-K]);

        freq[a[i]]++;
        if(freq[a[i]] == 1)
            mex_set.erase(a[i]);

        printf("%d%c", *mex_set.begin(), (" \n")[i==N-1]);
    }
}

```

Sliding Window Median

```

#include
#include
#include

using namespace std;
using namespace __gnu_pbds;
typedef pair pii;
const int maxN = 2e5;

```

```

int N, K, x[maxN];
tree, rb_tree_tag,
tree_order_statistics_node_update> T;

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < K; i++){
        scanf("%d", &x[i]);
        T.insert({x[i], i});
    }

    printf("%d%c", (*T.find_by_order((K-1)/2)).first, (" \n")[N==K]);
    for(int i = K; i < N; i++){
        scanf("%d", &x[i]);
        T.insert({x[i], i});
        T.erase({x[i-K], i-K});
        printf("%d%c", (*T.find_by_order((K-1)/2)).first, (" \n")[i==N-1]);
    }
}

```

Sliding Window Cost

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, K, x[maxN];
ll losum, hisum;
multiset lo, hi;

ll cost(){
    ll median = *lo.rbegin();
    ll locost = (median * lo.size()) - losum;
    ll hicost = hisum - (median * hi.size());
    return locost + hicost;
}

void lowToHigh(){
    ll val = *lo.rbegin();
    lo.erase(lo.lower_bound(val));
    hi.insert(val);
    losum -= val;
    hisum += val;
}

void highToLow(){
    ll val = *hi.begin();
    lo.insert(val);
    hi.erase(hi.lower_bound(val));
    losum += val;
    hisum -= val;
}

void adjust(){
    int tot = (int) (lo.size() + hi.size());
    if(lo.size() < (tot+1)/2) highToLow();
    else if(lo.size() > (tot+1)/2) lowToHigh();
}

```

```

void erase(ll val){
    ll median = *lo.rbegin();
    if(val > median){
        hi.erase(hi.lower_bound(val));
        hisum -= val;
    } else {
        lo.erase(lo.lower_bound(val));
        losum -= val;
    }
    adjust();
}

void insert(ll val){
    if(lo.size() == 0){
        lo.insert(val);
        losum += val;
        return;
    }

    ll median = *lo.rbegin();
    if(val > median){
        hi.insert(val);
        hisum += val;
    } else {
        lo.insert(val);
        losum += val;
    }
    adjust();
}

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < K; i++){
        scanf("%d", &x[i]);
        insert(x[i]);
    }

    printf("%lld%c", cost(), (" \n")[K==N]);
    for(int i = K; i < N; i++){
        scanf("%d", &x[i]);
        insert(x[i]);
        erase(x[i-K]);
        printf("%lld%c", cost(), (" \n")[i==N-1]);
    }
}

```

Sliding Window Inversions

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+5;

int N, K, a[maxN], ds[maxN];

void update(int idx, int val){
    for(int i = idx; i <= N; i += -i&i)
        ds[i] += val;
}

```

```
int query(int idx){
    int sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += ds[i];
    return sum;
}

void compress_vals(){
    vector vals;
    for(int i = 0; i < N; i++)
        vals.push_back(a[i]);
    sort(vals.begin(), vals.end());

    unordered_map mp;
    mp.reserve(N);
    for(int i = 0; i < N; i++)
        if(mp.find(vals[i]) == mp.end())
            mp[vals[i]] = i+1;

    for(int i = 0; i < N; i++)
        a[i] = mp[a[i]];
}

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < N; i++)
        scanf("%d", &a[i]);
    compress_vals();

    ll inversions = 0;
    for(int i = 0; i < K; i++){
        inversions += i - query(a[i]);
        update(a[i], 1);
    }

    printf("%lld%c", inversions, (" \n")[K==N]);
    for(int i = K; i < N; i++){
        inversions -= query(a[i-K]-1);
        update(a[i-K], -1);

        inversions += (K-1) - query(a[i]);
        update(a[i], 1);

        printf("%lld%c", inversions, (" \n")[i==N-1]);
    }
}
```

Bitwise

Counting Bits

```
#include

using namespace std;
typedef long long ll;

ll N;
map dp;

ll f(ll a){
    if(a == 0) return 0;
    if(dp[a]) return dp[a];
    if(a&1) return dp[a] = 2*f(a/2)+(a/2)+1;
    else return dp[a] = f(a/2)+f(a/2-1)+(a/2);
}

int main(){
    scanf("%lld", &N);
    printf("%lld\n", f(N));
}
```

Maximum Xor Subarray

```
#include

using namespace std;

struct Node {
    Node *c[2];
};

int N, xum, best;
Node *root;

void update(int x){
    Node *cur = root;
    for(int i = 30; i >= 0; i--){
        if(x&(1<c[1]) cur->c[1] = new Node();
        cur = cur->c[1];
    } else {
        if(!cur->c[0]) cur->c[0] = new Node();
        cur = cur->c[0];
    }
}

int query(int x){
    int res = 0;
    Node *cur = root;
    for(int i = 30; i >= 0; i--){
        if(x&(1<c[0])){
```

```

        res += (1<c[0];
    } else cur = cur->c[1];
} else {
    if(cur->c[1]){
        res += (1<c[1];
    } else cur = cur->c[0];
}
}
return res;
}

int main(){
    scanf("%d", &N);
    root = new Node();
    update(0);
    for(int i = 0, x; i < N; i++){
        scanf("%d", &x);
        xum ^= x;
        update(xum);
        best = max(best, query(xum));
    }
    printf("%d\n", best);
}

```

Maximum Xor Subset

```

#include

using namespace std;
const int logX = 32;

int N, basis[logX];

void insert_vector(int mask){
    for(int i = logX-1; i >= 0; i--){
        if((mask&(1<= 0; i--){
            if(!basis[i])
                basis[i] = mask;
            else if((basis[i] ^ mask) < basis[i])
                basis[i] = basis[i] ^ mask;
        }
    }
}

```

Number of Subset Xors

```

#include

using namespace std;
typedef long long ll;
const int logX = 32;

int N, basis[logX];

bool insert_vector(int mask){

```

```
for(int i = logX-1; i >= 0; i--){
    if((mask&(1<
```

Xor Pyramid Peak

```
#include

using namespace std;
const int maxN = 2e5+5;

int N, D, a, xum;

int two_divisibility(int x){
    int res = 0;
    while(x){
        x >>= 1;
        res += x;
    }
    return res;
}

int main(){
    scanf("%d", &N);
    D = two_divisibility(N-1);
    for(int i = 0; i < N; i++){
        scanf("%d", &a);
        int d1 = two_divisibility(i);
        int d2 = two_divisibility(N-i-1);
        if(D - d1 - d2 == 0)
            xum ^= a;
    }
    printf("%d\n", xum);
}
```

Xor Pyramid Diagonal

```
#include

using namespace std;
const int maxN = 2e5;

int N, a[maxN];

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%d", &a[i]);

    for(int i = 0; i < N; i++){
        int ans = a[0];
        for(int subset = i; subset > 0; subset = (subset-1)&i)
            ans ^= a[subset];
    }
}
```

```
        printf("%d%c", ans, (" \n")[i==N-1]);  
    }  
}
```

Construction

Inverse Inversions

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e6+1;

int N, idx, ans[maxN];
ll K;
deque DQ;

int main(){
    scanf("%d %lld", &N, &K);
    for(int i = 1; i <= N; i++)
        DQ.push_back(i);

    for(int k = N-1; k >= 0; k--){
        if(k <= K){
            ans[++idx] = DQ.back();
            DQ.pop_back();
            K -= k;
        } else {
            ans[++idx] = DQ.front();
            DQ.pop_front();
        }
    }

    for(int i = 1; i <= idx; i++)
        printf("%d%c", ans[i], (" \n")[i==idx]);
}
```

Monotone Subsequences

```
#include

using namespace std;

int N, K;

void solve(){
    scanf("%d %d", &N, &K);
    if(K*K < N){
        printf("IMPOSSIBLE\n");
        return;
    }

    vector ans;
    int l = 1, r = K;
    while(true){
        for(int i = r; i >= l; i--)
            ans.push_back(i);
```

```

        if(r == N) break;
        l = r+1;
        r = min(r+K, N);
    }

    for(int i = 0; i < N; i++)
        printf("%d%c", ans[i], (" \n")[i==N-1]);
}

int main(){
    int T;
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        solve();
    }
}

```

Chess Tournament

```

#include
#include
#include

using namespace std;
using namespace __gnu_pbds;
typedef pair pii;

int N, K;
vector added, ans;
tree, rb_tree_tag,
tree_order_statistics_node_update> T;

int main(){
    scanf("%d", &N);
    for(int i = 1, x; i <= N; i++){
        scanf("%d", &x);
        T.insert({x, i});
    }

    for(int i = N-1; i >= 0; i--){
        pii P = *T.find_by_order(i);
        T.erase(T.find_by_order(i));
        int degu = P.first;
        int u = P.second;

        added.clear();
        for(int j = 1; j <= degu; j++){
            if(i-j < 0 || (*T.find_by_order(i-j)).first == 0){
                printf("IMPOSSIBLE\n");
                return 0;
            }

            pii Q = *T.find_by_order(i-j);
            T.erase(T.find_by_order(i-j));
            int degv = Q.first;
            int v = Q.second;

            ans.push_back({min(u, v), max(u, v)});
            added.push_back({degv-1, v});
        }
    }
}

```

```
        for(pii Q : added)
            T.insert({Q.first, Q.second});
    }

    sort(ans.begin(), ans.end());
    K = (int) ans.size();
    printf("%d\n", K);
    for(int i = 0; i < K; i++)
        printf("%d %d\n", ans[i].first, ans[i].second);
}
```

Advanced Graph Problems

Tree Traversals

```
#include

using namespace std;
const int maxN = 1e5+5;

int N, timer, A[maxN], B[maxN], C[maxN], idx[maxN];
int L[maxN], R[maxN];

int solve(int l1 = 0, int r1 = N-1, int l2 = 0, int r2 = N-1){
    if(l1 > r1 || l2 > r2) return 0;
    int root = A[l1];
    int m2 = idx[root];
    int len = m2-l2;
    L[root] = solve(l1+1, l1+len, l2, m2-1);
    R[root] = solve(l1+len+1, r1, m2+1, r2);
    return root;
}

void dfs(int u){
    if(L[u]) dfs(L[u]);
    if(R[u]) dfs(R[u]);
    C[timer++] = u;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++) scanf("%d", &A[i]);
    for(int i = 0; i < N; i++){
        scanf("%d", &B[i]);
        idx[B[i]] = i;
    }

    int root = solve();
    dfs(root);
    for(int i = 0; i < N; i++)
        printf("%d%c", C[i], (" \n")[i==N-1]);
}
```

Course Schedule II

```
#include

using namespace std;
const int maxN = 2e5;

int N, M, a, b, idx, in[maxN], ans[maxN];
vector G[maxN];
priority_queue Q;

int main(){
```

```

scanf("%d %d", &N, &M);
for(int i = 0; i < M; i++){
    scanf("%d %d", &a, &b);
    G[b].push_back(a);
    in[a]++;
}

for(int i = 1; i <= N; i++)
    if(in[i] == 0)
        Q.push(i);

idx = N;
while(!Q.empty()){
    int u = Q.top(); Q.pop();
    ans[idx--] = u;
    for(int v : G[u]){
        in[v]--;
        if(in[v] == 0)
            Q.push(v);
    }
}

for(int i = 1; i <= N; i++)
    printf("%d%c", ans[i], (" \n")[i==N]);
}

```

Acyclic Graph Edges

```

#include

using namespace std;
int N, M, a, b;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        printf("%d %d\n", min(a, b), max(a, b));
    }
}

```

Strongly Connected Edges

```

#include

using namespace std;
typedef pair pii;
const int maxN = 1e5+1;
const int maxM = 2e5+1;

int N, M, timer, tin[maxN], low[maxN];
bool bridge_exists, vis[maxN], used[maxM];

```

```

vector G[maxN];
pii edges[maxM];

void dfs(int u = 1, int p = 0){
    vis[u] = true;
    tin[u] = low[u] = ++timer;
    for(pii P : G[u]){
        int v = P.first;
        int id = P.second;
        if(used[id]) continue;

        used[id] = true;
        edges[id] = {u, v};

        if(vis[v]){
            low[u] = min(low[u], tin[v]);
        } else {
            dfs(v, u);
            low[u] = min(low[u], low[v]);
            if(low[v] > tin[u])
                bridge_exists = true;
        }
    }
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back({b, i});
        G[b].push_back({a, i});
    }

    dfs();
    if(timer == N && !bridge_exists){
        for(int i = 0; i < M; i++)
            printf("%d %d\n", edges[i].first, edges[i].second);
    } else printf("IMPOSSIBLE\n");
}

```

Even Outdegree Edges

```

#include

using namespace std;
typedef pair pii;
const int maxN = 1e5+5, maxM = 2e5+5;

int N, M, timer, in[maxN];
bool even[maxN];
vector G[maxN];
pii ans[maxM];

void dfs(int u, int p = -1){
    in[u] = ++timer;
    for(pii e : G[u]){
        int v = e.first, id = e.second;
        if(v != p){
            if(!in[v]){
                dfs(v, u);
                if(even[v]){

```

```

        ans[id] = {u, v};
        even[u] ^= true;
    } else {
        ans[id] = {v, u};
        even[v] ^= true;
    }
} else if(in[v] < in[u]){
    even[u] ^= true;
    ans[id] = {u, v};
}
}
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back({b, i});
        G[b].push_back({a, i});
    }

    fill(even+1, even+N+1, true);
    for(int i = 1; i <= N; i++)
        if(!in[i])
            dfs(i);

    bool good = true;
    for(int i = 1; i <= N; i++)
        good &= even[i];

    if(good){
        for(int i = 0; i < M; i++)
            printf("%d %d\n", ans[i].first, ans[i].second);
    } else printf("IMPOSSIBLE\n");
}

```

Graph Girth

```

#include

using namespace std;
const int maxN = 2501;

bool vis[maxN];
int N, M, best, p[maxN], dist[maxN];
vector G[maxN];

void reset(){
    fill(vis+1, vis+N+1, false);
    fill(dist+1, dist+N+1, 0);
    fill(p+1, p+N+1, 0);
}

void bfs(int start){
    queue Q;
    Q.push(start);
    vis[start] = true;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        for(int v : G[u]){

```

```

        if(!vis[v]){
            dist[v] = dist[u]+1;
            vis[v] = true;
            p[v] = u;
            Q.push(v);
        } else if(v != u[p]){
            best = min(best, dist[u]+dist[v]+1);
        }
    }
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    best = N+1;
    for(int start = 1; start <= N; start++){
        reset();
        bfs(start);
    }

    printf("%d\n", best == N+1 ? -1 : best);
}

```

Transfer Speeds Sum

```

#include

using namespace std;
typedef long long ll;
typedef array edge;
const int maxN = 2e5+1;

int N, ds[maxN];
ll ans;
vector edges;

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d", &N);
    for(int i = 0, a, b, x; i < N-1; i++){
        scanf("%d %d %d", &a, &b, &x);
    }
}

```

```

        edges.push_back({a, b, x});
    }

    sort(edges.begin(), edges.end(), [](const edge& e1, const edge& e2){
        return e1[2] > e2[2];
    });

    fill(ds, ds+N, -1);
    for(const edge& e: edges){
        int a = find(e[0]), b = find(e[1]);
        int sza = -ds[a], szb = -ds[b];

        ll x = e[2];
        ans += x * sza * szb;
        merge(a, b);
    }
    printf("%lld\n", ans);
}

```

Network Breakdown

```

#include

using namespace std;
const int maxN = 1e5+1;
const int maxM = 2e5;
typedef pair pii;

int N, M, K, a, b, cnt, ds[maxN], ans[maxM];
pii edges[maxM], queries[maxM];
set S;

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d %d", &N, &M, &K);
    fill(ds+1, ds+N+1, -1);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        if(b > a) swap(a, b);
        edges[i] = {a, b};
    }
    for(int i = 0; i < K; i++){
        scanf("%d %d", &a, &b);
        if(b > a) swap(a, b);
        queries[i] = {a, b};
        S.insert({a, b});
    }
}

```

```

cnt = N;
for(int i = 0; i < M; i++)
    if(S.find(edges[i]) == S.end())
        if(merge(edges[i].first, edges[i].second))
            cnt--;

for(int i = K-1; i >= 0; i--){
    ans[i] = cnt;
    if(merge(queries[i].first, queries[i].second))
        cnt--;
}

for(int i = 0; i < K; i++)
    printf("%d%c", ans[i], (" \n")[i==K-1]);
}

```

Tree Isomorphism I

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, treeID, name[2][maxN];
vector G[2][maxN];
map<int> mp;

void reset(){
    for(int t = 0; t < 2; t++)
        for(int i = 1; i <= N; i++)
            G[t][i].clear();
}

void dfs(int t, int u, int p){
    vector childNames;
    for(int v : G[t][u]){
        if(v != p){
            dfs(t, v, u);
            childNames.push_back(name[t][v]);
        }
    }
    sort(childNames.begin(), childNames.end());
    if(!mp[childNames]) mp[childNames] = ++treeID;
    name[t][u] = mp[childNames];
}

void solve_case(){
    scanf("%d", &N);
    reset();
    for(int t = 0; t < 2; t++){
        for(int i = 0, a, b; i < N-1; i++){
            scanf("%d %d", &a, &b);
            G[t][a].push_back(b);
            G[t][b].push_back(a);
        }
        dfs(t, 1, -1);
    }
    printf("%s\n", name[0][1] == name[1][1] ? "YES" : "NO");
}

```

```
int main(){
    int T;
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        solve_case();
    }
}
```

Tree Isomorphism II

```
#include

using namespace std;
const int maxN = 1e5+1;

int N, treeID, sz[2][maxN], name[2][maxN];
vector centroids[2], G[2][maxN];
map<int> mp;

void reset(){
    mp.clear();
    treeID = 0;
    for(int t = 0; t < 2; t++){
        centroids[t].clear();
        for(int i = 1; i <= N; i++){
            sz[t][i] = name[t][i] = 0;
            G[t][i].clear();
        }
    }
}

void dfs1(int t, int u, int p){
    sz[t][u] = 1;
    bool is_centroid = true;
    for(int v : G[t][u]){
        if(v != p){
            dfs1(t, v, u);
            sz[t][u] += sz[t][v];
            if(sz[t][v] > N/2) is_centroid = false;
        }
    }
    if(N-sz[t][u] > N/2) is_centroid = false;
    if(is_centroid) centroids[t].push_back(u);
}

void dfs2(int t, int u, int p){
    vector childNames;
    for(int v : G[t][u]){
        if(v != p){
            dfs2(t, v, u);
            childNames.push_back(name[t][v]);
        }
    }
    sort(childNames.begin(), childNames.end());
    if(!mp[childNames]) mp[childNames] = ++treeID;
    name[t][u] = mp[childNames];
}

void solve_case(){
    scanf("%d", &N);
    reset();
```

```

for(int t = 0; t < 2; t++){
    for(int i = 0, a, b; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[t][a].push_back(b);
        G[t][b].push_back(a);
    }
    dfs1(t, 1, -1);
}

for(int root1 : centroids[0]){
    for(int root2 : centroids[1]){
        dfs2(0, root1, -1);
        dfs2(1, root2, -1);
        if(name[0][root1] == name[1][root2]){
            printf("YES\n");
            return;
        }
    }
}
printf("NO\n");
}

int main(){
    int T;
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        solve_case();
    }
}

```

Flight Route Requests

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, ds[maxN], vis[maxN];
vector G[maxN], CC[maxN];

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

bool dfs(int u){
    vis[u] = -1;
    bool hascycle = false;
    for(int v : G[u]){
        if(vis[v] == -1) return true;
    }
}

```

```

        else if(vis[v] == 0)    hascycle |= dfs(v);
    }
    vis[u] = 1;
    return hascycle;
}

int main(){
    scanf("%d %d", &N, &M);

    fill(ds+1, ds+N+1, -1);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        merge(a, b);
    }

    int K = 0;
    unordered_map getID;
    for(int u = 1; u <= N; u++){
        int rep = find(u);
        if(!getID[rep])
            getID[rep] = ++K;
        CC[getID[rep]].push_back(u);
    }

    int ans = 0;
    for(int k = 1; k <= K; k++){
        int sz = (int) CC[k].size();
        bool hascycle = false;
        for(int u : CC[k])
            if(!hascycle && vis[u] == 0)
                hascycle |= dfs(u);
        ans += (hascycle ? sz : sz-1);
    }
    printf("%d\n", ans);
}

```

Critical Cities

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, p[maxN], sdom[maxN], idom[maxN], label[maxN];
int timer, tin[maxN], inv[maxN], dsu[maxN];
vector ans, G[maxN], GR[maxN], DT[maxN], bucket[maxN];

void dfs(int u = 1){
    tin[u] = ++timer;
    inv[timer] = u;
    label[timer] = sdom[timer] = dsu[timer] = timer;
    for(int v : G[u]){
        if(!tin[v]){
            dfs(v);
            p[tin[v]] = tin[u];
        }
        GR[tin[v]].push_back(tin[u]);
    }
}

```

```

int find(int u, bool x = false){
    if(u == dsu[u]) return x ? -1 : u;
    int v = find(dsu[u], true);
    if(v < 0) return u;

    if(sdom[label[dsu[u]]] < sdom[label[u]])
        label[u] = label[dsu[u]];

    dsu[u] = v;
    return x ? v : label[u];
}

void build_dominator_tree(){
    dfs();

    for(int u = N; u > 0; u--){
        for(int v : GR[u])
            sdom[u] = min(sdom[u], sdom[find(v)]);
        if(u > 1) bucket[sdom[u]].push_back(u);
        for(int v : bucket[u])
            idom[v] = (sdom[find(v)] == sdom[v] ? sdom[v] : find(v));
        if(u > 1) dsu[u] = p[u];
    }

    for(int u = 2; u <= N; u++){
        if(idom[u] != sdom[u])
            idom[u] = idom[idom[u]];
        DT[inv[u]].push_back(inv[idom[u]]);
        DT[inv[idom[u]]].push_back(inv[u]);
    }
}

bool dfs_dominator_tree(int u = 1, int p = -1){
    bool good = (u == N);
    for(int v : DT[u])
        if(v != p)
            good |= dfs_dominator_tree(v, u);
    if(good) ans.push_back(u);
    return good;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
    }

    build_dominator_tree();
    dfs_dominator_tree();

    sort(ans.begin(), ans.end());
    int K = (int) ans.size();
    printf("%d\n", K);
    for(int i = 0; i < K; i++)
        printf("%d%c", ans[i], (" \n")[i==K-1]);
}

```

Visiting Cities

```

#include

using namespace std;
typedef long long ll;
typedef pair edge;
typedef pair node;
const int maxN = 1e5+1;
const ll INF = 0x3f3f3f3f3f3f3f3f;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution distrib((ll) 1e9, (ll) 2e9);
const ll MOD1 = distrib(rng);
const ll MOD2 = distrib(rng);

int N, M;
ll dist[2][maxN], ways[2][2][maxN];
vector G[2][maxN];
vector ans;

void init(){
    for(int t = 0; t < 2; t++){
        for(int i = 1; i <= N; i++){
            dist[t][i] = INF;
        }
    }

    void dijkstra(int type, int source){
        priority_queue, greater> Q;
        dist[type][source] = 0;
        ways[type][0][source] = 1;
        ways[type][1][source] = 1;
        Q.push({0, source});
        while(!Q.empty()){
            int u = Q.top().second;
            ll d = Q.top().first;
            Q.pop();

            if(d > dist[type][u]) continue;
            for(edge e : G[type][u]){
                int v = e.first;
                ll w = e.second;
                if(dist[type][v] > d+w){
                    ways[type][0][v] = ways[type][0][u];
                    ways[type][1][v] = ways[type][1][u];
                    dist[type][v] = d+w;
                    Q.push({d+w, v});
                } else if(dist[type][v] == d+w){
                    ways[type][0][v] += ways[type][0][u];
                    ways[type][0][v] %= MOD1;
                    ways[type][1][v] += ways[type][1][u];
                    ways[type][1][v] %= MOD2;
                }
            }
        }
    }

}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b, c; i < M; i++){
        scanf("%d %d %d", &a, &b, &c);
        G[0][a].push_back({b, c});
        G[1][b].push_back({a, c});
    }
    init();
    dijkstra(0, 1);
    dijkstra(1, N);

    ll tot1 = ways[0][0][N], tot2 = ways[0][1][N];
    for(int u = 1; u <= N; u++){
        ll ways1 = ways[0][0][u] * ways[1][0][u] % MOD1;

```

```

ll ways2 = ways[0][1][u] * ways[1][1][u] % MOD2;
if(ways1 == tot1 && ways2 == tot2 && dist[0][u] + dist[1][u] == dist[0][N])
    ans.push_back(u);
}

int K = (int) ans.size();
printf("%d\n", K);
for(int i = 0; i < K; i++)
    printf("%d%c", ans[i], (" \n")[i==K-1]);
}

```

Network Renovation

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, K;
vector leaves, G[maxN];

void dfs(int u = 1, int p = -1){
    if((int) G[u].size() == 1) leaves.push_back(u);
    for(int v : G[u])
        if(v != p)
            dfs(v, u);
}

int main(){
    scanf("%d", &N);
    for(int i = 0, a, b; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs();

    int K = (int) leaves.size();
    printf("%d\n", (K+1)/2);
    for(int i = 0; i < (K+1)/2; i++)
        printf("%d %d\n", leaves[i], leaves[i+K/2]);
}

```

Creating Offices

```

#include

using namespace std;
const int maxN = 2e5+1, logN = 20;
const int INF = 0x3f3f3f3f;

```

```

int N, D, ct[maxN], sz[maxN], best[maxN];
int timer, p[maxN][logN], d[maxN], in[maxN], out[maxN];
bool vis[maxN];
vector G[maxN];

void dfs(int u = 1, int par = 1){
    in[u] = ++timer;
    d[u] = d[par]+1;
    p[u][0] = par;
    for(int i = 1; i < logN; i++)
        p[u][i] = p[p[u][i-1]][i-1];
    for(int v : G[u])
        if(v != par)
            dfs(v, u);
    out[u] = ++timer;
}

bool ancestor(int u, int v){
    return in[u] <= in[v] && out[u] >= out[v];
}

int lca(int u, int v){
    if(ancestor(u, v)) return u;
    if(ancestor(v, u)) return v;
    for(int i = logN-1; i >= 0; i--)
        if(!ancestor(p[u][i], v))
            u = p[u][i];
    return p[u][0];
}

int dist(int u, int v){
    return d[u] + d[v] - 2*d[lca(u, v)];
}

int find_size(int u, int p = -1){
    if(vis[u]) return 0;

    sz[u] = 1;
    for(int v : G[u])
        if(v != p)
            sz[u] += find_size(v, u);

    return sz[u];
}

int find_centroid(int u, int p, int n){
    for(int v : G[u])
        if(v != p)
            if(!vis[v] && sz[v] > n/2)
                return find_centroid(v, u, n);

    return u;
}

void build_centroid_tree(int u = 1, int p = -1){
    find_size(u);

    int c = find_centroid(u, -1, sz[u]);
    vis[c] = true;
    ct[c] = p;

    for(int v : G[c])
        if(!vis[v])
            build_centroid_tree(v, c);
}

void update(int u){
    best[u] = 0;
    int v = u;
    while(ct[v] != -1){

```

```

        v = ct[v];
        best[v] = min(best[v], dist(u, v));
    }
}

int query(int u){
    int ans = best[u];
    int v = u;
    while(ct[v] != -1){
        v = ct[v];
        ans = min(ans, best[v] + dist(u, v));
    }
    return ans;
}

int main(){
    scanf("%d %d", &N, &D);
    for(int i = 0, a, b; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs();
    build_centroid_tree();
    fill(best+1, best+N+1, INF);

    vector order(N);
    for(int i = 0; i < N; i++)
        order[i] = i+1;
    sort(order.begin(), order.end(), [](int a, int b){
        return d[a] == d[b] ? a < b : d[a] > d[b];
    });

    vector ans;
    for(int u : order){
        int dist_to_office = query(u);
        if(dist_to_office >= D){
            ans.push_back(u);
            update(u);
        }
    }

    sort(ans.begin(), ans.end());
    int K = (int) ans.size();
    printf("%d\n", K);
    for(int i = 0; i < K; i++)
        printf("%d%c", ans[i], (" \n")[i==K-1]);
}

```

Counting

Empty String

```
#include

using namespace std;
typedef long long ll;
const int maxN = 505;
const ll MOD = 1e9+7;

int N;
char S[maxN];
ll fac[maxN], inv[maxN], dp[maxN][maxN];

ll fastpow(ll x, ll b){
    ll res = 1;
    while(b){
        if(b&1) res = (res * x) % MOD;
        x = (x * x) % MOD;
        b >>= 1;
    }
    return res;
}

ll choose(int n, int k){
    if(k < 0 || n < k) return 0;
    return fac[n] * inv[k] % MOD * inv[n-k] % MOD;
}

void init(){
    fac[0] = inv[0] = 1;
    for(int i = 1; i < maxN; i++){
        fac[i] = (fac[i-1] * i) % MOD;
        inv[i] = fastpow(fac[i], MOD-2);
    }

    for(int i = 0; i < maxN; i++)
        for(int j = i; j < maxN; j++)
            dp[i][j] = -1;
}

ll solve(int l, int r){
    if((r-l+1)&1) return 0;
    if(l > r) return 1;
    if(dp[l][r] != -1) return dp[l][r];

    ll cnt = 0;
    for(int m = l+1; m <= r; m++){
        if(S[l] == S[m]){
            ll subcases = solve(l+1, m-1) * solve(m+1, r) % MOD;
            ll aftercombine = subcases * choose((r-l+1)/2, (m-l+1)/2) % MOD;
            cnt = (cnt + aftercombine) % MOD;
        }
    }

    return dp[l][r] = cnt;
}

int main(){
    scanf("%s", S);
    N = (int) strlen(S);
    init();
```

```
printf("%lld\n", N&1 ? 0 : solve(0, N-1));
}
```

Permutation Inversions

```
#include

using namespace std;
typedef long long ll;
const int maxN = 501;
const int maxK = maxN*(maxN-1)/2;
const ll MOD = 1e9+7;

int N, K;
ll dp[maxN][maxK];

void init(){
    for(int i = 1; i < maxN; i++){
        int r = i*(i-1)/2;
        dp[i][0] = dp[i][r] = 1;
        for(int j = 1; j <= r/2; j++){
            dp[i][j] = (dp[i-1][j] + dp[i][j-1]) % MOD;
            if(j-i >= 0)
                dp[i][j] = (dp[i][j] - dp[i-1][j-i] + MOD) % MOD;
        }
        for(int j = r/2+1; j < r; j++)
            dp[i][j] = dp[i][r-j];
    }
}

int main(){
    init();
    scanf("%d %d", &N, &K);
    printf("%lld\n", dp[N][K]);
}
```

Counting Bishops

```
#include

using namespace std;
typedef long long ll;
const int maxN = 501;
const ll MOD = 1e9+7;

int N, K;
ll fac[maxN], inv[maxN];
ll T[maxN], B[2*maxN], W[2*maxN];

ll fastpow(ll x, ll b){
    ll res = 1;
    while(b){

```

```

        if(b&1) res = (res * x) % MOD;
        x = (x * x) % MOD;
        b >>= 1;
    }
    return res;
}

ll choose(int n, int k){
    if(k < 0 || n < k) return 0;
    return fac[n] * inv[k] % MOD * inv[n-k] % MOD;
}

ll stirling2(int n, int k){
    ll sum = 0;
    for(int i = 0; i <= k; i++){
        ll a = (i&1 ? -1 : 1);
        ll b = choose(k, i);
        ll c = fastpow(k-i, n);
        ll term = a * b % MOD * c % MOD;
        sum = (sum + term + MOD) % MOD;
    }
    return sum * inv[k] % MOD;
}

void compute_T(int n){
    for(int k = 0; k <= n; k++){
        T[k] = stirling2(n+1, n+1-k);
    }
}

void init_choose(){
    fac[0] = inv[0] = 1;
    for(int i = 1; i < maxN; i++){
        fac[i] = (fac[i-1] * i) % MOD;
        inv[i] = fastpow(fac[i], MOD-2);
    }
}

ll calculate(int n, int k){
    if(n == 1) return 1;
    if(k >= 2*n) return 0;
    memset(B, 0, sizeof(B));
    memset(W, 0, sizeof(W));

    int m = n/2, odd = n&1;
    for(int k = 0; k <= m+odd; k++){
        compute_T(2*m-1-k+odd);
        ll a = choose(m+odd, k);
        ll b = choose(m, k);
        for(int i = 0; i+k < n; i++){
            B[i+k] += (a * T[i]) % MOD;
            B[i+k] %= MOD;
            W[i+k] += (b * T[i]) % MOD;
            W[i+k] %= MOD;
        }
    }

    ll sum = 0;
    for(int b = 0; b <= min(N-1, K); b++){
        int w = K-b;
        sum += (B[b] * W[w]) % MOD;
        sum %= MOD;
    }
    return sum;
}

int main(){
    init_choose();
    scanf("%d %d", &N, &K);

```

```
    printf("%lld\n", calculate(N, K));  
}
```

Counting Sequences

```
#include  
  
using namespace std;  
typedef long long ll;  
const int maxN = 1e6+1;  
const ll MOD = 1e9+7;  
  
int N, K;  
ll fac[maxN], inv[maxN];  
  
ll fastpow(ll x, ll b){  
    ll res = 1;  
    while(b){  
        if(b&1) res = (res * x) % MOD;  
        x = (x * x) % MOD;  
        b >>= 1;  
    }  
    return res;  
}  
  
void init_choose(){  
    fac[0] = inv[0] = 1;  
    for(int i = 1; i < maxN; i++){  
        fac[i] = (fac[i-1] * i) % MOD;  
        inv[i] = fastpow(fac[i], MOD-2);  
    }  
}  
  
ll choose(int n, int k){  
    if(k < 0 || k > n) return 0;  
    return fac[n] * inv[k] % MOD * inv[n-k] % MOD;  
}  
  
ll T(int n, int k){  
    ll sum = 0;  
    for(int j = 0; j <= k; j++){  
        ll a = (j&1 ? -1 : 1);  
        ll b = choose(k, j);  
        ll c = fastpow(k-j, n);  
        ll term = (a * b % MOD * c % MOD + MOD) % MOD;  
        sum = (sum + term) % MOD;  
    }  
    return sum;  
}  
  
int main(){  
    init_choose();  
    scanf("%d %d", &N, &K);  
    printf("%lld\n", T(N, K));  
}
```

Grid Paths II

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e6+1;
const int maxM = 1002;
const ll MOD = 1e9+7;

int N, M, x, y;
ll fact[maxN], inv[maxN], dp[maxM];
struct point {int x, y;} p[maxM];

ll inverse(ll a){
    ll res = 1;
    ll b = MOD-2;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

void init(){
    fact[0] = inv[0] = 1;
    for(int i = 1; i < maxN; i++){
        fact[i] = i * fact[i-1] % MOD;
        inv[i] = inverse(fact[i]);
    }
}

ll choose(int n, int k){
    return fact[n] * inv[n-k] % MOD * inv[k] % MOD;
}

int main(){
    init();
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &x, &y);
        p[i] = {x, y};
    }
    p[M] = {N, N};
    sort(p, p+M+1, [](point a, point b){
        return a.x == b.x ? a.y < b.y : a.x < b.x;
    });

    for(int i = 0; i <= M; i++){
        dp[i] = choose(p[i].x+p[i].y-2, p[i].x-1);
        ll subtract = 0;
        for(int j = 0; j < i; j++){
            if(p[i].x >= p[j].x && p[i].y >= p[j].y){
                int dx = p[i].x-p[j].x, dy = p[i].y-p[j].y;
                subtract += dp[j] * choose(dx+dy, dx);
                subtract %= MOD;
            }
        }
        dp[i] = (dp[i] - subtract + MOD) % MOD;
    }
    printf("%lld\n", dp[M]);
}
```

Counting Permutations

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1000;
const ll MOD = 1e9+7;

int N;
ll dp[maxN+1];

void init(){
    dp[0] = dp[1] = 1;
    dp[2] = dp[3] = 0;
    for(int i = 4; i <= maxN; i++){
        ll a = (i+1) * dp[i-1] % MOD;
        ll b = (i-2) * dp[i-2] % MOD;
        ll c = (i-5) * dp[i-3] % MOD;
        ll d = (i-3) * dp[i-4] % MOD;

        dp[i] = (a-b-c+d);
        while(dp[i] < 0)
            dp[i] += MOD;
        dp[i] %= MOD;
    }
}

int main(){
    init();
    scanf("%d", &N);
    printf("%lld\n", dp[N]);
}
```

Grid Completion

```
#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;
const int maxN = 505;

int N, p[maxN], q[maxN], C[6];
bool inp[maxN], inq[maxN];
char S[maxN];
ll fact[maxN], inv[maxN];

ll inverse(ll x){
    ll res = 1;
    ll b = MOD-2;
    while(b){
        if(b&1) res = (res * x) % MOD;
        x = (x * x) % MOD;
        b >>= 1;
    }
}
```

```

    }
    return res;
}

ll choose(int x, int y){
    return (fact[x] * inv[y] % MOD) * inv[x-y] % MOD;
}

void init(){
    fact[0] = inv[0] = 1;
    for(int i = 1; i <= N; i++){
        fact[i] = (fact[i-1] * i) % MOD;
        inv[i] = (inv[i-1] * inverse(i)) % MOD;
    }
}

ll f(int i, int j, int k){
    ll res = (choose(C[0], i)
        * choose(C[1], j) % MOD
        * choose(C[2], k) % MOD
        * choose(C[3], i) % MOD);

    res = (res * fact[i] % MOD
        * fact[C[4]-i-j] % MOD
        * fact[C[5]-i-k] % MOD);

    if((i+j+k) % 2 == 1)
        res = (MOD - res);

    return res;
}

int main(){
    scanf("%d", &N);
    init();
    for(int i = 0; i < N; i++){
        scanf(" %s", S);
        p[i] = q[i] = -1;
        for(int j = 0; j < N; j++){
            if(S[j] == 'A') { p[i] = j; inp[j] = true; }
            if(S[j] == 'B') { q[i] = j; inq[j] = true; }
        }
    }

    for(int i = 0; i < N; i++){
        if(p[i] == -1 && q[i] == -1) C[0]++;
        if(p[i] == -1 && q[i] != -1 && !inp[q[i]]) C[1]++;
        if(p[i] != -1 && q[i] == -1 && !inq[p[i]]) C[2]++;
    }

    for(int i = 0; i < N; i++){
        if(!inp[i] && !inq[i]) C[3]++;
        if(!inp[i]) C[4]++;
        if(!inq[i]) C[5]++;
    }

    ll ans = 0;
    for(int i = 0; i <= min(C[0], C[3]); i++)
        for(int j = 0; j <= C[1]; j++)
            for(int k = 0; k <= C[2]; k++)
                ans = (ans + f(i, j, k)) % MOD;
    printf("%lld\n", ans);
}

```

Functional Graph Distribution

```
#include

using namespace std;
typedef long long ll;
const int maxN = 5001;
const ll MOD = 1e9+7;

int N;
ll pown[maxN], fac[maxN], inv[maxN], S[maxN][maxN];

ll inverse(ll x){
    ll res = 1;
    ll b = MOD-2;
    while(b){
        if(b&1) res = (res * x) % MOD;
        x = (x * x) % MOD;
        b >>= 1;
    }
    return res;
}

void init_powers(){
    pown[0] = 1;
    for(int i = 1; i < maxN; i++){
        pown[i] = (pown[i-1] * N) % MOD;
    }
}

void init_choose(){
    fac[0] = inv[0] = 1;
    for(int i = 1; i < maxN; i++){
        fac[i] = (fac[i-1] * i) % MOD;
        inv[i] = inverse(fac[i]);
    }
}

void init_stirling(){
    S[1][1] = 1;
    for(int n = 2; n < maxN; n++){
        for(int k = 1; k <= n; k++){
            S[n][k] = (S[n-1][k-1] - (n-1) * S[n-1][k]) % MOD;
        }
    }
}

ll choose(int n, int k){
    if(k < 0 || k > n) return 0;
    return fac[n] * inv[k] % MOD * inv[n-k] % MOD;
}

ll stirling1(int n, int k){
    return abs(S[n][k]);
}

ll T(int n, int k){
    ll sum = 0;
    for(int j = 0; j <= n-1; j++){
        ll a = choose(n-1, j);
        ll b = pown[n-1-j];
        ll c = stirling1(j+1, k);
        sum += a * b % MOD * c % MOD;
        sum %= MOD;
    }
    return sum;
}

int main(){
    scanf("%d", &N);
    init_powers();
```

```
init_choose();  
init_stirling();  
for(int k = 1; k <= N; k++)  
    printf("%lld\n", T(N, k));  
}
```

Additional I

Shortest Subsequence

```
#include

using namespace std;
const int maxN = 1e6+5;

int N, cnt;
char S[maxN], ch[4] = {'A', 'C', 'G', 'T'};
bool has[4];
vector segs;
map mp;

void init(){
    mp['A'] = 0;
    mp['C'] = 1;
    mp['G'] = 2;
    mp['T'] = 3;
}

int main(){
    init();
    scanf(" %s", S);

    N = (int) strlen(S);
    for(int i = 0; i < N; i++){
        int c = mp[S[i]];
        if(!has[c]){
            has[c] = true;
            cnt++;
            if(cnt == 4){
                fill(has, has+4, false);
                segs.push_back(i);
                cnt = 0;
            }
        }
    }

    for(int i : segs)
        printf("%c", S[i]);
    for(int i = 0; i < 4; i++){
        if(!has[i]){
            printf("%c\n", ch[i]);
            return 0;
        }
    }
}
```

Swap Game

```
#include
```

```

using namespace std;

const int N = 9;
const int x[] = {0, 1, 2, 3, 4, 5, 1, 2, 4, 5, 7, 8};
const int y[] = {3, 4, 5, 6, 7, 8, 0, 1, 3, 4, 6, 7};

unordered_map dist;
queue Q;
string perm;

void init(){
    perm = "123456789";

    dist[perm] = 1;
    Q.push(perm);
    while(!Q.empty()){
        perm = Q.front(); Q.pop();
        int d = dist[perm];
        for(int i = 0; i < 12; i++){
            swap(perm[x[i]], perm[y[i]]);
            if(!dist[perm]){
                dist[perm] = d+1;
                Q.push(perm);
            }
            swap(perm[x[i]], perm[y[i]]);
        }
    }
}

int main(){
    init();
    for(int i = 0; i < N; i++)
        scanf("%c", &perm[i]);
    printf("%d\n", dist[perm]-1);
}

```

Multiplication Table

```

#include

using namespace std;
typedef long long ll;

ll N;

ll f(ll x){
    ll cnt = 0;
    for(ll i = 1; i <= N; i++)
        cnt += min(N, x/i);
    return cnt;
}

int main(){
    scanf("%lld", &N);
    ll lo = 0, hi = N*N;
    while(hi-lo > 1){
        ll mid = lo + (hi-lo)/2;
        if(f(mid) < (N*N+1)/2) lo = mid;
        else hi = mid;
    }
}

```

```
printf("%lld\n", lo+1);  
}
```

Bubble Sort Rounds I

```
#include  
  
using namespace std;  
typedef pair pii;  
const int maxN = 2e5+1;  
  
int N, x[maxN];  
vector sorted;  
map inv;  
  
int main(){  
    scanf("%d", &N);  
    for(int i = 0; i < N; i++){  
        scanf("%d", &x[i]);  
        sorted.push_back({x[i], i});  
    }  
    sort(sorted.begin(), sorted.end(), greater());  
  
    int worst_dist = 0;  
    for(int i = 0; i < N; i++){  
        int targ = N-i-1;  
        int start = sorted[i].second;  
        worst_dist = max(worst_dist, start - targ);  
    }  
    printf("%d\n", worst_dist);  
}
```

Advertisement

```
#include  
  
using namespace std;  
typedef long long ll;  
const int maxN = 2e5+5;  
  
int N, l[maxN], r[maxN];  
ll best, a[maxN];  
  
int main(){  
    scanf("%d", &N);  
    for(int i = 1; i <= N; i++){  
        scanf("%lld", &a[i]);  
    }  
  
    for(int i = 1; i <= N; i++){  
        l[i] = i-1;  
        while(a[l[i]] >= a[i])  
            l[i] = l[l[i]];  
    }  
}
```

```

    }
    for(int i = N; i >= 1; i--){
        r[i] = i+1;
        while(a[r[i]] >= a[i])
            r[i] = r[r[i]];
    }

    for(int i = 1; i <= N; i++)
        best = max(best, (r[i]-l[i]-1)*a[i]);
    printf("%lld\n", best);
}

```

Special Substrings

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, M;
char S[maxN];
bool exists[26];
map ch;
map, ll> dp;

bool containsEach(vector V){
    for(int i = 0; i < (int) V.size(); i++){
        if(V[i] <= 0)
            return false;
    }
    return true;
}

int main(){
    scanf("%s", S);
    N = (int) strlen(S);
    for(int i = 0; i < N; i++){
        int c = (int) (S[i] - 'a');
        if(!exists[c]){
            ch[S[i]] = M++;
            exists[c] = true;
        }
    }

    ll ans = 0;
    vector freq(M);
    dp[freq]++;
    for(int i = 0; i < N; i++){
        int c = ch[S[i]];
        freq[c]++;
        if(containsEach(freq))
            for(int j = 0; j < M; j++)
                freq[j]--;
        ans += dp[freq];
        dp[freq]++;
    }
    printf("%lld\n", ans);
}

```

Counting LCM Arrays

```
#include

using namespace std;
typedef long long ll;
typedef pair pii;
const ll MOD = 1e9+7;

struct Matrix2x2 {
    ll a, b, c, d;
    Matrix2x2(ll a, ll b, ll c, ll d) : a(a), b(b), c(c), d(d) {}

    Matrix2x2 operator*(const Matrix2x2& other) const {
        return Matrix2x2(
            (a * other.a + b * other.c) % MOD,
            (a * other.b + b * other.d) % MOD,
            (c * other.a + d * other.c) % MOD,
            (c * other.b + d * other.d) % MOD
        );
    }
};

Matrix2x2 fast_pow(Matrix2x2 base, ll exp){
    Matrix2x2 result(1, 0, 0, 1);
    while(exp > 0){
        if(exp % 2 == 1)
            result = result * base;
        base = base * base;
        exp /= 2;
    }
    return result;
}

vector prime_factor(int k){
    vector factors;
    for(int i = 2; i * i <= k; i++){
        int count = 0;
        while(k % i == 0){
            k /= i;
            count++;
        }
        if(count > 0)
            factors.push_back({i, count});
    }
    if(k > 1)
        factors.push_back({k, 1});
    return factors;
}

ll solve(int n, int k){
    vector factors = prime_factor(k);

    ll ans = 1;
    for(const auto& [p, e] : factors){
        Matrix2x2 base(1, 1, e, 0);
        Matrix2x2 result = fast_pow(base, n+1);
        ans = (ans * result.a) % MOD;
    }
    return ans;
}

int main(){
    int T;
```

```

scanf("%d", &T);
for(int t = 0, n, k; t < T; t++){
    scanf("%d %d", &n, &k);
    printf("%lld\n", solve(n, k));
}
}

```

Square Subsets

```

#include

using namespace std;
const int maxX = 5005;
const int SIZE = 700;
const int MOD = 1e9+7;

int N, ans;
bool p[maxX];
vector primes;
bitset basis[700];

void generate_primes(){
    for(int i = 2; i < maxX; i++) p[i] = true;
    for(int i = 2; i < maxX; i++){
        if(p[i]){
            primes.push_back(i);
            for(int j = 2*i; j < maxX; j += i)
                p[j] = false;
        }
    }
}

bitset prime_factor(int x){
    bitset factors;
    for(int i = 0; i < (int) primes.size() && primes[i] <= x; i++){
        int power = 0;
        while(x % primes[i] == 0){
            x /= primes[i];
            power++;
        }
        factors[i] = (power % 2 == 1);
    }
    return factors;
}

bool insert_vector(bitset mask){
    for(int i = SIZE-1; i >= 0; i--){
        if(mask[i] == 0) continue;
        if(!basis[i].any()){
            basis[i] = mask;
            return true;
        }
        mask ^= basis[i];
    }
    return false;
}

int main(){
    scanf("%d", &N);
    generate_primes();

```

```

int ans = 1;
for(int i = 0, x; i < N; i++){
    scanf("%d", &x);
    bitset factors = prime_factor(x);
    if(!insert_vector(factors))
        ans = (ans * 2) % MOD;
}
printf("%d\n", ans);
}

```

Stack Weights

```

#include

using namespace std;
const int maxN = 2e5+1, SIZE = 4*maxN;
const int INF = 0x3f3f3f3f;

int N, lo[SIZE], hi[SIZE], delta[SIZE], mx[SIZE], mn[SIZE];

void push(int i){
    if(delta[i] != 0){
        delta[2*i] += delta[i];
        delta[2*i+1] += delta[i];
        delta[i] = 0;
    }
}

void pull(int i){
    mn[i] = min(mn[2*i]+delta[2*i], mn[2*i+1]+delta[2*i+1]);
    mx[i] = max(mx[2*i]+delta[2*i], mx[2*i+1]+delta[2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r) return;
    int m = l+(r-l)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void increment(int i, int l, int r, int v){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        delta[i] += v; return;
    }

    push(i);
    increment(2*i, l, r, v);
    increment(2*i+1, l, r, v);
    pull(i);
}

int minimum(int i, int l, int r){
    if(l > hi[i] || r < lo[i]) return INF;
    if(l <= lo[i] && hi[i] <= r) return mn[i]+delta[i];

    push(i);
    int lmin = minimum(2*i, l, r);
    int rmin = minimum(2*i+1, l, r);
}

```

```

    pull(i);

    return min(lmin, rmin);
}

int maximum(int i, int l, int r){
    if(l > hi[i] || r < lo[i])    return -INF;
    if(l <= lo[i] && hi[i] <= r)    return mx[i]+delta[i];

    push(i);
    int lmax = maximum(2*i, l, r);
    int rmax = maximum(2*i+1, l, r);
    pull(i);

    return max(lmax, rmax);
}

void place_coin(int c, int s){
    int v = (s == 1 ? -1 : 1);
    increment(1, 1, c, v);
}

char query_stacks(){
    int totmin = minimum(1, 1, N);
    int totmax = maximum(1, 1, N);
    if(totmin >= 0 && totmax <= 0)    return '?';
    else if(totmin >= 0)            return '<';
    else if(totmax <= 0)            return '>';
    else                            return '?';
}

int main(){
    scanf("%d", &N);
    init(1, 1, N);
    for(int i = 0, c, s; i < N; i++){
        scanf("%d %d", &c, &s);
        place_coin(c, s);
        printf("%c\n", query_stacks());
    }
}

```

Pyramid Array

```

#include

using namespace std;
typedef pair pii;
const int maxN = 2e5+1;

int N, ds[maxN];
pii X[maxN];

void update(int idx){
    for(int i = idx; i <= N; i += -i&i) ds[i]++;
}

int query(int idx){
    int cnt = 0;
    for(int i = idx; i; i -= -i&i)
        cnt += ds[i];
    return cnt;
}

```

```

}

int main(){
    scanf("%d", &N);
    for(int i = 0, x; i < N; i++){
        scanf("%d", &x);
        X[i] = {x, i+1};
    }
    sort(X, X+N, [](pii a, pii b){
        return a.first > b.first;
    });

    long long ans = 0;
    for(int k = 0; k < N; k++){
        int idx = X[k].second;
        int l = query(idx);
        ans += min(l, k-l);
        update(idx);
    }
    printf("%lld\n", ans);
}

```

Bit Inversions

```

#include

using namespace std;
const int maxN = 2e5+5;
const int SIZE = 4*maxN;

int N, M, mp[maxN], lo[SIZE], hi[SIZE], dp[3][SIZE];
char S[maxN];

int len(int i){
    return hi[i]-lo[i]+1;
}

int combine(int a, int b){
    return ((a<0)^(b<0)) ? a : a+b;
}

bool allsame(int i){
    return abs(dp[2][i]) == len(i);
}

void pull(int i){
    dp[0][i] = (allsame(2*i) ? combine(dp[0][2*i], dp[0][2*i+1]) : dp[0][2*i]);
    dp[1][i] = (allsame(2*i+1) ? combine(dp[1][2*i+1], dp[1][2*i]) : dp[1][2*i+1]);
    dp[2][i] = max(abs(combine(dp[1][2*i], dp[0][2*i+1])), max(dp[2][2*i], dp[2][2*i+1]));
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        mp[l] = i;
        dp[0][i] = dp[1][i] = (S[l-1] == '0' ? -1 : 1);
        dp[2][i] = abs(dp[0][i]);
        return;
    }
    int m = (l+r)/2;
    init(2*i, l, m);

```

```

    init(2*i+1, m+1, r);
    pull(i);
}

void update(int idx){
    int i = mp[idx];
    dp[0][i] *= -1;
    dp[1][i] = dp[0][i];

    i >>= 1;
    while(i){
        pull(i);
        i >>= 1;
    }
}

int query(){
    return dp[2][1];
}

int main(){
    scanf("%s %d", S, &M);
    N = (int) strlen(S);
    init(1, 1, N);
    for(int i = 0, x; i < M; i++){
        scanf("%d", &x);
        update(x);
        printf("%d%c", query(), (" \n")[i==M-1]);
    }
}

```

Writing Numbers

```

#include

using namespace std;
typedef long long ll;

ll p[19];

void init(){
    p[0] = 1;
    for(int i = 1; i <= 18; i++)
        p[i] = p[i-1] * 10;
}

ll check(ll N){
    ll cnt = 0, l = N, r = 0;
    for(int i = 0; l; i++){
        int d = l % 10;
        l /= 10;
        cnt += (d == 1 ? l*p[i]+r+1 : (l+(d!=0))*p[i]);
        r += p[i] * d;
    }
    return cnt;
}

int main(){
    ll K;
    scanf("%lld", &K);
    ll lo = 1, hi = 1e18;

```

```

init();
while(hi-lo > 1){
    ll mid = lo+(hi-lo)/2;
    if(check(mid) <= K) lo = mid;
    else hi = mid;
}
printf("%lld\n", lo);
}

```

Maximum Building I

```

#include

using namespace std;
const int maxN = 1002;

int N, M, best, h[maxN], l[maxN], r[maxN];
char S[maxN][maxN];

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++)
        scanf(" %s", S[i]);

    h[0] = h[M+1] = -1;
    for(int i = 1; i <= N; i++){
        for(int j = 1; j <= M; j++){
            h[j] = (S[i-1][j-1] == '*' ? 0 : h[j]+1);

            int smaller;
            for(int j = 1; j <= M; j++){
                smaller = j-1;
                while(h[smaller] >= h[j])
                    smaller = l[smaller];
                l[j] = smaller;
            }
            for(int j = M; j >= 1; j--){
                smaller = j+1;
                while(h[smaller] >= h[j])
                    smaller = r[smaller];
                r[j] = smaller;
            }

            for(int j = 1; j <= M; j++){
                int base = r[j]-l[j]-1;
                best = max(best, base * h[j]);
            }
        }

    printf("%d\n", best);
}

```

Sorting Methods

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+5;

int N, x[maxN], ds[maxN];
ll bit[maxN], ans[4];
set S;
set::iterator it;

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

void update(int idx, int val){
    for(int i = idx; i < maxN; i += -i&i)
        bit[i] += val;
}

ll query(int idx){
    int sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += bit[i];
    return sum;
}

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x[i]);

        int K = N;
        ans[3] = N;
        fill(ds+1, ds+N+1, -1);
        for(int i = 1; i <= N; i++){
            ans[0] += (i-query(x[i])-1);
            update(x[i], 1);

            if(!merge(i, x[i]))
                ans[1] += (-ds[find(i)]-1);

            S.insert(x[i]);
            it = S.lower_bound(x[i]);
            if(++it != S.end())
                S.erase(it);

            if(x[N-i+1] == K){
                K--;
                ans[3]--;
            }
        }
        ans[2] = N - (int) S.size();

        for(int i = 0; i < 4; i++)
            printf("%lld%c", ans[i], (" \n")[i==3]);
    }
}

```

Cyclic Array

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, last[maxN];
ll K, x[2*maxN];

int main(){
    scanf("%d %lld", &N, &K);
    for(int i = 0; i < N; i++){
        scanf("%lld", &x[i]);
        x[i+N] = x[i];
    }

    int r = 0;
    ll sum = 0;
    for(int l = 0; l < N; l++){
        while(r < l+N && sum + x[r] <= K){
            sum += x[r];
            r++;
        }
        last[l] = r % N;
        sum -= x[l];
    }

    int opt = 0;
    for(int i = 0; i < N; i++)
        opt = last[opt];

    int cnt = 1;
    sum = x[opt];
    for(int i = opt+1; i < N+opt; i++){
        if(sum + x[i] <= K) sum += x[i];
        else {
            sum = x[i];
            cnt++;
        }
    }

    printf("%d\n", cnt);
}
```

List of Sums

```
#include

using namespace std;
typedef long long ll;
const int maxN = 101, maxM = 4951;

int N, M;
```

```
multiset Bvals;
ll A[maxN], B[maxM];

bool check(ll a0){
    A[0] = a0;
    Bvals.clear();
    for(int i = 0; i < M; i++){
        Bvals.insert(B[i]);

        for(int i = 1; i < N; i++){
            for(int j = 0; j < i-1; j++){
                if(Bvals.find(A[j]+A[i-1]) == Bvals.end())
                    return false;
                Bvals.erase(Bvals.lower_bound(A[j]+A[i-1]));
            }
            A[i] = *Bvals.begin() - A[0];
        }

        // Confirm that the last element, A[N-1], works. We desire
        // the sum with the other A[i] to produce the remaining Bvals
        for(int j = 0; j < N-1; j++){
            if(Bvals.find(A[j]+A[N-1]) == Bvals.end())
                return false;
            Bvals.erase(Bvals.lower_bound(A[j]+A[N-1]));
        }

        return true;
    }
}

int main(){
    scanf("%d", &N);
    M = N*(N-1)/2;
    for(int i = 0; i < M; i++){
        scanf("%lld", &B[i]);
    }
    sort(B, B+M);

    for(int i = 1; i < M; i++){
        ll a0 = (B[0] + B[1] - B[i])/2;
        if(1 <= a0 && a0 <= B[0]/2 && check(a0)){
            for(int j = 0; j < N; j++){
                printf("%lld%c", A[j], (" \n")[j==N-1]);
            }
            return 0;
        }
    }

    printf("IMPOSSIBLE\n");
}
```

Additional II

Increasing Array II

```
#include

using namespace std;
typedef long long ll;

int N, x;
priority_queue Q;

int main(){
    scanf("%d", &N);
    ll ans = 0;
    for(int i = 0; i < N; i++){
        scanf("%d", &x);
        Q.push(x);
        ans += Q.top() - x;
        Q.pop();
        Q.push(x);
    }
    printf("%lld\n", ans);
}
```

Food Division

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, A[maxN];

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++) scanf("%d", &A[i]);
    for(int i = 1, b; i <= N; i++){
        scanf("%d", &b);
        A[i] += (A[i-1] - b);
    }
    sort(A+1, A+N+1);

    ll ans = 0;
    for(int i = 1; i <= N; i++)
        ans += abs(A[i] - A[(N+1)/2]);
    printf("%lld\n", ans);
}
```

Swap Round Sorting

```
#include

using namespace std;
typedef pair pii;
const int maxN = 2e5+1;

bool vis[maxN];
int N, a[maxN];
vector cycle;
vector ans;

bool zero_rounds(){
    for(int i = 1; i <= N; i++){
        if(a[i] != i)
            return false;

        printf("0\n");
        return true;
    }

    bool one_round(bool second = false){
        for(int i = 1; i <= N; i++){
            if(a[i] != i){
                if(i != a[a[i]]) { ans.clear(); return false; }
                else if(i < a[i]) ans.push_back({i, a[i]});
            }
        }

        if(!second) printf("1\n");
        printf("%d\n", (int) ans.size());
        for(pii p : ans){
            printf("%d %d\n", p.first, p.second);
            swap(a[p.first], a[p.second]);
        }
        return true;
    }

    void dfs(int u){
        vis[u] = true;
        cycle.push_back(u);
        if(!vis[a[u]]) dfs(a[u]);
    }

    int main(){
        scanf("%d", &N);
        for(int i = 1; i <= N; i++){
            scanf("%d", &a[i]);
        }

        if(!zero_rounds() && !one_round()){
            printf("2\n");

            for(int i = 1; i <= N; i++){
                if(!vis[i]){
                    dfs(i);
                    int K = (int) cycle.size();
                    for(int u = 0; u < K/2; u++){
                        ans.push_back({cycle[u], cycle[K-u-1]});
                        swap(a[cycle[u]], a[cycle[K-u-1]]);
                    }
                    cycle.clear();
                }
            }

            printf("%d\n", (int) ans.size());
            for(pii p : ans)
                printf("%d %d\n", p.first, p.second);
        }
    }
}
```

```

        ans.clear();
        one_round(true);
    }
}

```

School Excursion

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, a, b, ds[maxN];
multiset sizes;
bitset dp;

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d", &N, &M);
    fill(ds+1, ds+N+1, -1);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        merge(a, b);
    }

    for(int i = 1; i <= N; i++)
        if(find(i) == i)
            sizes.insert(-ds[i]);

    dp[0] = 1;
    for(int sz : sizes)
        dp |= (dp <

```

Coin Grid

```

#include

```

```

using namespace std;
typedef pair pii;
const int maxN = 105;

char S[maxN];
bool used[maxN], inZ[2][maxN];
int N, cnt, mt[maxN];
vector G[maxN];

bool kuhns(int u){
    if(used[u]) return false;

    used[u] = true;
    for(int v : G[u]){
        if(!mt[v] || kuhns(mt[v])){
            mt[v] = u;
            return true;
        }
    }

    return false;
}

void dfs(int t, int u){
    inZ[t][u] = true;
    if(t == 0 && !inZ[1][mt[u]]){
        dfs(1, mt[u]);
    } else if(t == 1){
        for(int v : G[u])
            if(!inZ[0][v])
                dfs(0, v);
    }
}

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf(" %s", S);
        for(int j = 1; j <= N; j++){
            if(S[j-1] == 'o')
                G[j].push_back(i);
        }

        for(int i = 1; i <= N; i++){
            kuhns(i);
            fill(used+1, used+N+1, false);
        }

        cnt = 0;
        for(int i = 1; i <= N; i++){
            if(mt[i]){
                cnt++;
                used[mt[i]] = true;
            }
        }

        for(int i = 1; i <= N; i++)
            if(!used[i])
                dfs(1, i);

        printf("%d\n", cnt);
        for(int i = 1; i <= N; i++)
            if(inZ[0][i])
                printf("1 %d\n", i);
        for(int i = 1; i <= N; i++)
            if(!inZ[1][i])
                printf("2 %d\n", i);
    }
}

```

Programmers and Artists

```
#include

using namespace std;
typedef long long ll;
typedef pair pii;
const int maxN = 2e5+5;

int N, A, B;
pii cand[maxN];

multiset initial_artist_set(){
    vector all;
    for(int i = A; i < N; i++){
        all.push_back(cand[i].second);
    }
    sort(all.begin(), all.end());

    reverse(all.begin(), all.end());
    multiset best;
    for(int i = 0; i < B; i++){
        best.insert(all[i]);
    }
    return best;
}

int main(){
    scanf("%d %d %d", &A, &B, &N);
    for(int i = 0; i < N; i++){
        scanf("%d %d", &x, &y);
        cand[i] = {x, y};
    }
    sort(cand, cand+N, [](const pii a, const pii b){
        return a.first == b.first ? a.second < b.second : a.first > b.first;
    });

    ll pref = 0;
    priority_queue prefix_deltas;
    for(int i = 0; i < A; i++){
        pref += cand[i].first;
        prefix_deltas.push(cand[i].second - cand[i].first);
    }

    multiset remaining_art = initial_artist_set();
    ll suf = accumulate(remaining_art.begin(), remaining_art.end(), 0LL);

    ll ans = pref + suf;
    for(int i = A; i < A+B; i++){
        const int x = cand[i].first;
        const int y = cand[i].second;
        pref += x;

        prefix_deltas.push(y - x);
        pref += prefix_deltas.top();
        prefix_deltas.pop();

        auto worst_artist = remaining_art.lower_bound(y);
        int art_value = *worst_artist;
        remaining_art.erase(worst_artist);
        suf -= art_value;

        ans = max(ans, pref + suf);
    }
}
```

```
printf("%lld\n", ans);  
}
```

Coin Arrangement

```
#include  
  
using namespace std;  
typedef long long ll;  
const int maxN = 1e5;  
  
int N, a[2][maxN];  
  
int main(){  
    scanf("%d", &N);  
    for(int r = 0; r < 2; r++){  
        for(int i = 0; i < N; i++){  
            scanf("%d", &a[r][i]);  
            a[r][i]--;  
        }  
    }  
  
    ll ans = 0;  
    int top = 0, bot = 0;  
    for(int i = 0; i < N; i++){  
        top += a[0][i];  
        bot += a[1][i];  
  
        if((top < 0 && bot > 0) || (top > 0 && bot < 0)){  
            if(abs(top) < abs(bot)){  
                ans += abs(top);  
                bot += top;  
                top = 0;  
            } else {  
                ans += abs(bot);  
                top += bot;  
                bot = 0;  
            }  
        }  
  
        ans += abs(top + bot);  
    }  
    printf("%lld\n", ans);  
}
```

Grid Puzzle I

```
#include  
  
using namespace std;  
typedef pair pii;  
const int maxN = 105, maxM = 2605;
```

```

const int INF = 0x3f3f3f3f;

int N, rowTot, colTot, edgeID, p[maxN], cap[maxN][maxN];
bool vis[maxM];
vector path, F[maxN];
vector G[maxN];

int bfs(int s = 0, int t = 2*N+1){
    fill(p, p+2*N+2, -1);
    p[s] = -2;

    queue Q;
    Q.push({s, INF});
    while(!Q.empty()){
        int u = Q.front().first;
        int f = Q.front().second;
        Q.pop();

        for(int v : F[u]){
            if(p[v] == -1 && cap[u][v]){
                p[v] = u;
                int aug = min(f, cap[u][v]);
                if(v == t) return aug;
                Q.push({v, aug});
            }
        }
    }

    return 0;
}

void dfs(int u = 0){
    path.push_back(u);
    if(u == N) return;
    for(pii e : G[u]){
        int v = e.first;
        int id = e.second;
        if(cap[u][v] == 0 && !vis[id]){
            vis[id] = true;
            dfs(v);
            return;
        }
    }
}

int maxflow(int s = 0, int t = 2*N+1){
    int flow = 0, aug = 0;
    while(aug = bfs()){
        flow += aug;
        int u = t;
        while(u != s){
            int v = p[u];
            cap[v][u] -= aug;
            cap[u][v] += aug;
            u = v;
        }
    }
    return flow;
}

int main(){
    scanf("%d", &N);
    for(int i = 1, c; i <= N; i++){
        scanf("%d", &c);
        G[0].push_back({i, ++edgeID});
        F[0].push_back(i);
        F[i].push_back(0);
        cap[0][i] += c;
        rowTot += c;
    }
}

```

```

    for(int i = N+1, c; i <= 2*N; i++){
        scanf("%d", &c);
        G[i].push_back({2*N+1, ++edgeID});
        F[i].push_back(2*N+1);
        F[2*N+1].push_back(i);
        cap[i][2*N+1] += c;
        colTot += c;
    }
    for(int i = 1; i <= N; i++){
        for(int j = N+1; j <= 2*N; j++){
            G[i].push_back({j, ++edgeID});
            F[i].push_back(j);
            F[j].push_back(i);
            cap[i][j]++;
        }
    }

    int K = maxflow();
    if(rowTot != colTot || K != rowTot){
        printf("-1\n");
        return 0;
    }

    for(int i = 1; i <= N; i++){
        for(int j = 1; j <= N; j++)
            printf("%c", (cap[i][j+N] ? '.' : 'X'));
        printf("\n");
    }
}

```

Grid Puzzle II

```

#include

using namespace std;
const int maxN = 105, maxM = 2605;
const int INF = 0x3f3f3f3f;

int N, K, rowtot, coltot, edgecnt, p[maxN], d[maxN];
bool inq[maxN], vis[maxM];
vector path, G[maxN];

struct Edge {
    int u, v, r, c;
} edges[maxM], redges[maxM];

void read_input(){
    scanf("%d", &N);
    for(int v = 1, cap; v <= N; v++){
        scanf("%d", &cap);
        rowtot += cap;
        edgecnt++;
        G[0].push_back(edgecnt);
        G[v].push_back(-edgecnt);
        edges[edgecnt] = {0, v, cap, 0};
        redges[edgecnt] = {v, 0, 0, 0};
    }
    for(int u = N+1, cap; u <= 2*N; u++){
        scanf("%d", &cap);
        coltot += cap;
        edgecnt++;
    }
}

```

```

G[u].push_back(edgecnt);
G[2*N+1].push_back(-edgecnt);
edges[edgecnt] = {u, 2*N+1, cap, 0};
redges[edgecnt] = {2*N+1, u, 0, 0};
}
for(int u = 1; u <= N; u++){
    for(int v = N+1, cost; v <= 2*N; v++){
        scanf("%d", &cost);
        edgecnt++;
        G[u].push_back(edgecnt);
        G[v].push_back(-edgecnt);
        edges[edgecnt] = {u, v, 1, -cost};
        redges[edgecnt] = {v, u, 0, cost};
    }
}
}

void bellman_ford(int s = 0){
    fill(inq, inq+2*N+2, false);
    fill(d, d+2*N+2, INF);
    fill(p, p+2*N+2, 0);

    queue Q;
    Q.push(s);
    d[s] = 0;
    inq[s] = true;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        inq[u] = false;

        for(int i : G[u]){
            Edge e = (i < 0 ? redges[-i] : edges[i]);
            if(e.r > 0 && d[e.v] > d[u] + e.c){
                d[e.v] = d[u] + e.c;
                p[e.v] = i;
                if(!inq[e.v]){
                    inq[e.v] = true;
                    Q.push(e.v);
                }
            }
        }
    }
}

int minimum_cost_flow(int s = 0, int t = 2*N+1){
    int flow = 0, cost = 0;
    while(flow < K){
        bellman_ford(s);
        if(d[t] == INF) break;

        int aug = K-flow;
        int u = t;
        while(u != s){
            Edge e = (p[u] < 0 ? redges[-p[u]] : edges[p[u]]);
            aug = min(aug, e.r);
            u = e.u;
        }

        flow += aug;
        cost += aug * d[t];
        u = t;
        while(u != s){
            if(p[u] < 0){
                redges[-p[u]].r -= aug;
                edges[-p[u]].r += aug;
            } else {
                redges[p[u]].r += aug;
                edges[p[u]].r -= aug;
            }
            u = (p[u] < 0 ? redges[-p[u]].u : edges[p[u]].u);
        }
    }
}

```

```

    }
}
return (flow < K ? INF : cost);
}

int main(){
    read_input();

    K = rowtot;
    int max_reward = -minimum_cost_flow();
    if(rowtot != coltot || max_reward == -INF){
        printf("-1\n");
        return 0;
    }

    printf("%d\n", max_reward);
    for(Edge e : edges){
        if(e.u != 0 && e.v != 2*N+1){
            printf("%c", (e.r == 0 ? 'X' : '.'));
            if(e.v == 2*N) printf("\n");
        }
    }
}

```

Bit Substrings

```

#include

using namespace std;
typedef long long ll;
typedef double ld;
typedef complex cd;
const int maxN = 2e5+5;
const ld PI = acos(-1);

ll ans[maxN];
char S[maxN];

void fft(vector &a, bool inv){
    int N = (int) a.size();

    for(int i = 1, j = 0; i < N; i++){
        int bit = N>>1;
        for(; j&bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if(i < j)
            swap(a[i], a[j]);
    }

    for(int len = 2; len <= N; len <= 1){
        ld theta = 2*PI / len * (inv ? -1 : 1);
        cd wlen(cos(theta), sin(theta));
        for(int i = 0; i < N; i += len){
            cd w(1);
            for(int j = 0; j < len / 2; j++){
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }
}

```

```

    }
}

if(inv)
    for(cd &z : a)
        z /= N;
}

int next_two_pow(int x){
    return 1<<(32 - __builtin_clz(x));
}

void solve(int l, int r){
    if(l+1 == r){
        ans[(int) (S[l] - '0')]++;
        return;
    }

    int m = (l+r)/2;
    int sz = next_two_pow(r-l);
    vector A(sz), B(sz);
    for(int i = m-1, cnt = 0; i >= l; i--){
        if(S[i] == '1') cnt++;
        A[cnt] += 1;
    }
    for(int i = m, cnt = 0; i < r; i++){
        if(S[i] == '1') cnt++;
        B[cnt] += 1;
    }

    fft(A, false);
    fft(B, false);
    for(int i = 0; i < sz; i++)
        A[i] *= B[i];
    fft(A, true);

    for(int i = 0; i < sz && i < maxN; i++)
        ans[i] += llround(A[i].real());

    solve(l, m);
    solve(m, r);
}

int main(){
    scanf("%s", S);
    int N = (int) strlen(S);
    solve(0, N);
    for(int i = 0; i <= N; i++)
        printf("%lld%c", ans[i], (" \n")[i==N]);
}

```

Reversal Sorting

```

#include

using namespace std;
const int maxN = 2e5+5;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution dist(1, (int) 2e9);

```

```

struct Node {
    bool rev;
    int value, prior, sz;
    Node *p, *l, *r;

    Node(){}
    Node(int val, int pri){
        prior = pri;
        value = val;
        sz = 1;
        p = l = r = nullptr;
    }
};

int N, X[maxN];
Node *root, *ndptr[maxN];

int sz(Node *t){    return t ? t->sz : 0;    }
int val(Node *t){   return t ? t->value : -1; }

void flip(Node *t){
    if(!t) return;
    t->rev ^= true;
}

void pull(Node *t){
    if(!t) return;
    if(t->l) t->l->p = t;
    if(t->r) t->r->p = t;
    t->sz = sz(t->l) + sz(t->r) + 1;
}

void push(Node *t){
    if(!t) return;
    if(t->rev){
        swap(t->l, t->r);
        flip(t->l); flip(t->r);
        t->rev = false;
    }
}

Node* merge(Node *x, Node *y){
    if(!x || !y) return x ? x : y;

    push(x); push(y);
    if(x->prior < y->prior){
        x->r = merge(x->r, y);
        pull(x);
        return x;
    } else {
        y->l = merge(x, y->l);
        pull(y);
        return y;
    }
}

pair split(Node *x, int k){
    if(!x) return {nullptr, nullptr};

    pair y = {nullptr, nullptr};
    push(x);
    if(k <= sz(x->l)){
        y = split(x->l, k);
        x->l = y.second;
        pull(x);
        y.second = x;
    } else {
        y = split(x->r, k-sz(x->l)-1);
        x->r = y.first;
    }
}

```

```

        pull(x);
        y.first = x;
    }
    return y;
}

void heapify(Node *t){
    if(!t) return;

    Node *mx = t;
    if(t->l && t->l->prior > mx->prior) mx = t->l;
    if(t->r && t->r->prior > mx->prior) mx = t->r;
    if(mx != t){
        swap(t->prior, mx->prior);
        heapify(mx);
    }
}

Node* build(int x, int k){
    if(k == 0) return nullptr;

    int mid = k/2;
    Node *t = new Node(X[x+mid], dist(rng));
    ndptr[X[x+mid]] = t;
    t->l = build(x, mid);
    t->r = build(x+mid+1, k-mid-1);
    heapify(t);
    pull(t);
    return t;
}

void reverse(int x, int k){
    pair y, z;
    y = split(root, x-1);
    z = split(y.second, k);
    flip(z.first);
    y.second = merge(z.first, z.second);
    root = merge(y.first, y.second);
    root->p = nullptr;
}

int orderOf(int v){
    Node* t = ndptr[v];

    vector walk;
    while(t){
        walk.push_back(t);
        t = t->p;
    }
    reverse(walk.begin(), walk.end());
    for(Node* nd : walk){
        push(nd);
        pull(nd);
    }

    t = ndptr[v];
    int idx = sz(t->l);
    while(t){
        if(t->p && t == t->p->r)
            idx += sz(t->p->l) + 1;
        t = t->p;
    }
    return idx;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%d", &X[i]);
}

```

```

root = build(0, N);

printf("%d\n", N);
for(int l = 1; l <= N; l++){
    int r = orderOf(l)+1;
    reverse(l, r-1+1);
    printf("%d %d\n", l, r);
}
}

```

Book Shop II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 100, maxK = 1000, maxX = 1e5+1;

int N, X, h[maxN], s[maxN], k[maxN];
ll dp[maxX];

int main(){
    scanf("%d %d", &N, &X);
    for(int i = 0; i < N; i++) scanf("%d", &h[i]);
    for(int i = 0; i < N; i++) scanf("%d", &s[i]);
    for(int i = 0; i < N; i++) scanf("%d", &k[i]);

    fill(dp+1, dp+X+1, -1);
    for(int i = 0; i < N; i++){
        for(int b = 1; k[i] > 0; b++){
            int amnt = min(b, k[i]);
            k[i] -= b;
            int price = amnt * h[i];
            int pages = amnt * s[i];
            for(int j = X; j >= price; j--)
                if(dp[j-price] != -1)
                    dp[j] = max(dp[j], dp[j-price] + pages);
        }
    }

    for(int i = 1; i <= X; i++)
        dp[i] = max(dp[i], dp[i-1]);
    printf("%lld\n", dp[X]);
}

```

Mex Grid Queries

```

#include

using namespace std;

int x, y;

```

```
int main(){
    scanf("%d %d", &x, &y);
    printf("%d\n", (x-1)^(y-1));
}
```

Stick Divisions

```
#include

using namespace std;
typedef long long ll;

int N, x, d;
ll sum;
priority_queue, greater> sticks;

int main(){
    scanf("%d %d", &x, &N);
    for(int i = 0; i < N; i++){
        scanf("%d", &d);
        sticks.push(d);
    }
    while(sticks.size() > 1){
        int a = sticks.top(); sticks.pop();
        int b = sticks.top(); sticks.pop();
        sticks.push(a+b);
        sum += a + b;
    }
    printf("%lld\n", sum);
}
```

Coding Company

```
#include

using namespace std;
typedef long long ll;
const int maxN = 101, maxX = 5e3+1;
const ll MOD = 1e9+7;

int N, X, t[maxN];
ll dp[maxN][maxN][maxX];

int main(){
    scanf("%d %d", &N, &X);
    for(int i = 1; i <= N; i++)
        scanf("%d", &t[i]);
    sort(t+1, t+N+1);
    t[0] = t[1];

    dp[0][0][0] = 1;
```

```
for(int i = 1; i <= N; i++){
    for(int j = N; j >= 0; j--){
        for(int k = X; k >= 0; k--){
            ll cnt = dp[i-1][j][k];
            int newk = k + j * (t[i]-t[i-1]);
            if(newk > X) continue;

            dp[i][j][newk] = (dp[i][j][newk] + (j+1) * cnt) % MOD;
            if(j != N) dp[i][j+1][newk] = (dp[i][j+1][newk] + cnt) % MOD;
            if(j != 0) dp[i][j-1][newk] = (dp[i][j-1][newk] + j * cnt) % MOD;
        }
    }
}

ll tot = 0;
for(int i = 0; i <= X; i++)
    tot = (tot + dp[N][0][i]) % MOD;
printf("%lld\n", tot);
}
```
